MeetingNote Pro

화면 6종 · 상태 63종 · 오류 케이스 포함
도형 와이어프레임으로 그린 설계 화면

화면 뼈대는 도형으로, 상태마다 달라지는 지점을 표시. 확정 디자인은 publish 폴더

A-01. 사용자 여정 - 전체 흐름

신규 합류자 기준 6단계. 페르소나마다 진입점이 다름

1 **가입** `login.html` > 2 **팀 합류** `team.html` > 3 **녹취 업로드** `meetings.html` > 4 **요약·결정사항·할 일** `detail.html` > 5 **담당자 배정** `detail.html` > 6 **칸반 이동** `todos.html`

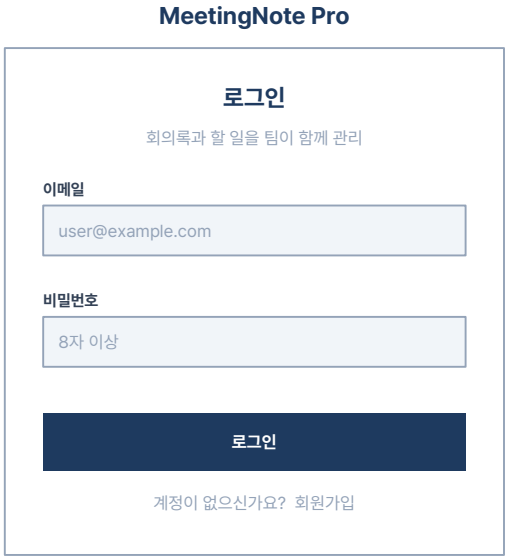
페르소나	첫 화면	가장 오래 머무는 화면	핵심 행동
팀 리더	`meetings.html`	`detail.html`	요약 · 결정사항 · 할 일 확인 후 배정
팀원	`todos.html`	`todos.html`	칸반에서 카드를 끌어 완료로
신규 합류자	`team.html`	`meetings.html`	검색으로 지난 결정 찾기

읽는 법: 아래 장부터 화면마다 상태를 하나씩. 번호는 B · C · D · E · F · J 순서, 공통 규격 N · G · I · H 는 맨 뒤

B. 로그인 / 회원가입

첫 화면. 인증 실패 시 뒤의 모든 화면이 차단

/login.html



상태 11종

01	로그인	
02	로그인 실패	401 INVALID_CREDENTIALS
03	세션 만료	401 TOKEN_EXPIRED
04	로그인 성공	200 {token, user}
05	회원가입 초기	201 {token, user}
06	입력 중	
07	처리 중	요청 대기
08	에러 - 형식	400 EMAIL_INVALID
09	에러 - 중복	409 EMAIL_DUPLICATED
10	에러 - 약한 비밀번호	400 PASSWORD_TOO_WEAK
11	에러 - 초대코드	404 INVITE_NOT_FOUND

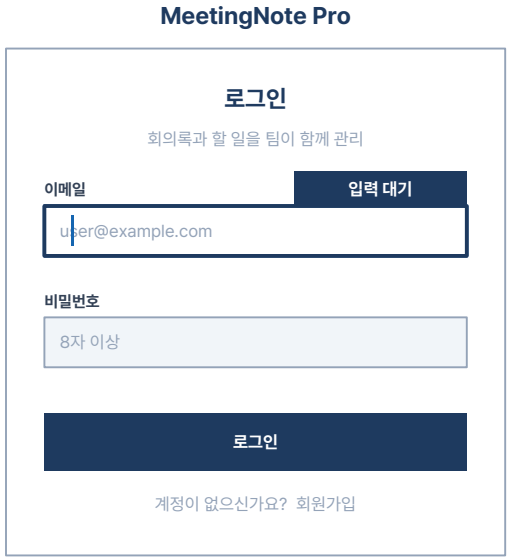
쓰이는 엔드포인트 `POST /api/auth/signup` · `POST /api/auth/login` · `GET /api/teams` · `POST /api/teams/join`

상태 11종: 정상 흐름뿐 아니라 빈 상태 · 오류 · 처리 중까지 화면으로 고정

B-01. 로그인 / 회원가입 - 로그인

초기 상태. 이메일과 비밀번호 둘뿐

/login.html



상태 1 / 11

응답

–

구현 대상 파일

`login.html`

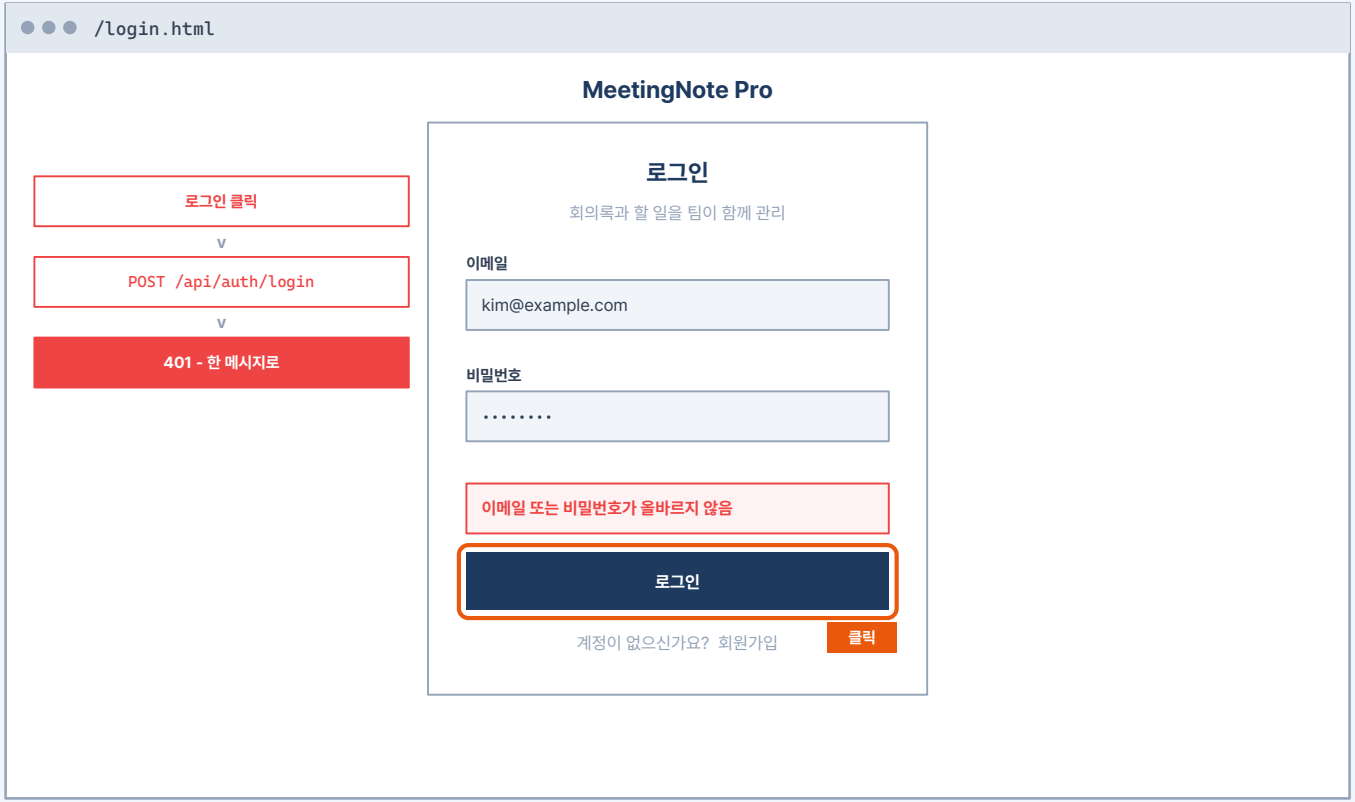
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join
```

B-02. 로그인 / 회원가입 - 로그인 실패

이메일 존재 여부를 노출하지 않고 한 메시지로 통일



상태 2 / 11

응답

`401 INVALID_CREDENTIALS`

구현 대상 파일

`login.html`

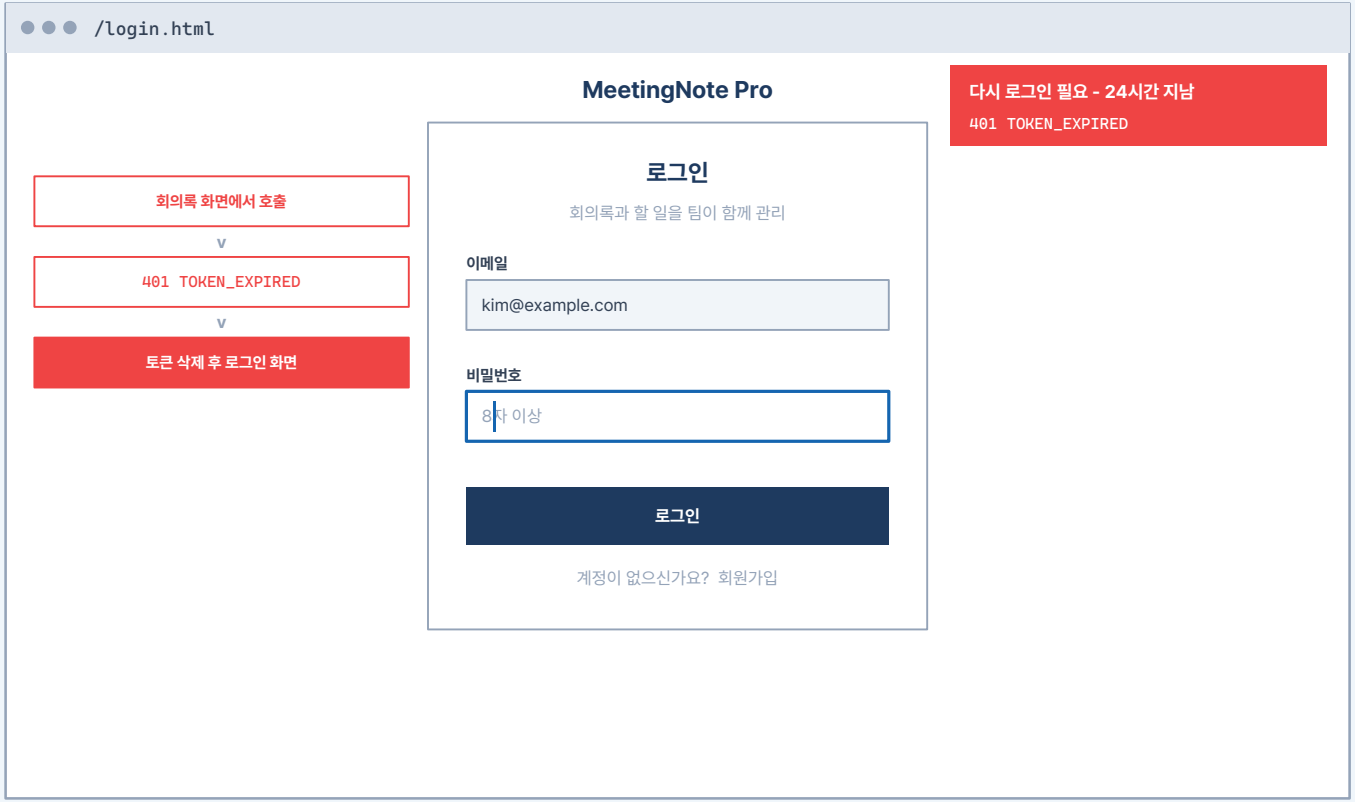
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join
```

B-03. 로그인 / 회원가입 - 세션 만료

24h 지난 토큰으로 호출하면 토큰을 지우고 로그인 화면으로



상태 3 / 11

응답

`401 TOKEN_EXPIRED`

구현 대상 파일

`login.html`

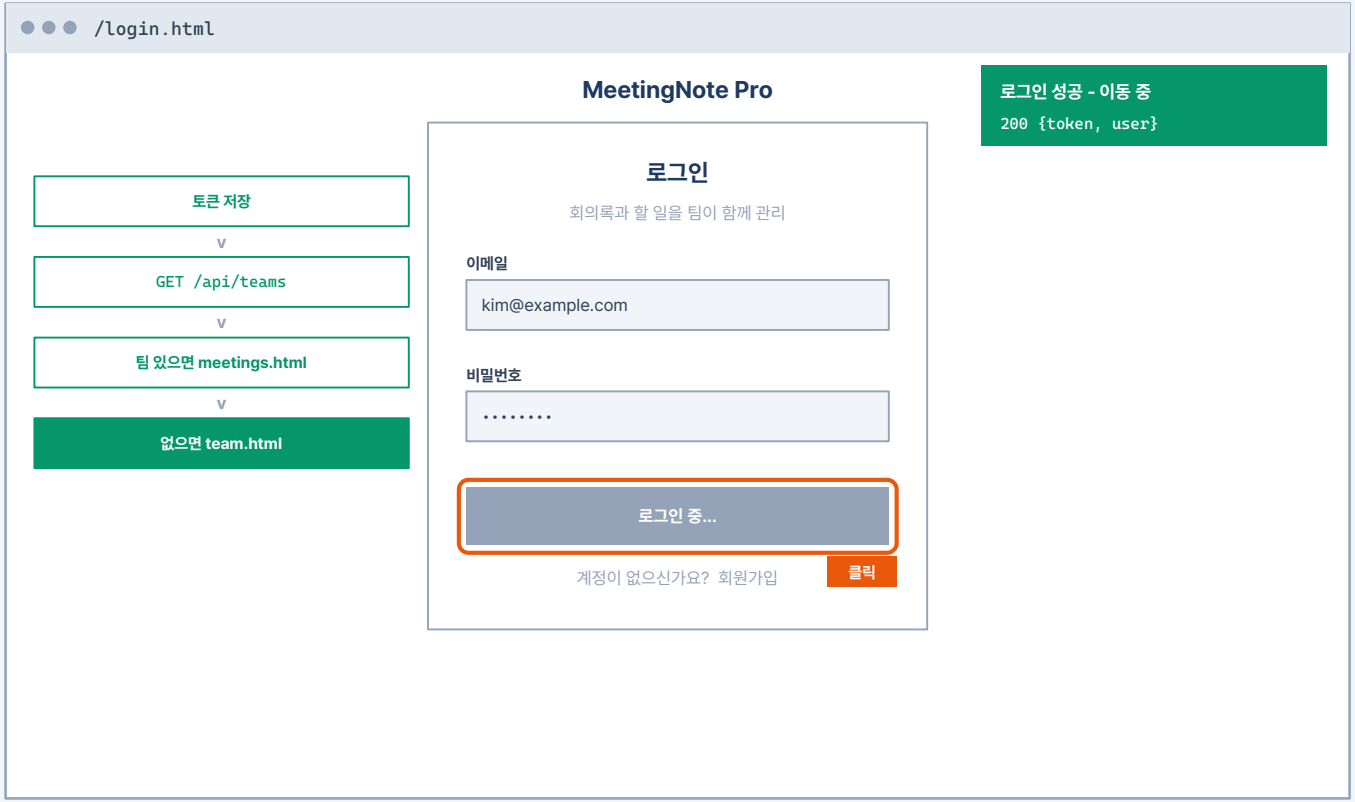
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join
```

B-04. 로그인 / 회원가입 - 로그인 성공

응답 {token, user}. 토큰을 localStorage 에 저장한 뒤 GET /api/teams 로 소속 팀 검사. 없으면 팀 설정 화면



상태 4 / 11

응답

`200 {token, user}`

구현 대상 파일

`login.html`

이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

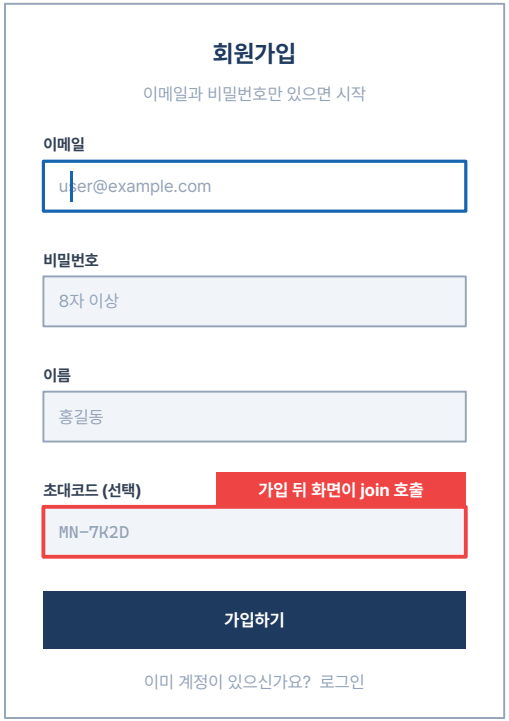
쓰이는 엔드포인트

```
POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join
```

B-05. 로그인 / 회원가입 - 회원가입 초기

이름 · 초대코드 칸은 가입에만. signup 은 초대코드를 받지 않음 - 가입 뒤 화면이 join 을 따로 호출. 비우면 팀 생성

/login.html



상태 5 / 11

응답

`201 {token, user}`

구현 대상 파일

`login.html`

이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join
```

B-06. 로그인 / 회원가입 - 입력 중

제출 전에는 오류를 띄우지 않음

/login.html



상태 6 / 11

응답

–

구현 대상 파일

`login.html`

이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join
```

B-07. 로그인 / 회원가입 - 처리 중

버튼을 잠가 2번 눌리지 않게 함

/login.html



상태 7 / 11

응답

요청 대기

구현 대상 파일

`login.html`

이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

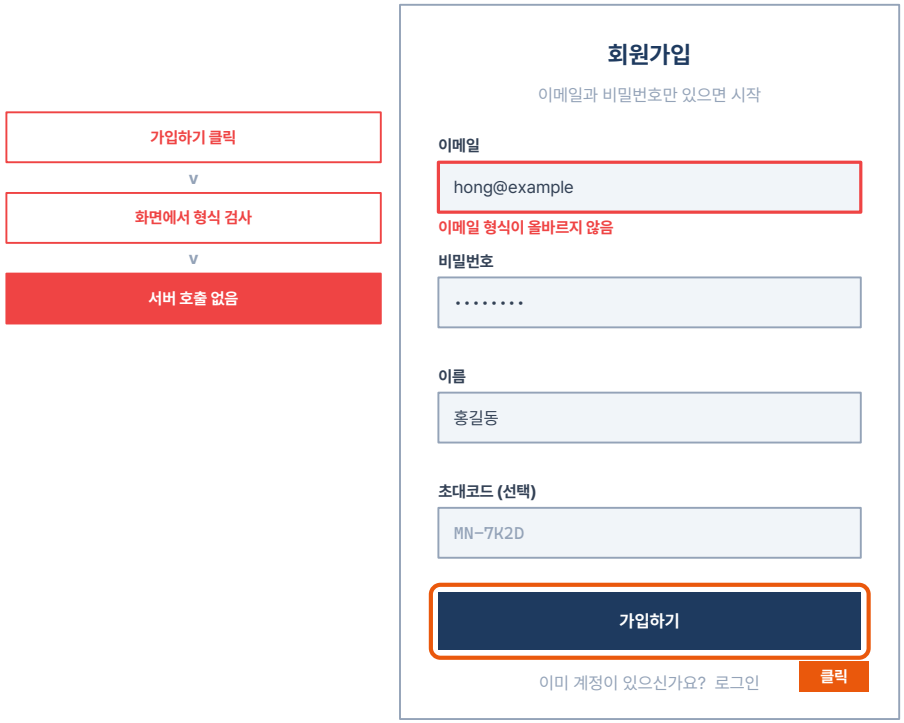
쓰이는 엔드포인트

```
POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join
```

B-08. 로그인 / 회원가입 - 에러 - 형식

이메일 형식은 화면에서 먼저 거름

/login.html



상태 8 / 11

응답

`400 EMAIL_INVALID`

구현 대상 파일

`login.html`

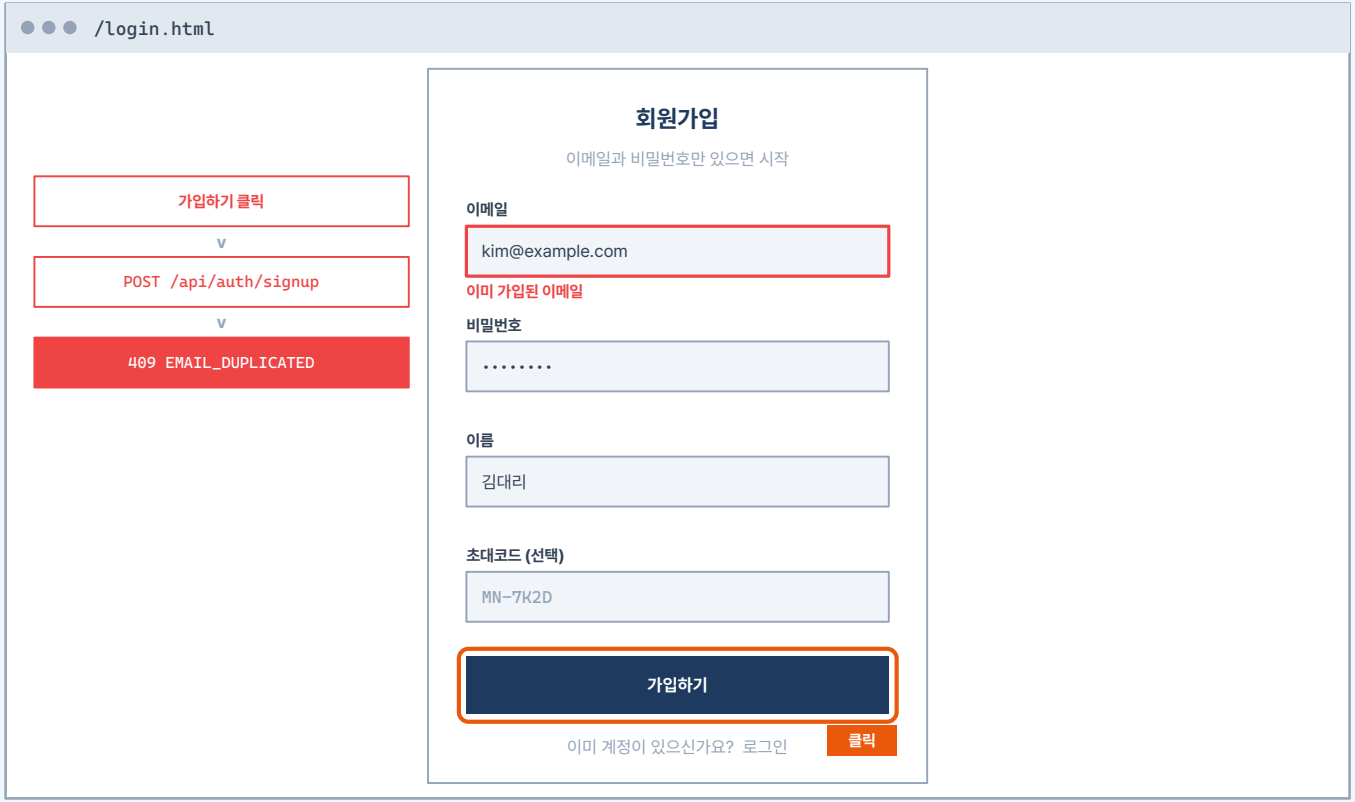
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join
```

B-09. 로그인 / 회원가입 - 에러 - 중복

중복은 서버만 알 수 있음



상태 9 / 11

응답

409 EMAIL_DUPLICATED

구현 대상 파일

`login.html`

이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

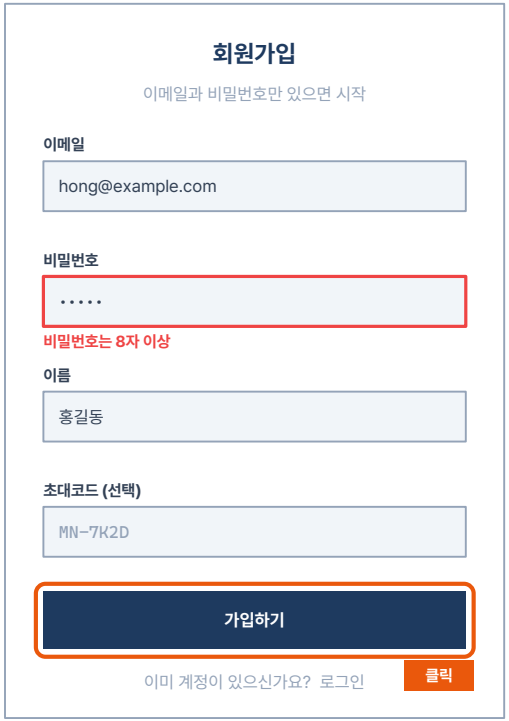
쓰이는 엔드포인트

POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join

B-10. 로그인 / 회원가입 - 에러 - 약한 비밀번호

8자 미만은 화면에서 거름

/login.html



상태 10 / 11

응답

`400 PASSWORD_TOO_WEAK`

구현 대상 파일

`login.html`

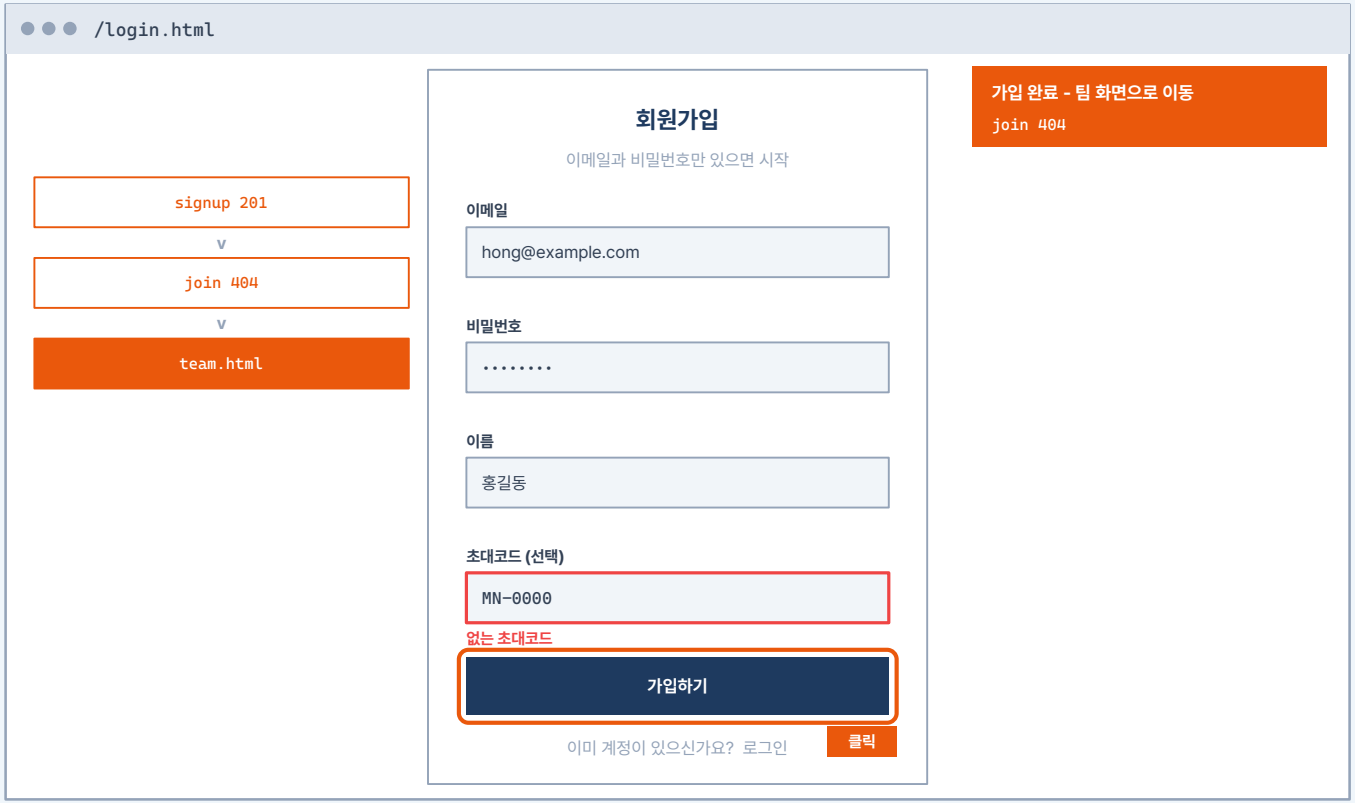
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join
```

B-11. 로그인 / 회원가입 - 에러 - 초대코드

가입은 되고 팀 합류만 실패. 팀 화면으로 보냄



상태 11 / 11

응답

`404 INVITE_NOT_FOUND`

구현 대상 파일

`login.html`

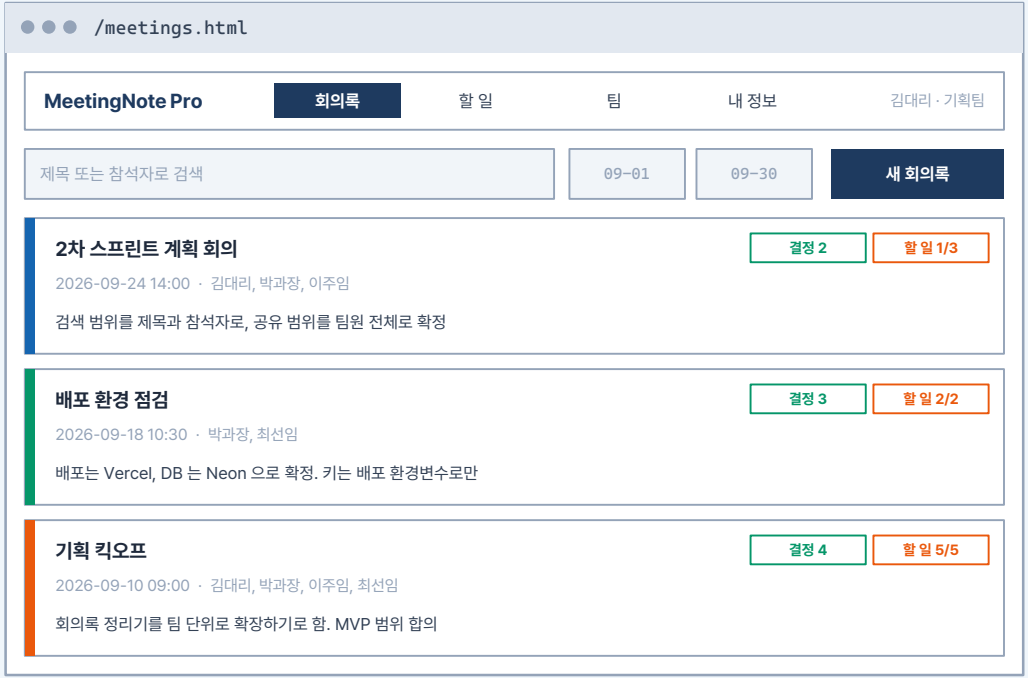
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/auth/signup
POST /api/auth/login
GET /api/teams
POST /api/teams/join
```

C. 회의록 목록

가장 오래 쓰는 화면. 검색과 등록이 한 화면에 있음



상태 11종

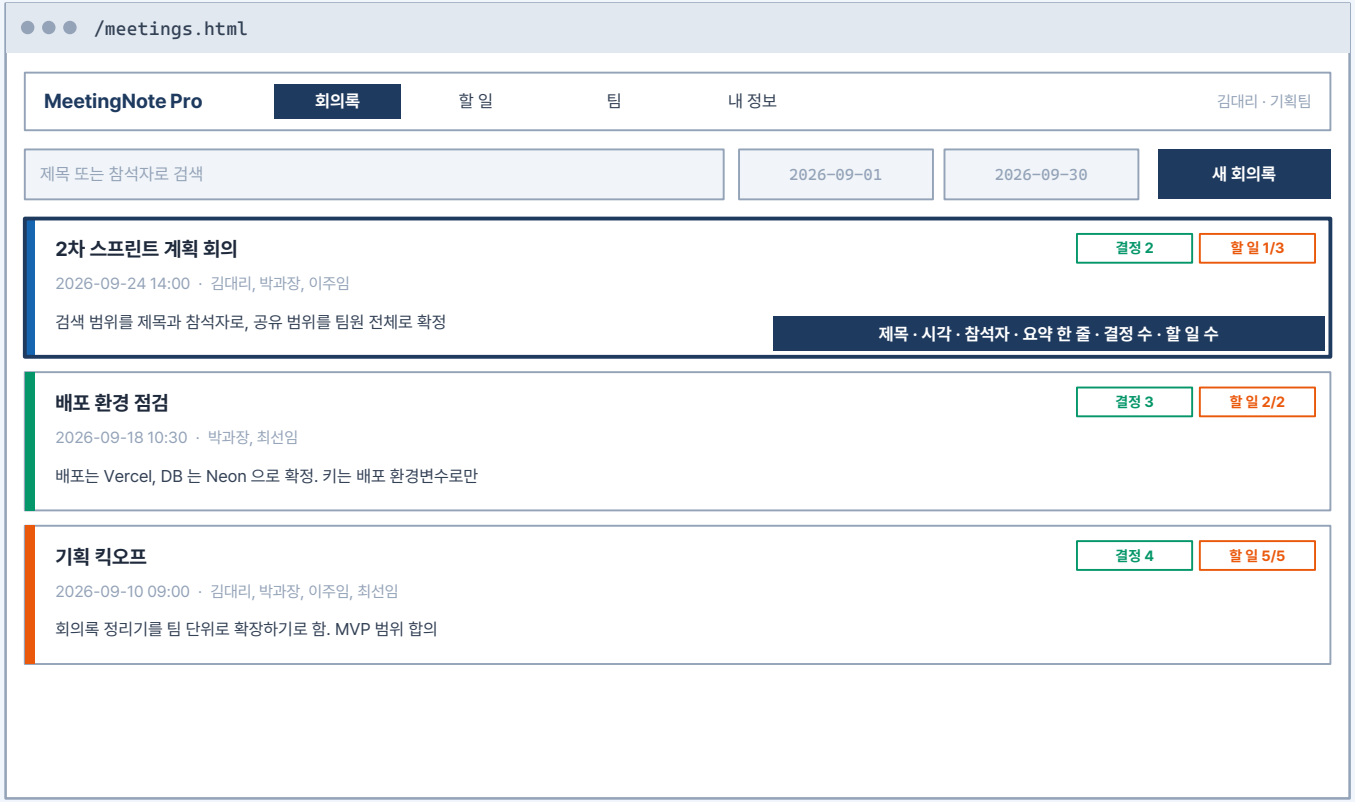
01	목록	200
02	빈 상태	200 + 빈 배열
03	검색	200 ?q=
04	검색 0건	200
05	새 회의록	
06	올리는 중	POST /api/upload
07	받아쓰기 완료	200 {body}
08	에러 - 형식	415 UNSUPPORTED_MEDIA_TYPE
09	에러 - 받아쓰기	502 VALIDATION_ERROR
10	정리 중	요청 대기
11	저장 완료	201

쓰이는 엔드포인트 `GET /api/teams` · `GET /api/teams/{id}/meetings` · `POST /api/teams/{id}/meetings` · `POST /api/upload`

상태 11종: 정상 흐름뿐 아니라 빈 상태 · 오류 · 처리 중까지 화면으로 고정

C-01. 회의록 목록 - 목록

카드 3열. 목록 응답에는 body 를 싣지 않음



상태 1 / 11

응답

200

구현 대상 파일

`meetings.html`

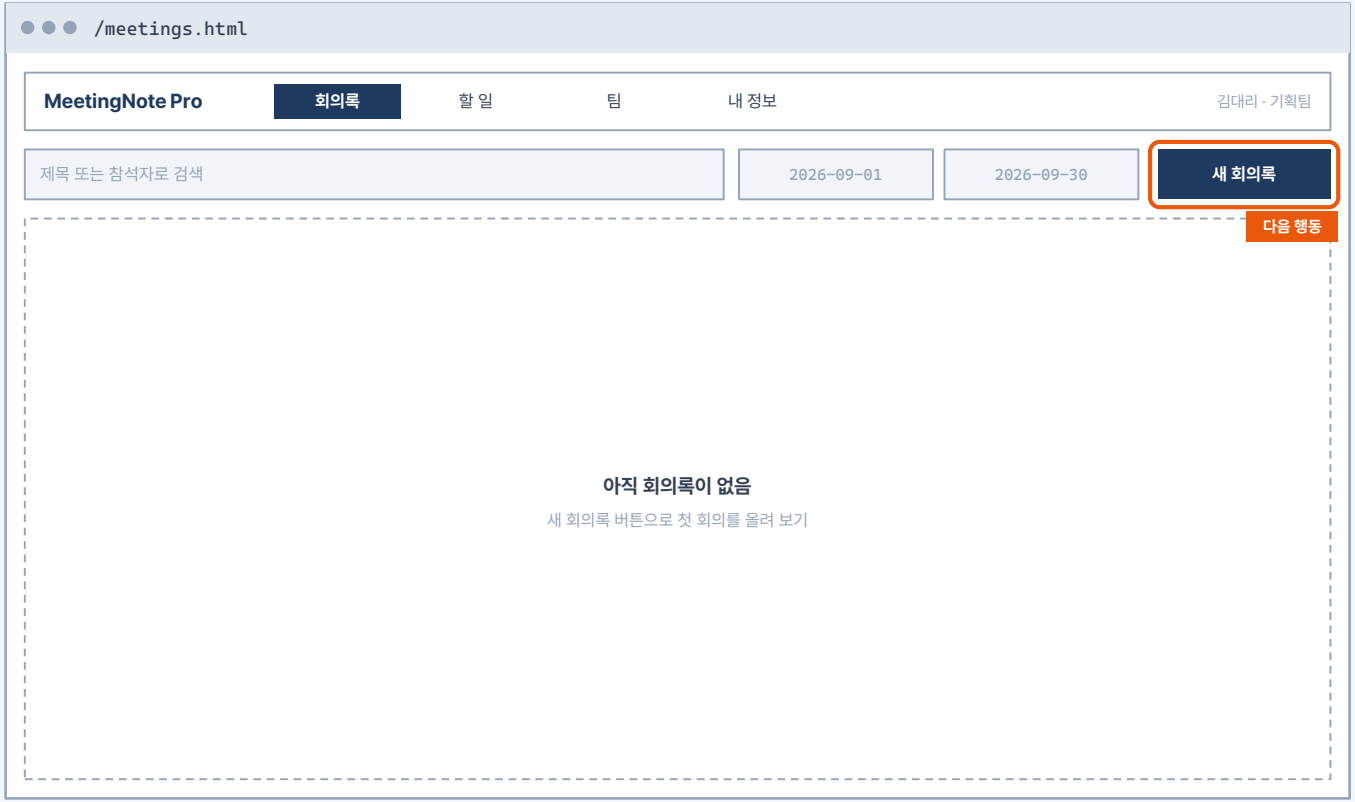
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

C-02. 회의록 목록 - 빈 상태

팀 합류 직후 첫 화면. 다음 행동을 한 줄로 안내



상태 2 / 11

응답

200 + 빈 배열

구현 대상 파일

`meetings.html`

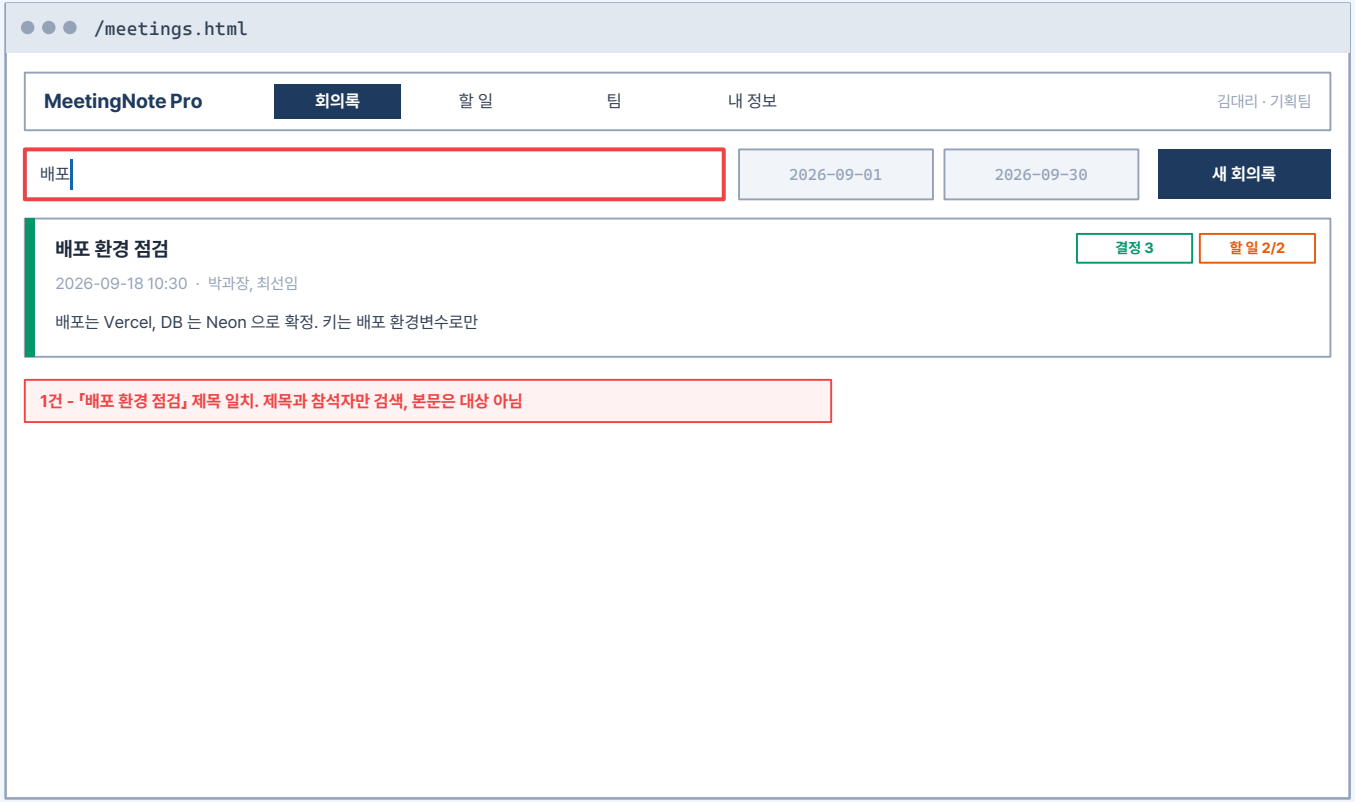
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

C-03. 회의록 목록 - 검색

제목과 참석자만 일치. 본문은 검색 대상이 아님



상태 3 / 11

응답

`200 ?q=`

구현 대상 파일

`meetings.html`

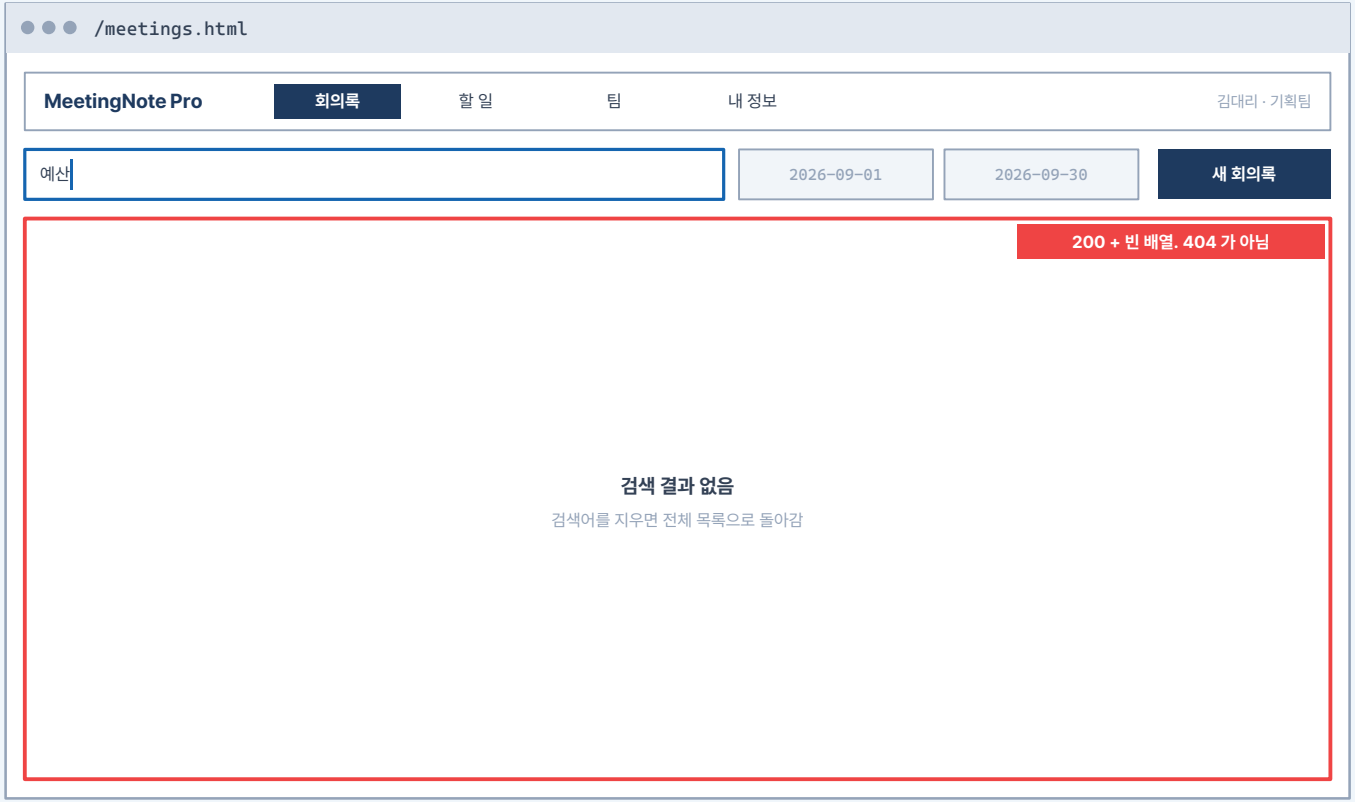
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

C-04. 회의록 목록 - 검색 0건

404 가 아니라 200 + 빈 배열



상태 4 / 11

응답

200

구현 대상 파일

`meetings.html`

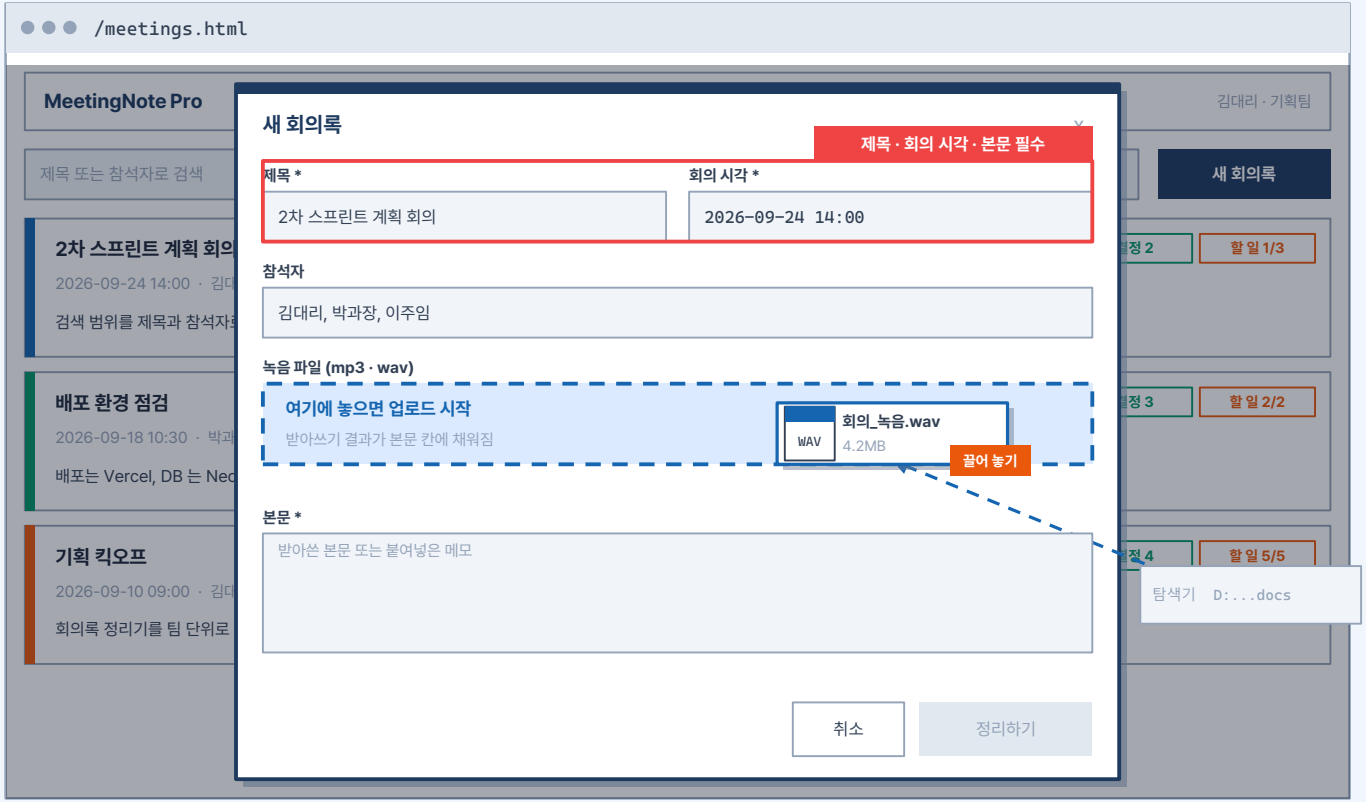
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

C-05. 회의록 목록 - 새 회의록

제목 · 회의 시각 · 본문이 필수



상태 5 / 11

응답

–

구현 대상 파일

`meetings.html`

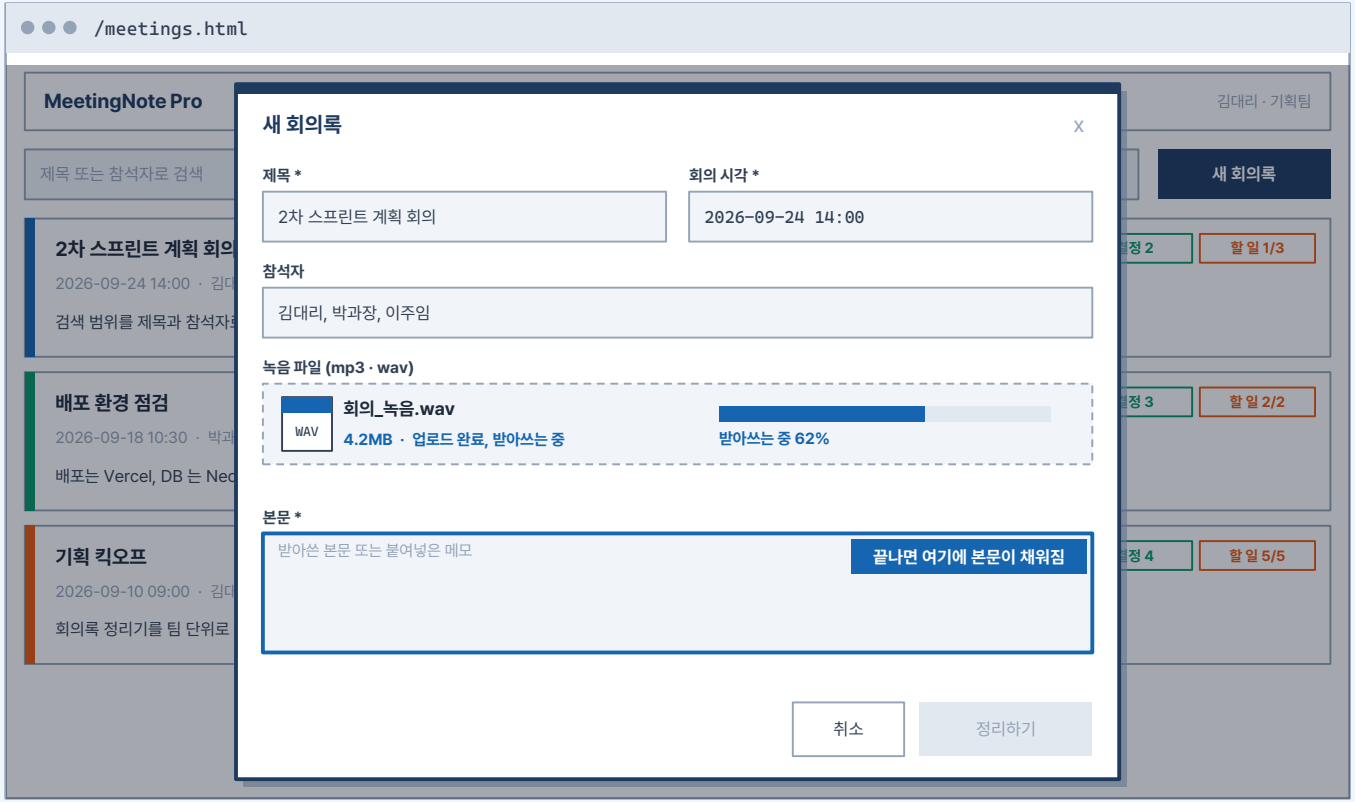
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

C-06. 회의록 목록 - 올리는 중

받아쓰기 진행률 표시. 끝나면 본문 칸이 채워짐



상태 6 / 11

응답

`POST /api/upload`

구현 대상 파일

`meetings.html`

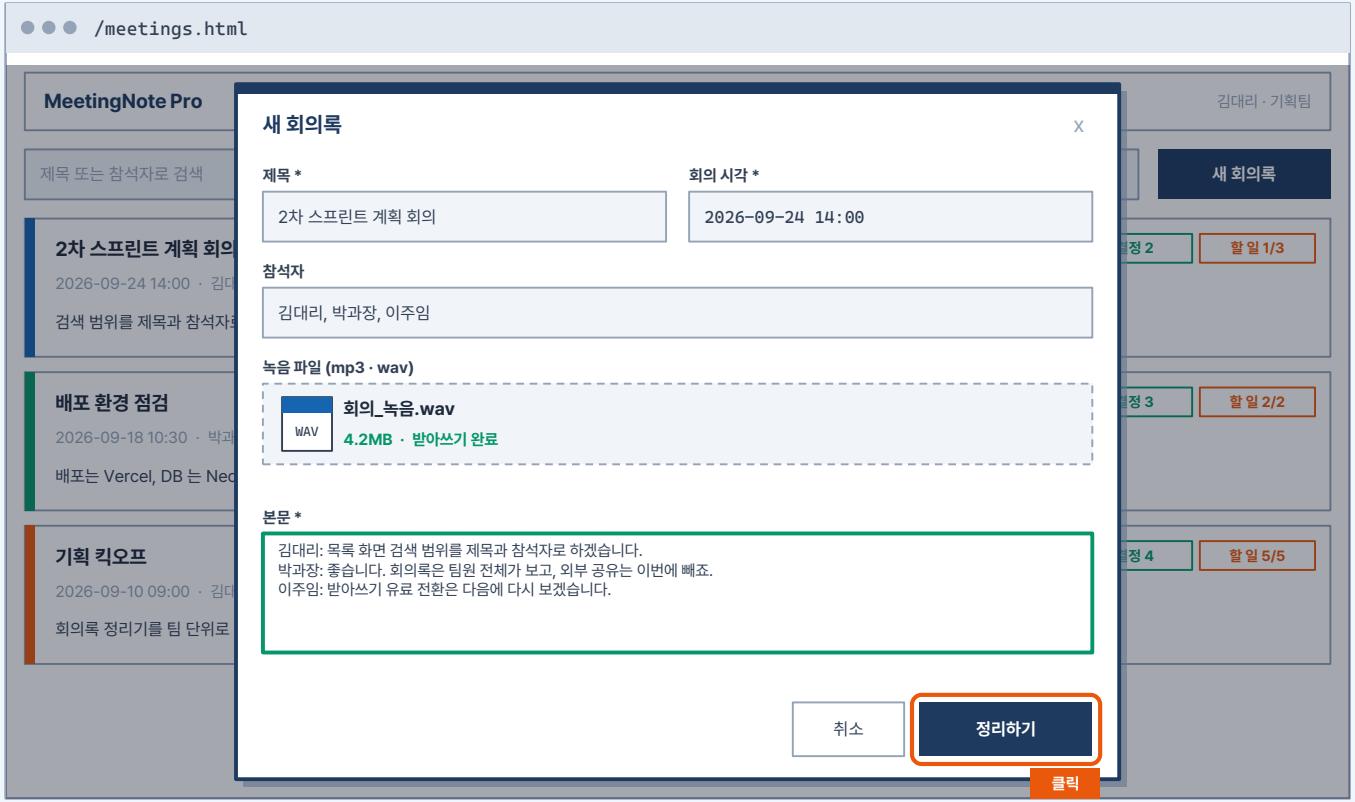
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

C-07. 회의록 목록 - 받아쓰기 완료

응답 {body}. 본문만 채워진 상태. 저장은 아직 아님



상태 7 / 11

응답

`200 {body}`

구현 대상 파일

`meetings.html`

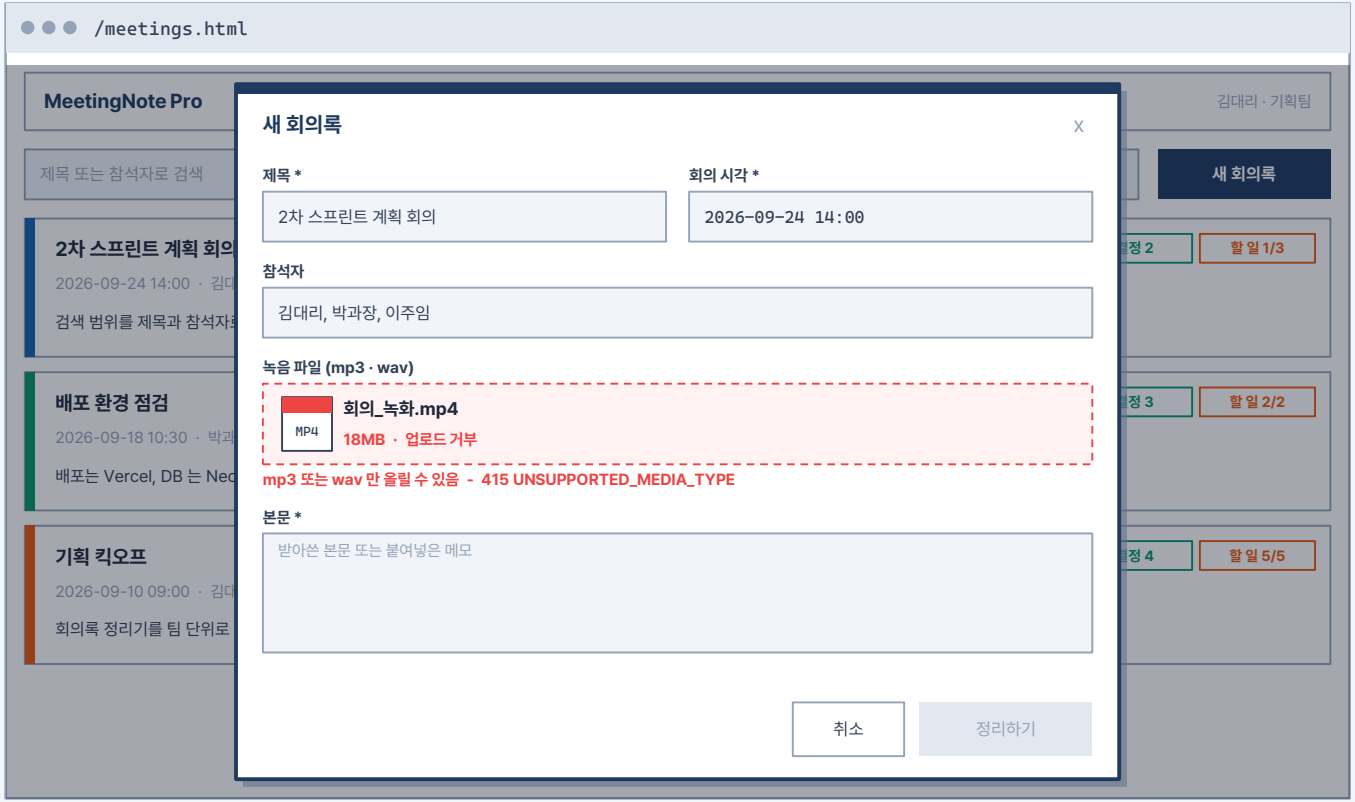
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

C-08. 회의록 목록 - 에러 - 형식

mp3 · wav 만. 확장자가 아니라 내용 형식으로 판정



상태 8 / 11

응답

`415 UNSUPPORTED_MEDIA_TYPE`

구현 대상 파일

`meetings.html`

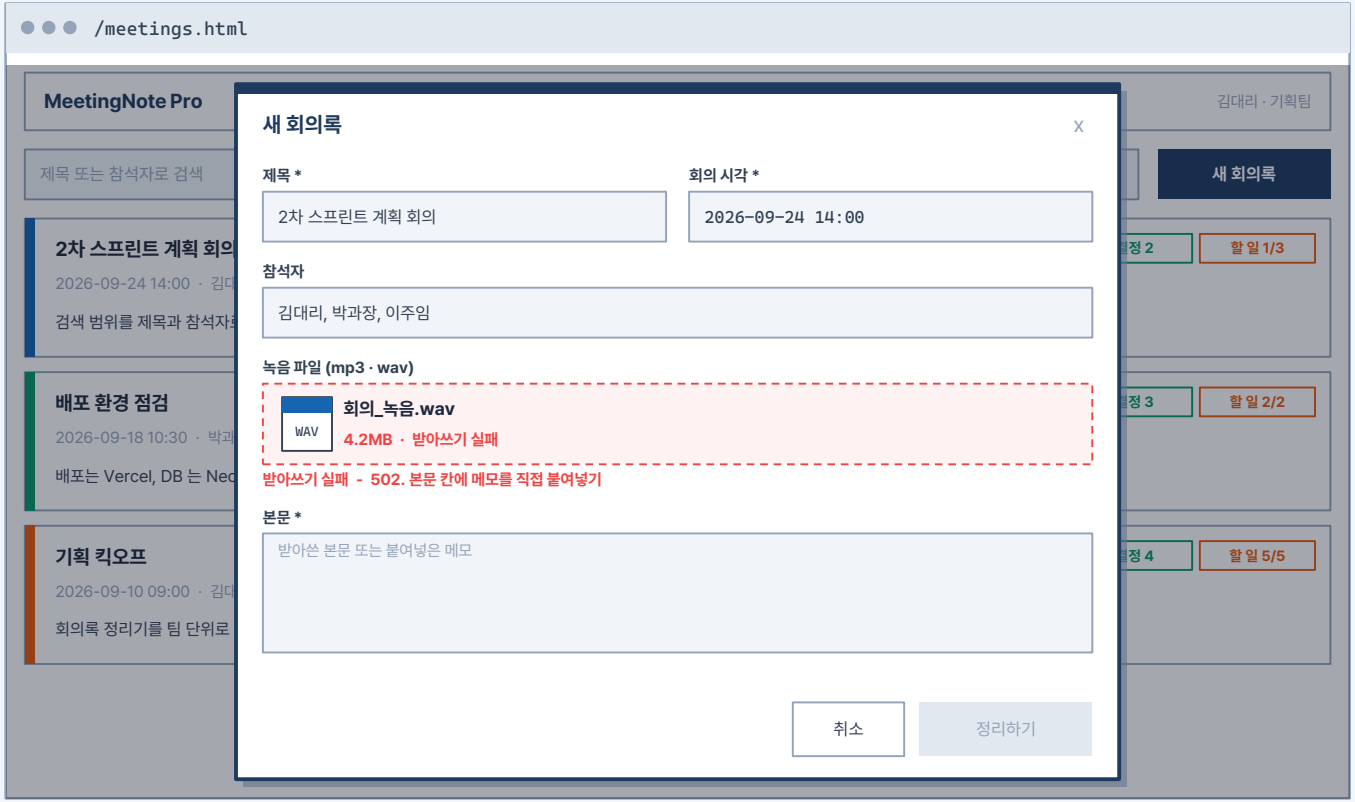
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

C-09. 회의록 목록 - 에러 - 받아쓰기

Gemini 응답 오류 · 키 없음. 본문 칸에 메모를 직접 붙여넣어 저장 가능



상태 9 / 11

응답

`502 VALIDATION_ERROR`

구현 대상 파일

`meetings.html`

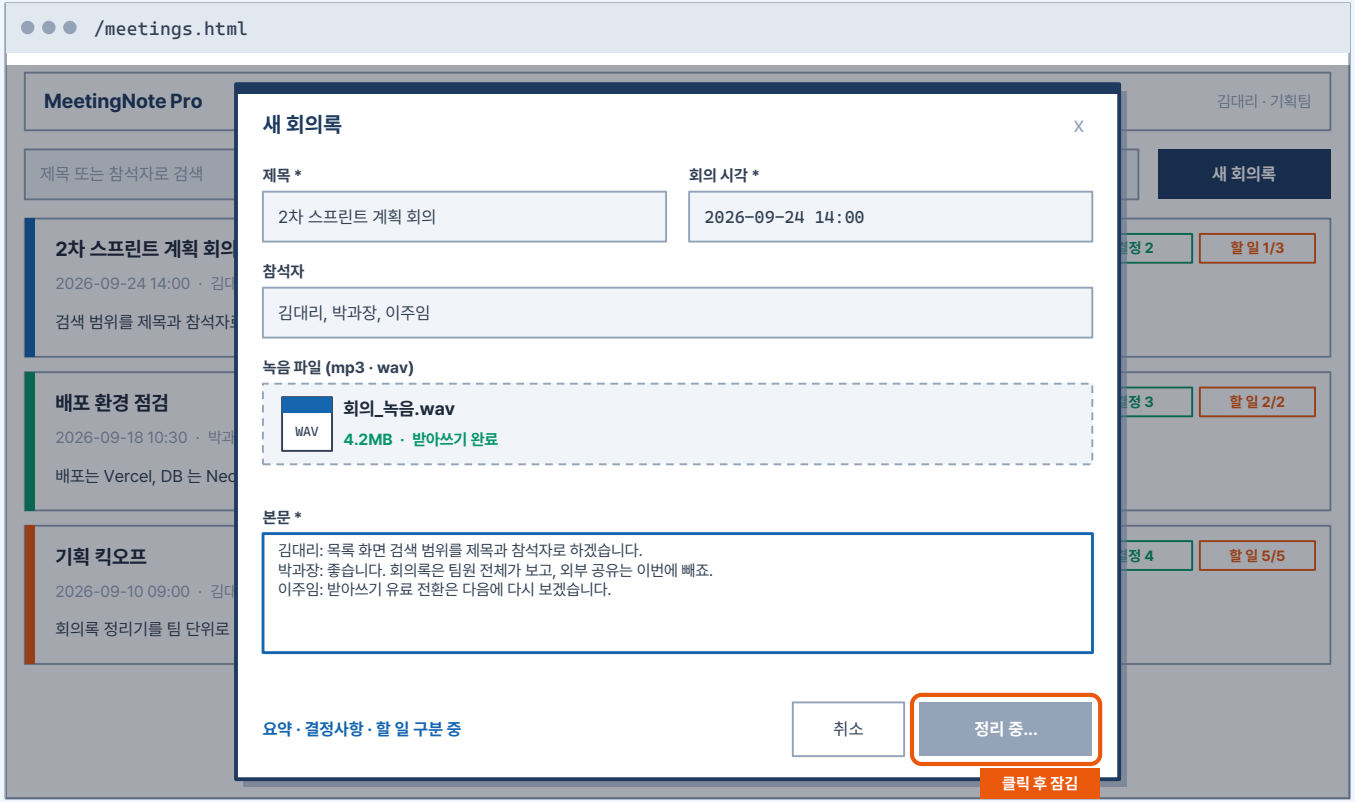
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

C-10. 회의록 목록 - 정리 중

저장하며 요약 · 결정사항 · 할 일 구분까지 한 번에. Gemini 호출이라 100ms 기준 예외



상태 10 / 11

응답

요청 대기

구현 대상 파일

`meetings.html`

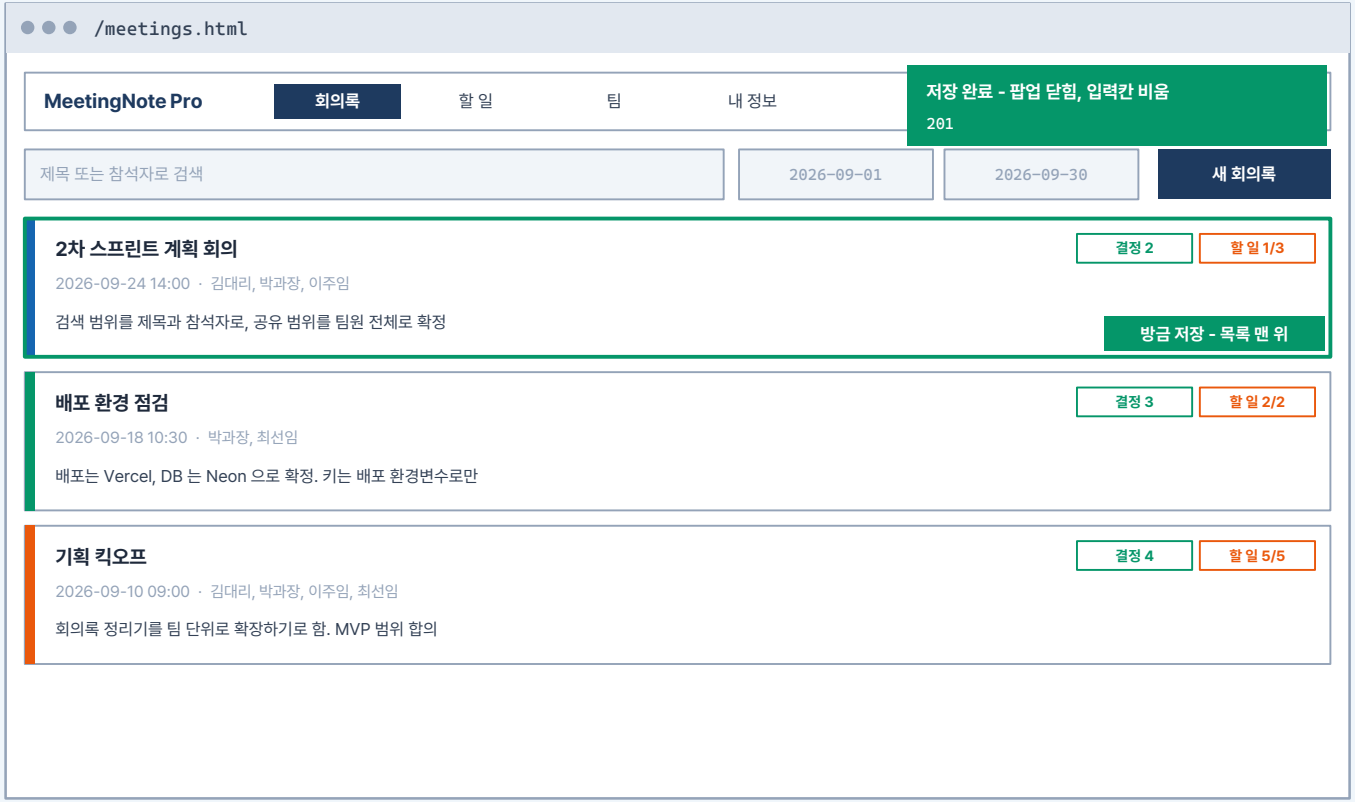
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

C-11. 회의록 목록 - 저장 완료

입력칸을 비워 같은 회의록이 2번 저장되지 않게 함



상태 11 / 11

응답

201

구현 대상 파일

`meetings.html`

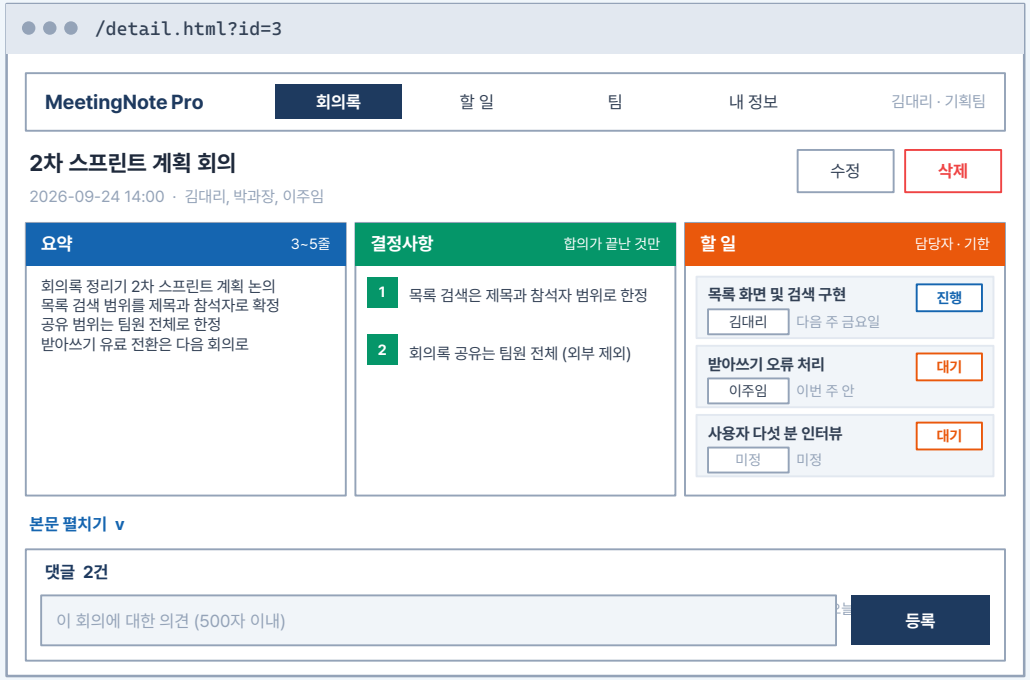
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams
GET /api/teams/{id}/meetings
POST /api/teams/{id}/meetings
POST /api/upload
```

D. 회의록 상세

요약 · 결정사항 · 할 일 구분 결과를 확인하고 할 일에 담당자를 붙이는 화면



상태 13종

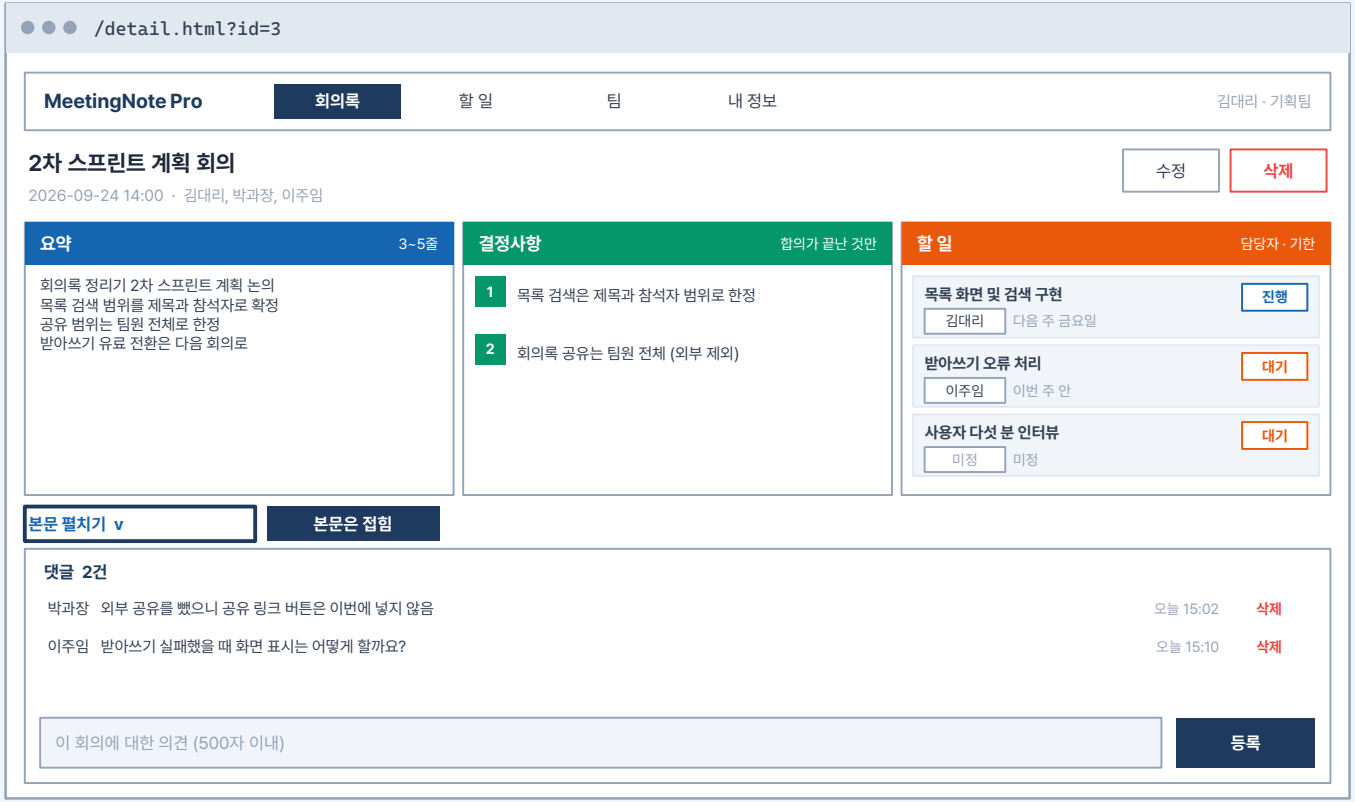
01	기본	200
02	수정	PUT 200
03	담당자 배정	PUT /api/todos/{id} 200
04	상태 변경	PUT 200
05	결정사항 없음	200
06	할 일 없음	200
07	본문 펼침	
08	삭제 확인	DELETE 204
09	댓글 입력	
10	댓글 등록	POST /api/meetings/{id}/comments 201
11	댓글 없음	200 + 빈 배열
12	댓글 삭제	DELETE /api/comments/{id} 204
13	없는 회의록	404 MEETING_NOT_FOUND

쓰이는 엔드포인트 `GET /api/meetings/{id}` · `GET /api/teams/{id}/members` · `PUT /api/meetings/{id}` · `DELETE /api/meetings/{id}` · `PUT /api/todos/{id}` · `POST /api/meetings/{id}/comments` · `GET /api/meetings/{id}/comments` · `DELETE /api/comments/{id}`

상태 13종: 정상 흐름뿐 아니라 빈 상태 · 오류 · 처리 중까지 화면으로 고정

D-01. 회의록 상세 - 기본

요약 · 결정사항 · 할 일 3단. 단건 응답에 body 와 딸린 할 일 포함. 본문은 접은 상태



상태 1 / 13

응답

200

구현 대상 파일

`detail.html`

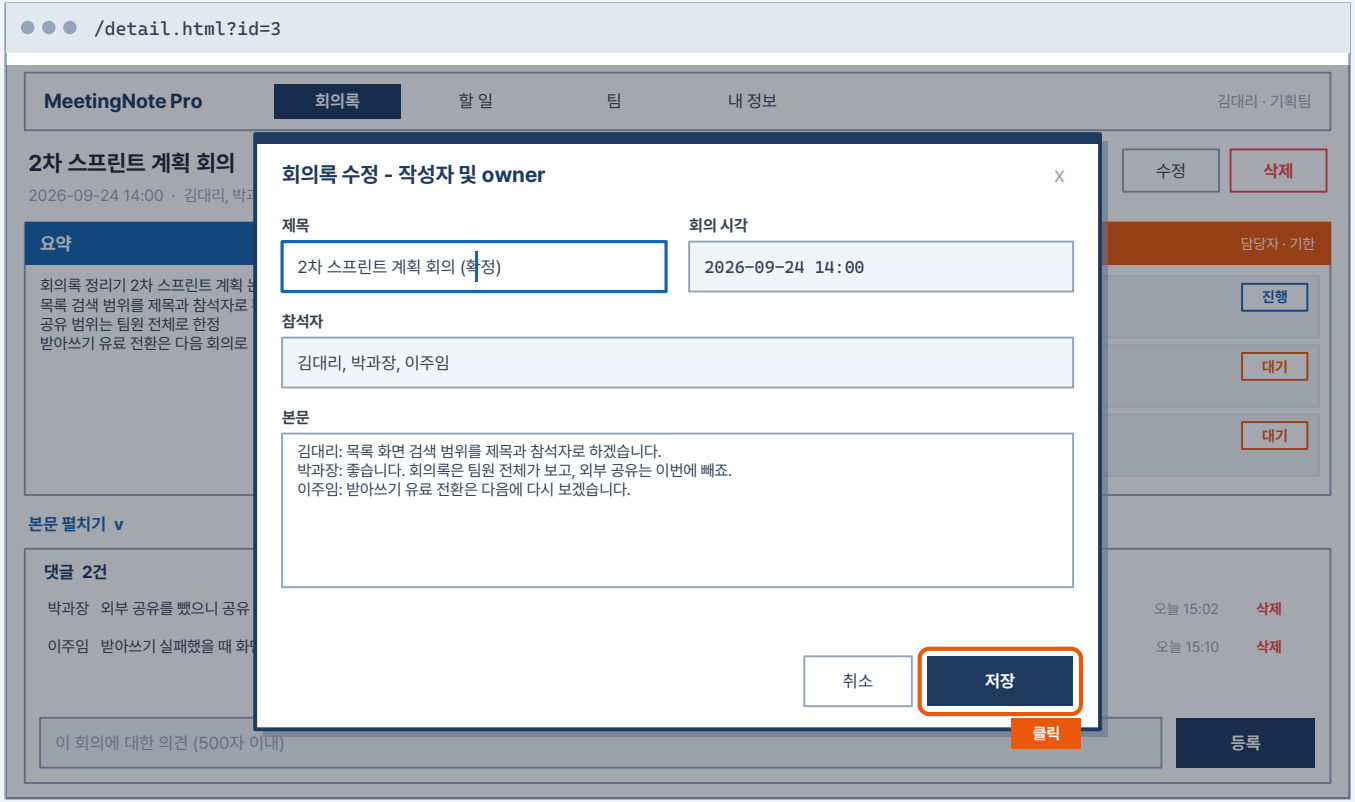
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-02. 회의록 상세 - 수정

제목 · 시각 · 참석자 · 본문을 고침



상태 2 / 13

응답

`PUT 200`

구현 대상 파일

`detail.html`

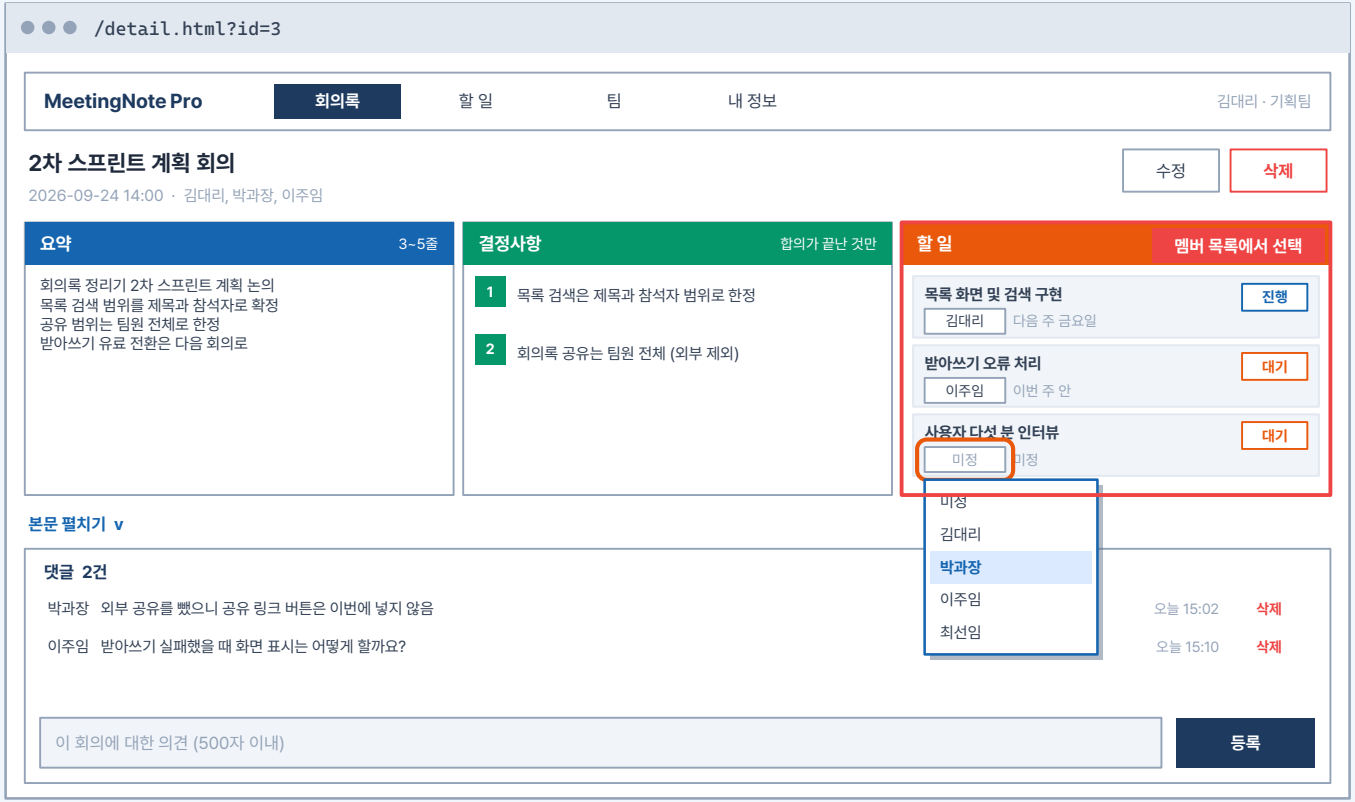
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-03. 회의록 상세 - 담당자 배정

미정 상태의 할 일에 담당자와 기한 지정. 담당자 후보는 GET /api/teams/{id}/members



상태 3 / 13

응답

```
PUT /api/todos/{id} 200
```

구현 대상 파일

`detail.html`

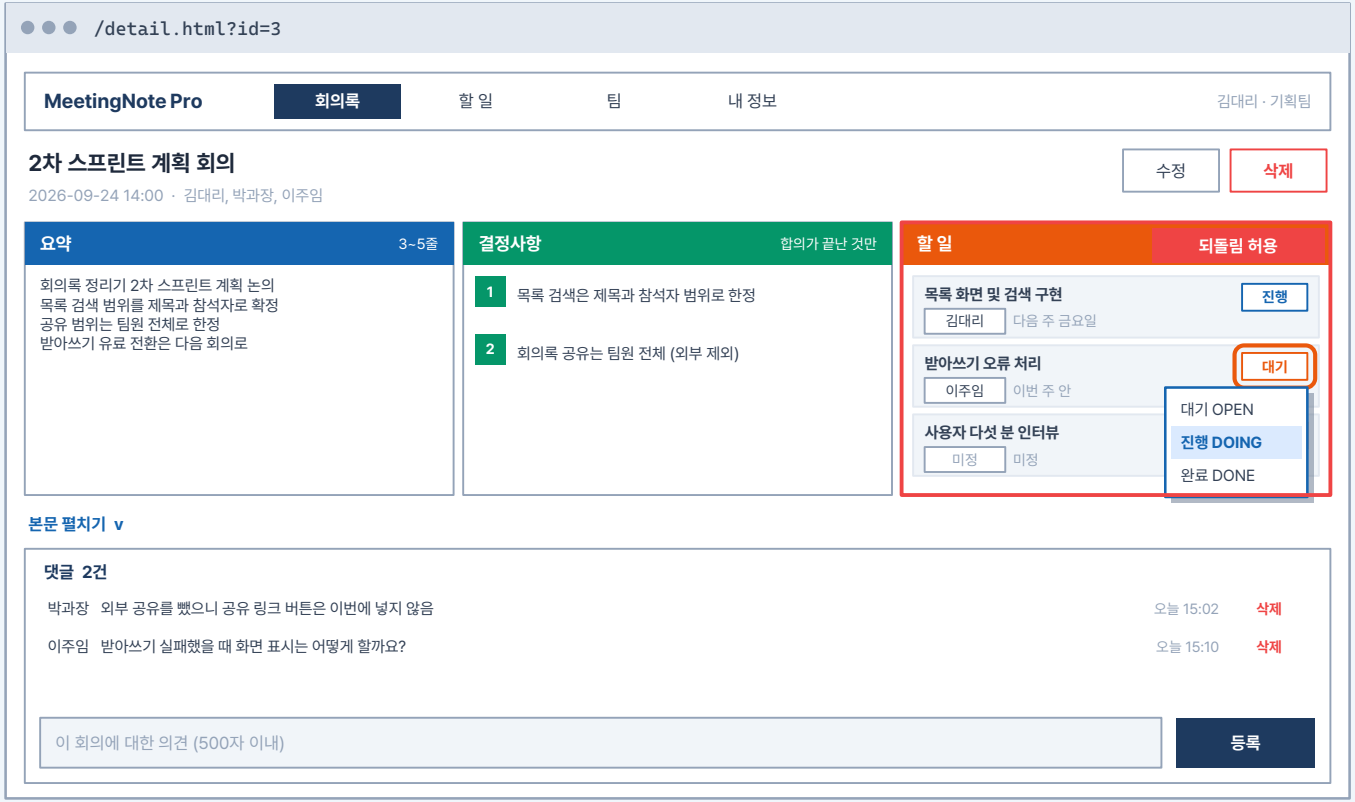
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-04. 회의록 상세 - 상태 변경

OPEN(대기) > DOING(진행) > DONE(완료) 3가지. 되돌림도 허용



상태 4 / 13

응답

`PUT 200`

구현 대상 파일

`detail.html`

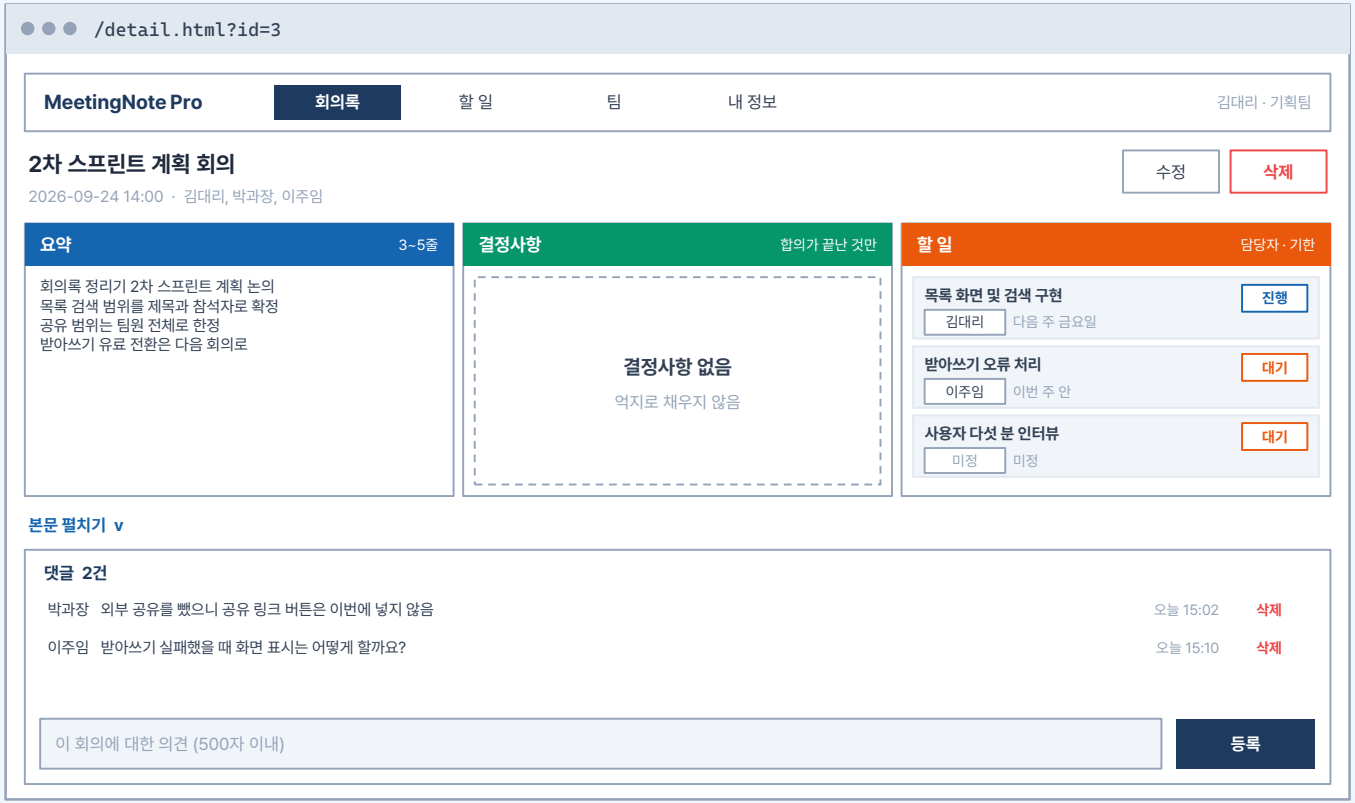
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-05. 회의록 상세 - 결정사항 없음

논의만 하고 정하지 않았으면 비움. 억지로 채우지 않음



상태 5 / 13

응답

200

구현 대상 파일

`detail.html`

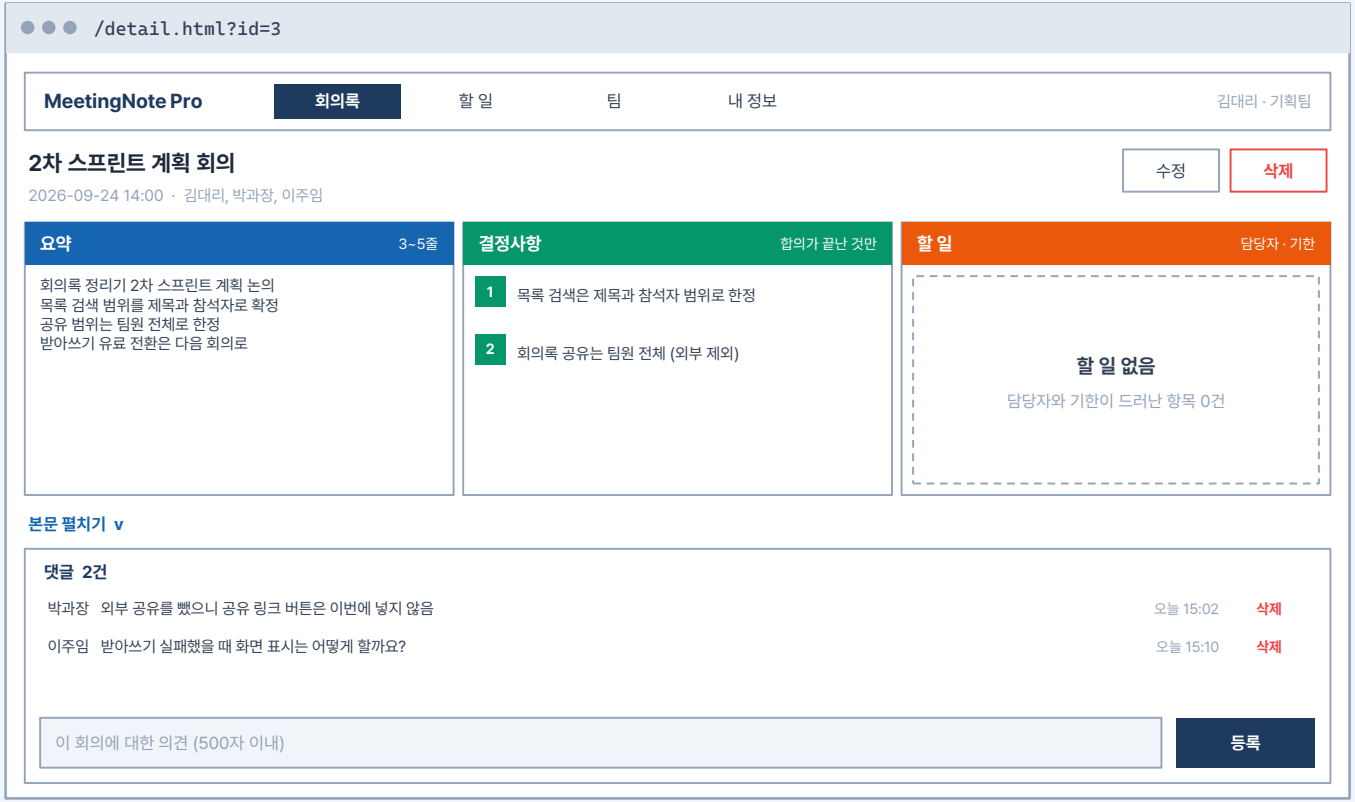
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-06. 회의록 상세 - 할 일 없음

담당자와 기한이 드러난 항목 0건



상태 6 / 13

응답

200

구현 대상 파일

`detail.html`

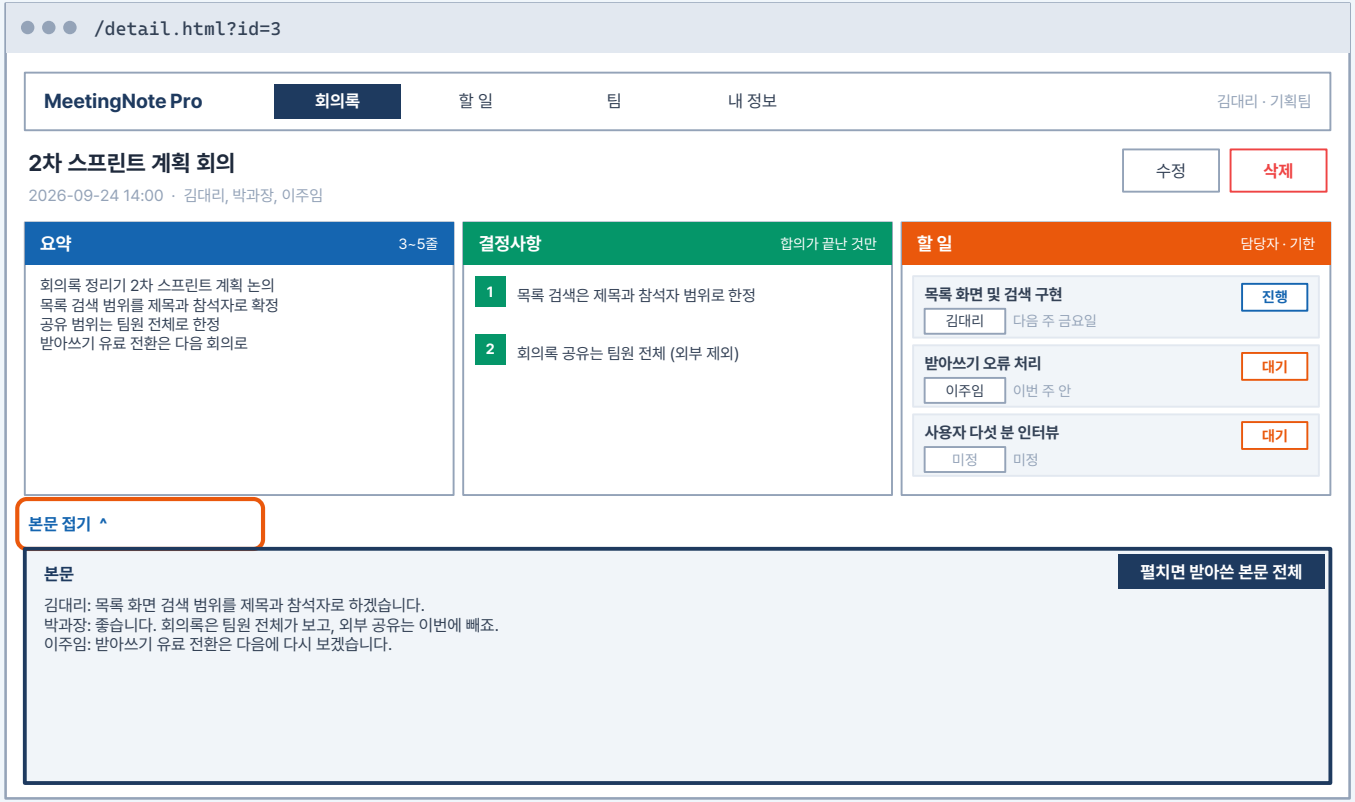
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-07. 회의록 상세 - 본문 펼침

요약 · 결정사항 · 할 일을 먼저 읽게 하는 배치



상태 7 / 13

응답

–

구현 대상 파일

`detail.html`

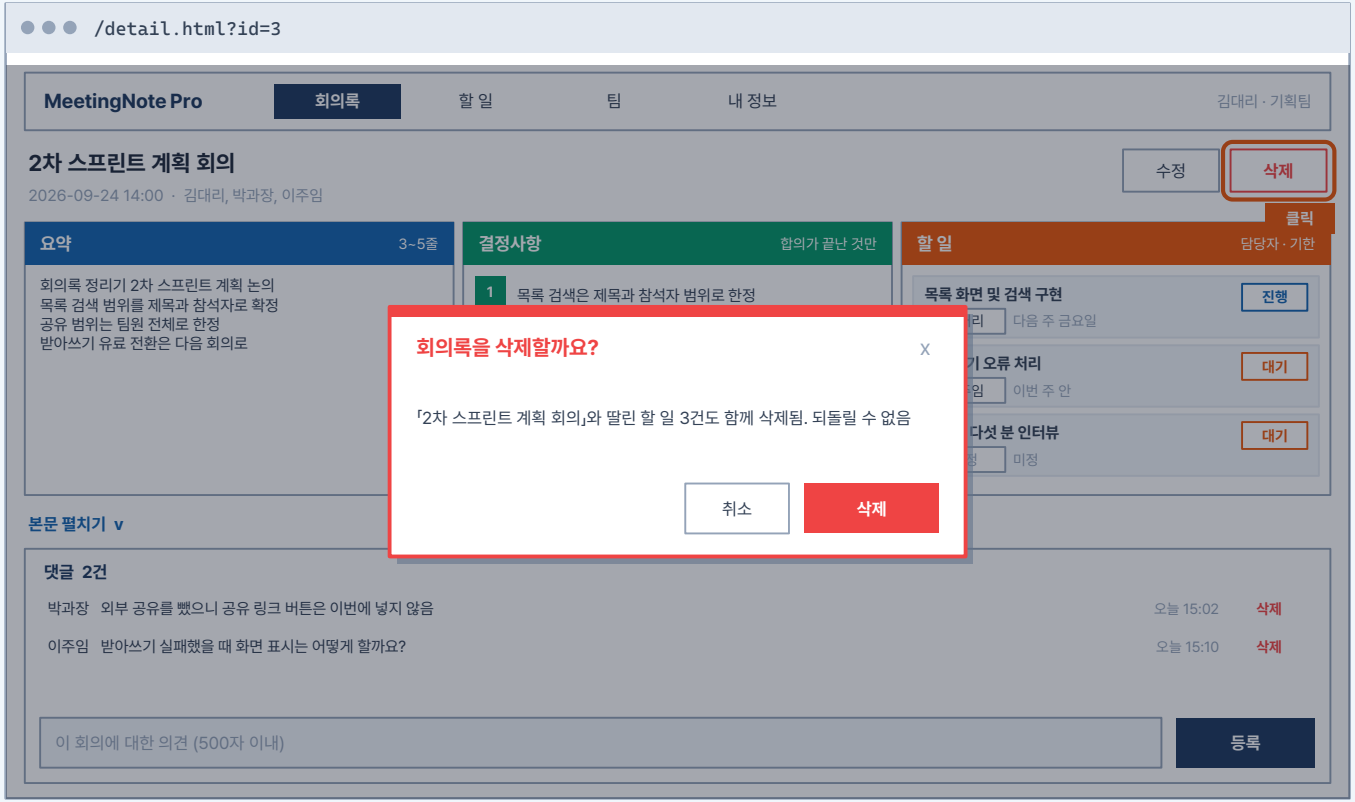
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-08. 회의록 상세 - 삭제 확인

딸린 할 일도 함께 사라진다고 미리 알림



상태 8 / 13

응답

`DELETE 204`

구현 대상 파일

`detail.html`

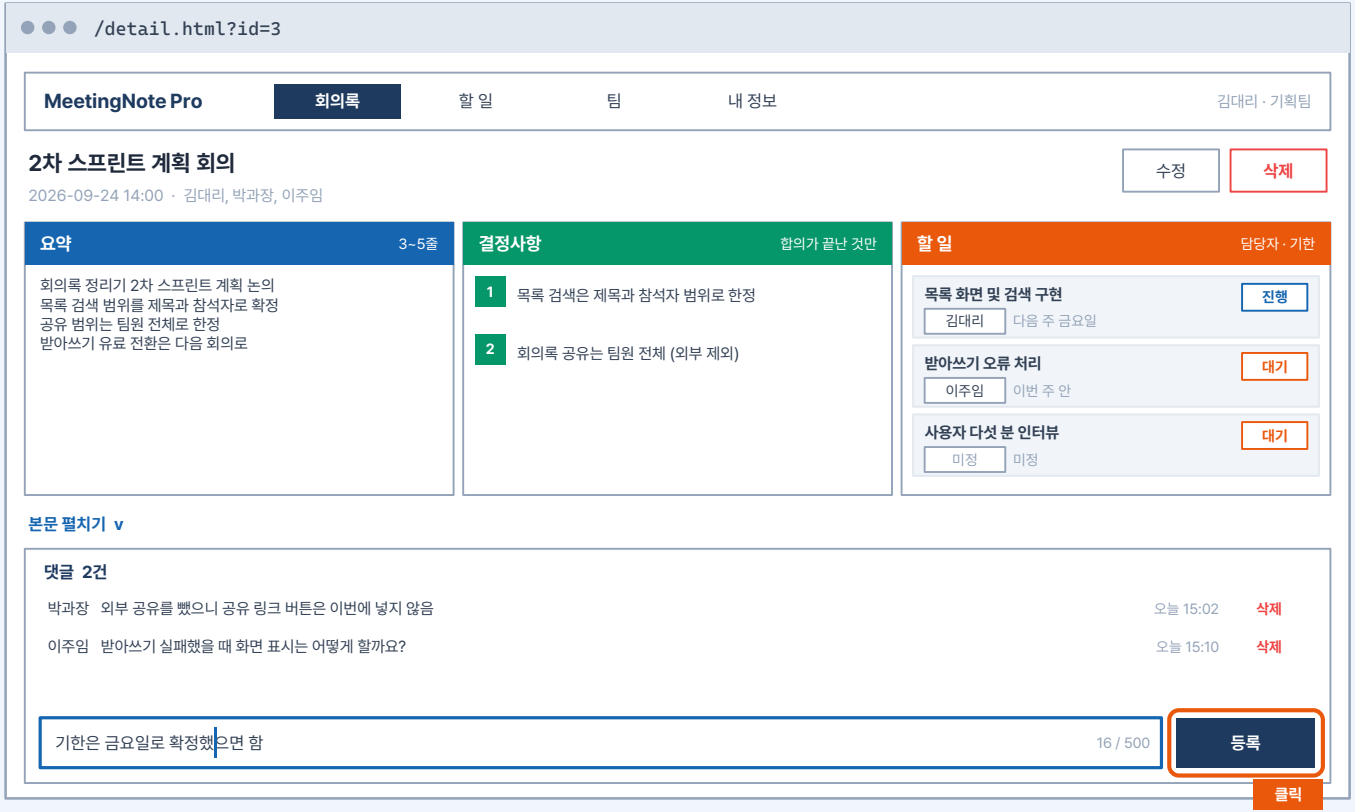
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-09. 회의록 상세 - 댓글 입력

결정에 의견이 있으면 여기에. 500자 이내



상태 9 / 13

응답

–

구현 대상 파일

`detail.html`

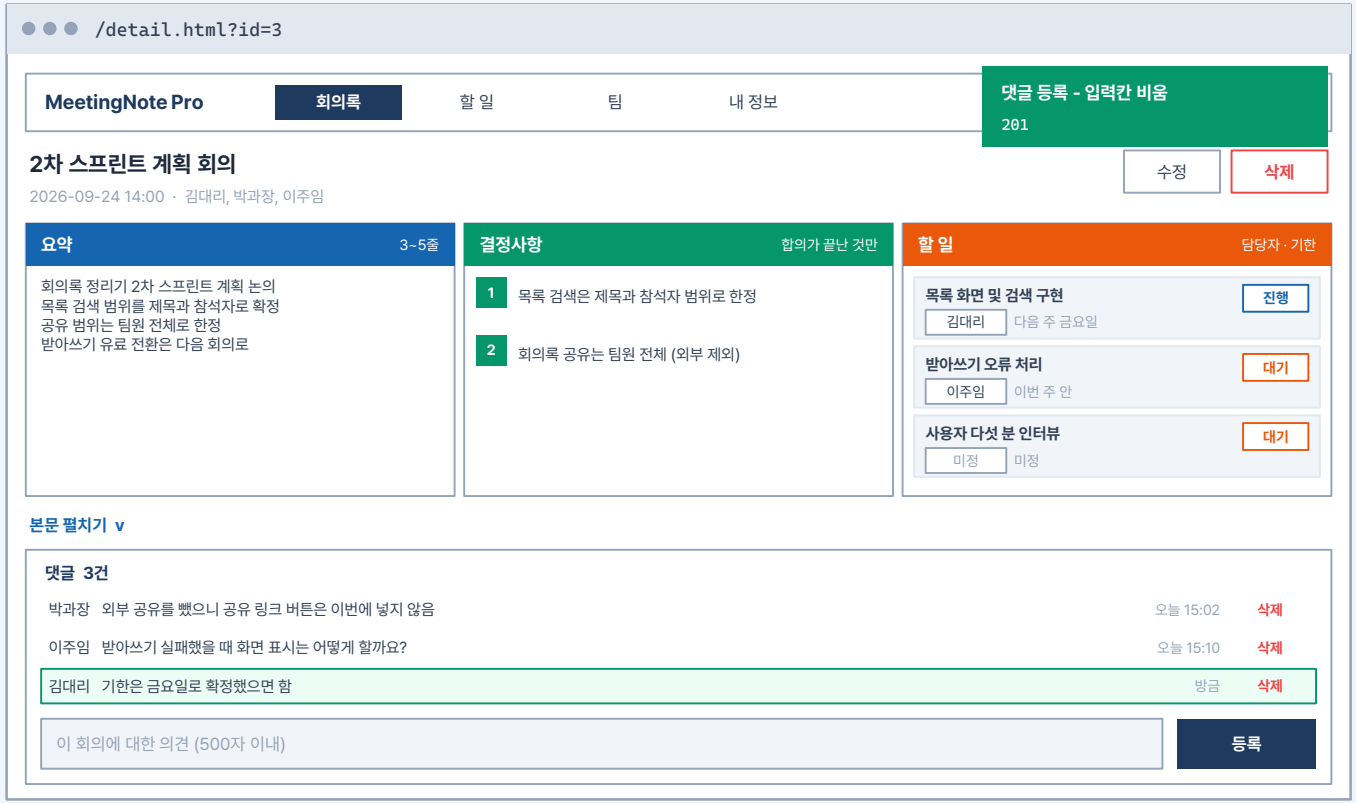
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-10. 회의록 상세 - 댓글 등록

201 후 입력칸 초기화. 활동 기록에도 기록



상태 10 / 13

응답

```
POST
/api/meetings/{id}/comments
201
```

구현 대상 파일

`detail.html`

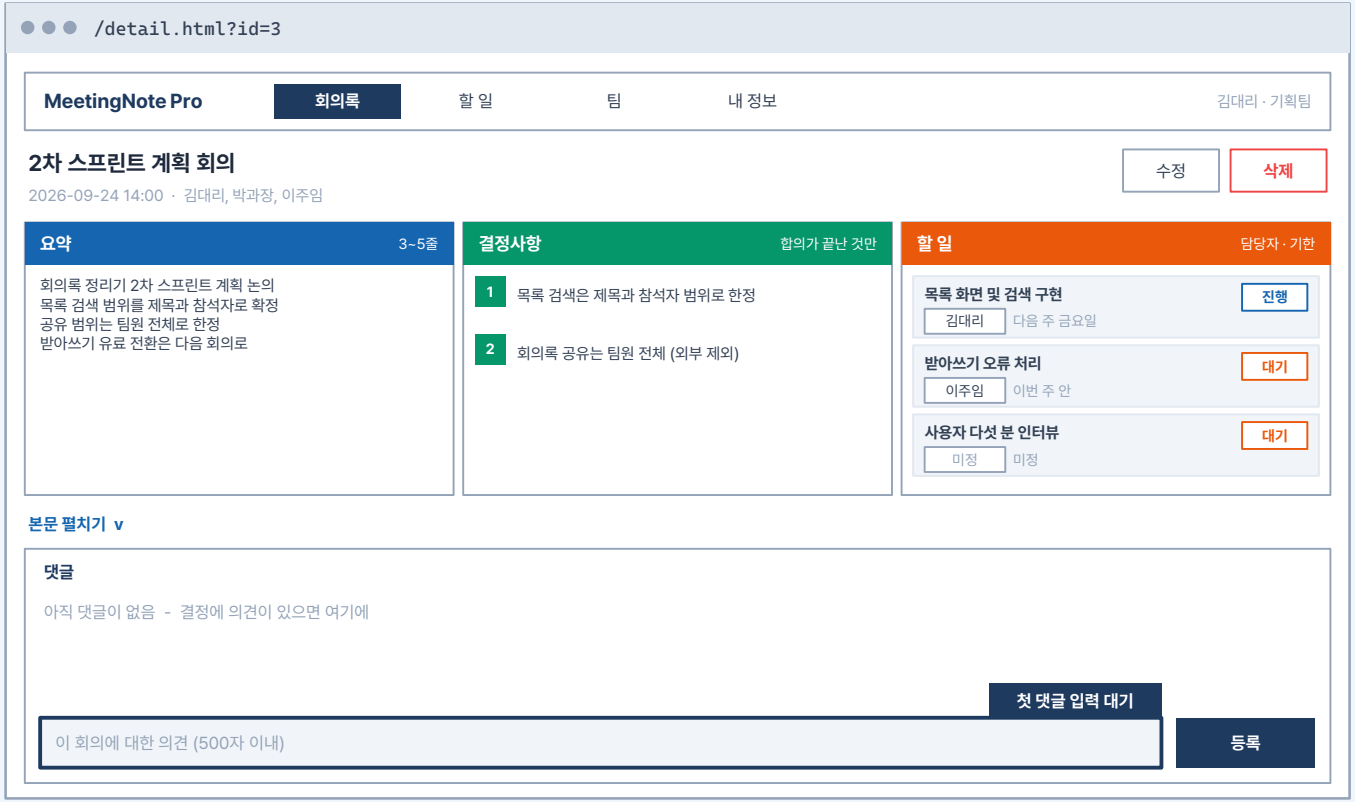
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-11. 회의록 상세 - 댓글 없음

댓글 0건이 기본 상태



상태 11 / 13

응답

`200 + 빈 배열`

구현 대상 파일

`detail.html`

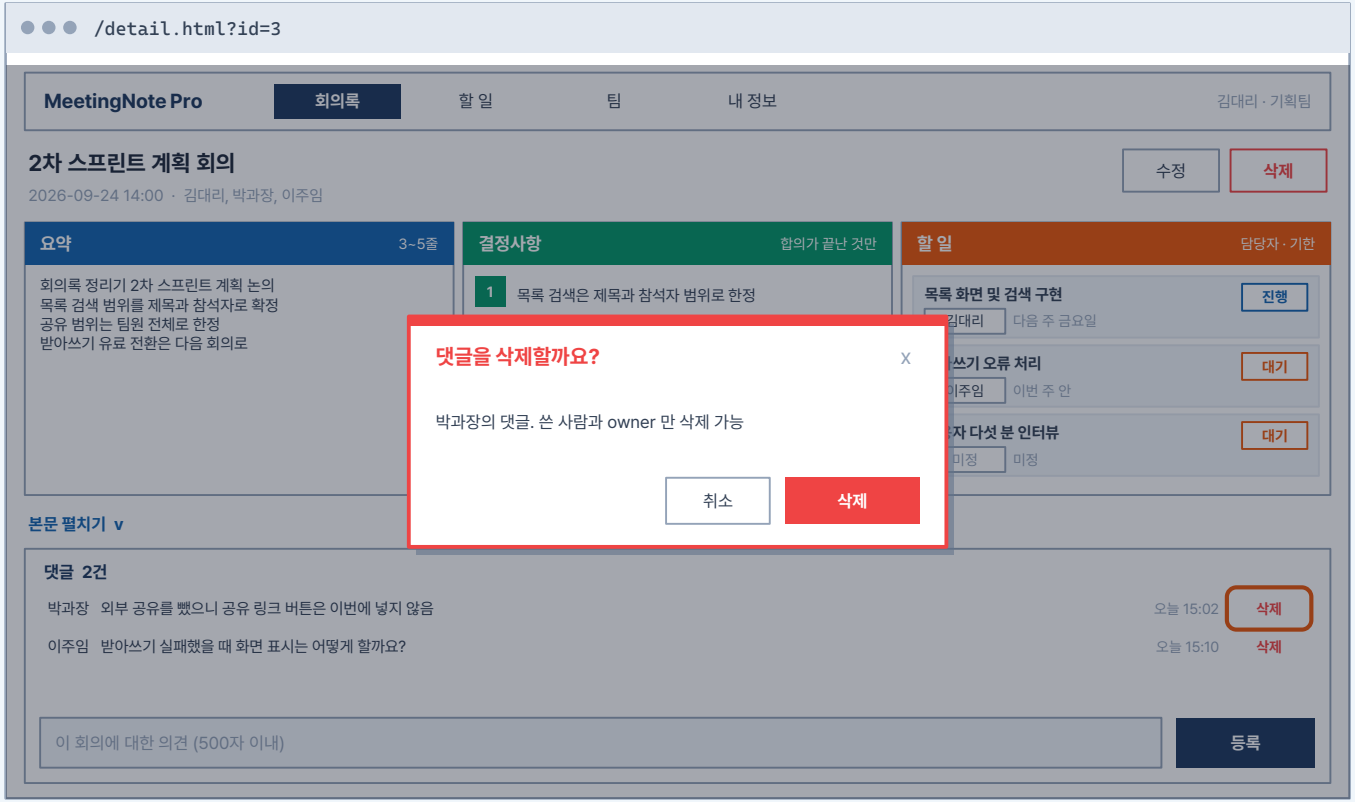
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-12. 회의록 상세 - 댓글 삭제

쓴 사람과 owner 만 지울 수 있음



상태 12 / 13

응답

```
DELETE /api/comments/{id}
204
```

구현 대상 파일

`detail.html`

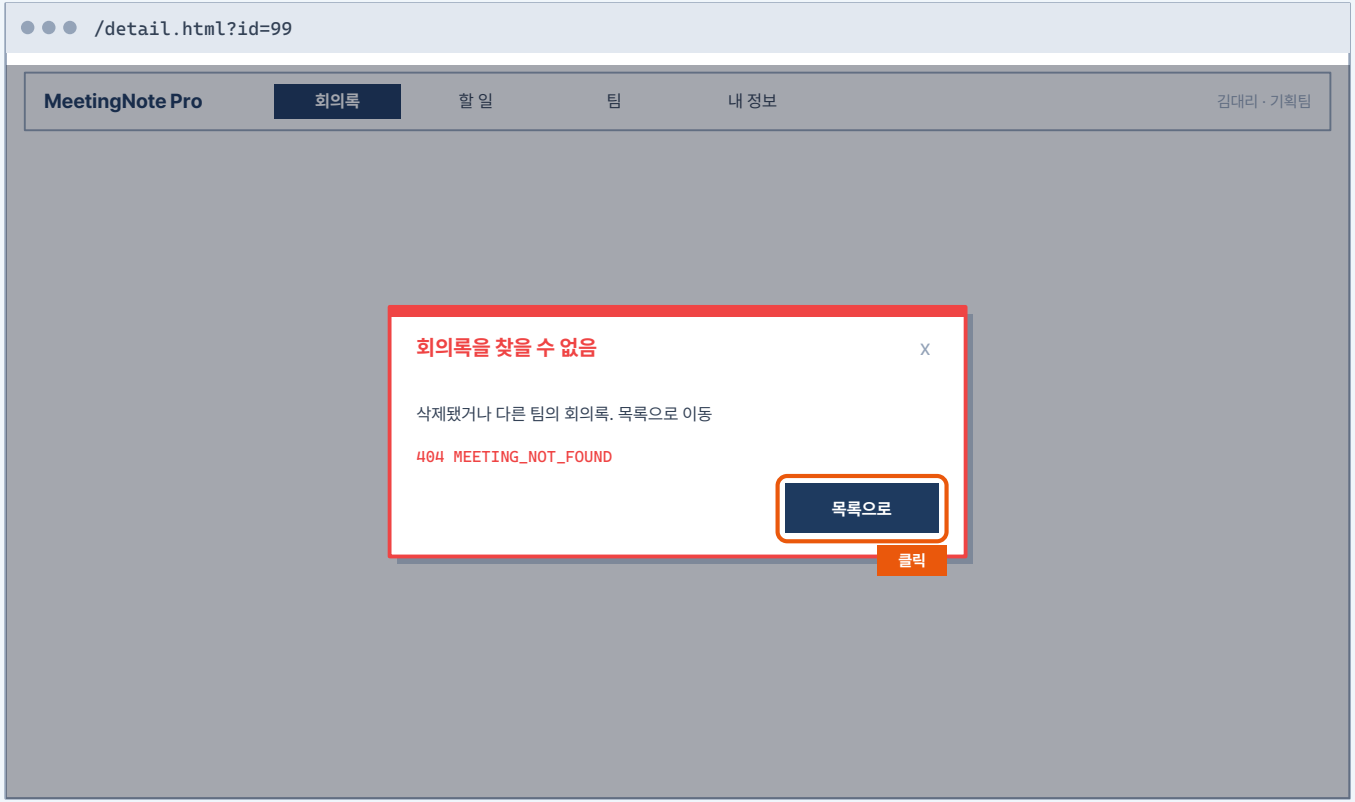
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

D-13. 회의록 상세 - 없는 회의록

목록으로 보냄



상태 13 / 13

응답

`404 MEETING_NOT_FOUND`

구현 대상 파일

`detail.html`

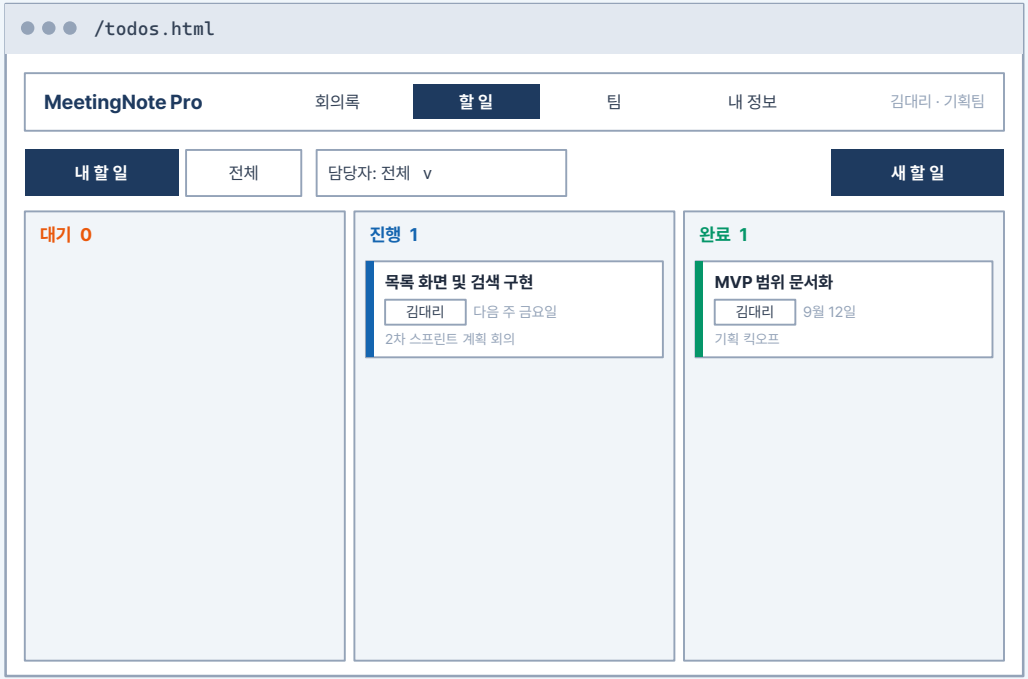
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/meetings/{id}
GET /api/teams/{id}/members
PUT /api/meetings/{id}
DELETE /api/meetings/{id}
PUT /api/todos/{id}
POST /api/meetings/{id}/comments
GET /api/meetings/{id}/comments
DELETE /api/comments/{id}
```

E. 할 일 칸반

할 일의 진행 상태를 보는 화면. 회의록에서 뽑히거나 직접 등록. 카드를 끌어 이동하거나 눌러서 칸 선택



상태 13종

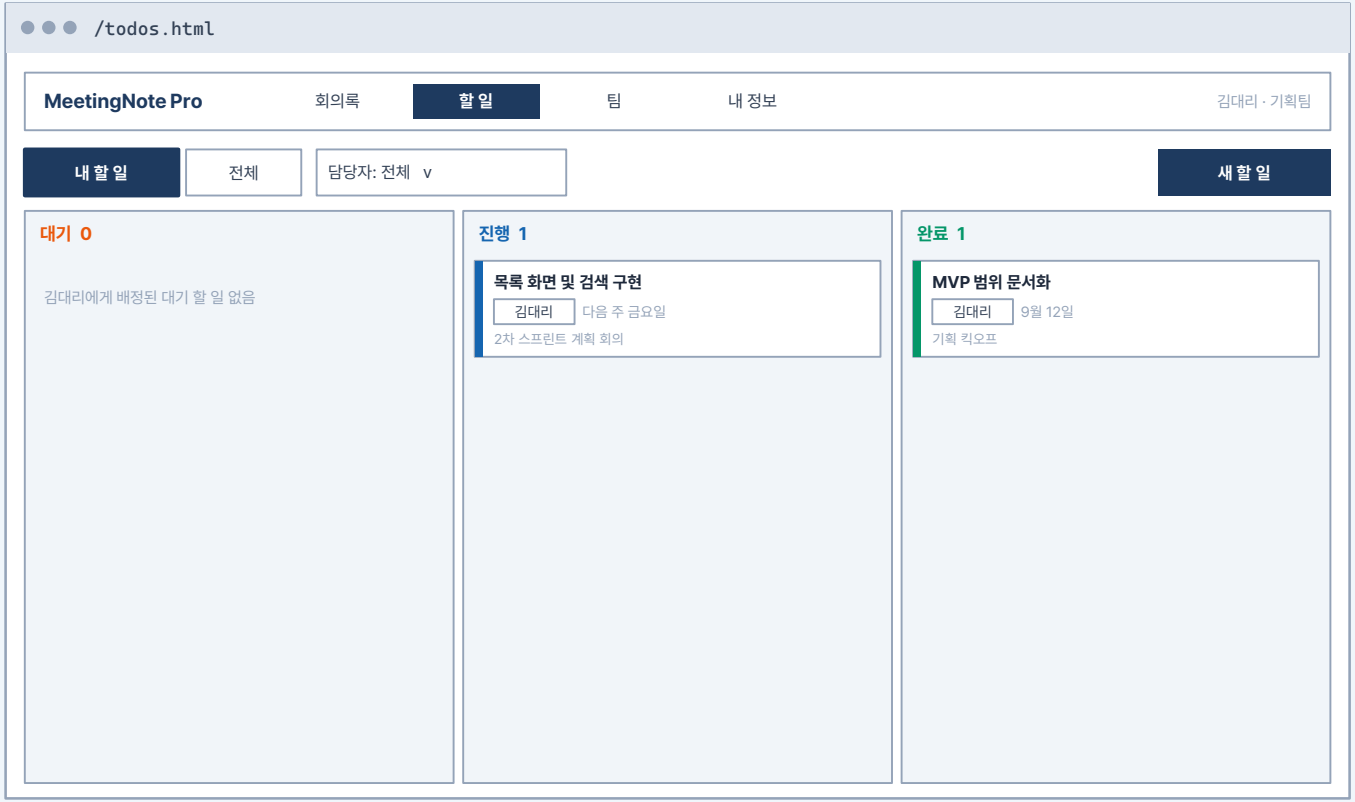
01	내 할 일	`GET /api/me/todos 200`
02	전체	`GET /api/teams/{id}/todos 200`
03	끌어 옮기는 중	
04	이동 완료	`PUT /api/todos/{id} 200`
05	눌러서 칸 고르기	`PUT /api/todos/{id} 200`
06	담당자 배정	`PUT /api/todos/{id} 200`
07	담당자 필터	
08	기한 지남	
09	담당자 미정	
10	완료만	
11	할 일 삭제	`DELETE /api/todos/{id} 204`
12	빈 상태	`200 + 빈 배열`
13	직접 등록	`POST /api/teams/{id}/todos 201`

쓰이는 엔드포인트 `GET /api/teams/{id}/todos` · `POST /api/teams/{id}/todos` · `PUT /api/todos/{id}` · `GET /api/me/todos` · `DELETE /api/todos/{id}` · `GET /api/teams/{id}/members`

상태 13종: 정상 흐름뿐 아니라 빈 상태 · 오류 · 처리 중까지 화면으로 고정

E-01. 할 일 칸반 - 내 할 일

로그인한 사람에게 배정된 것만



상태 1 / 13

응답

`GET /api/me/todos 200`

구현 대상 파일

`todos.html`

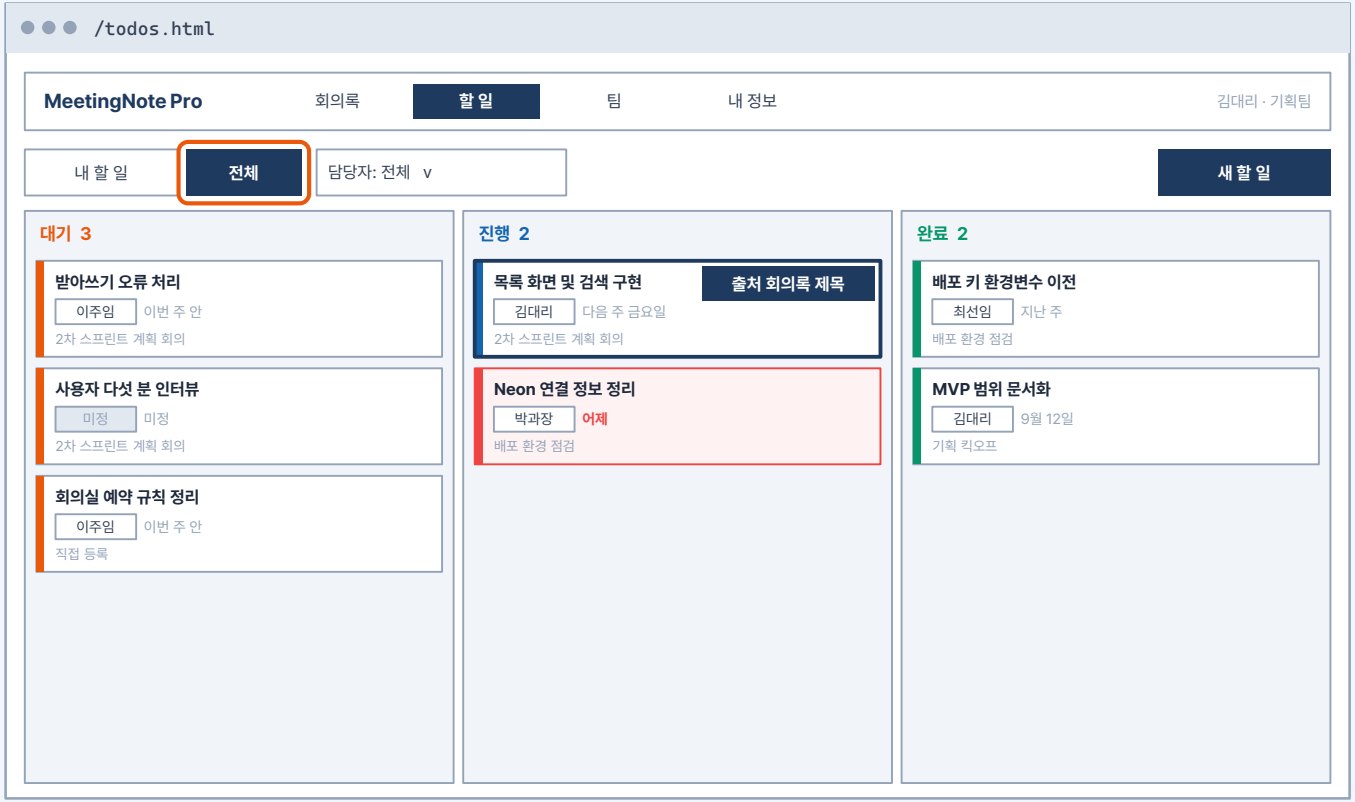
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-02. 할 일 칸반 - 전체

팀 전체. 회의록 제목이 함께 옴



상태 2 / 13

응답

```
GET /api/teams/{id}/todos
200
```

구현 대상 파일

`todos.html`

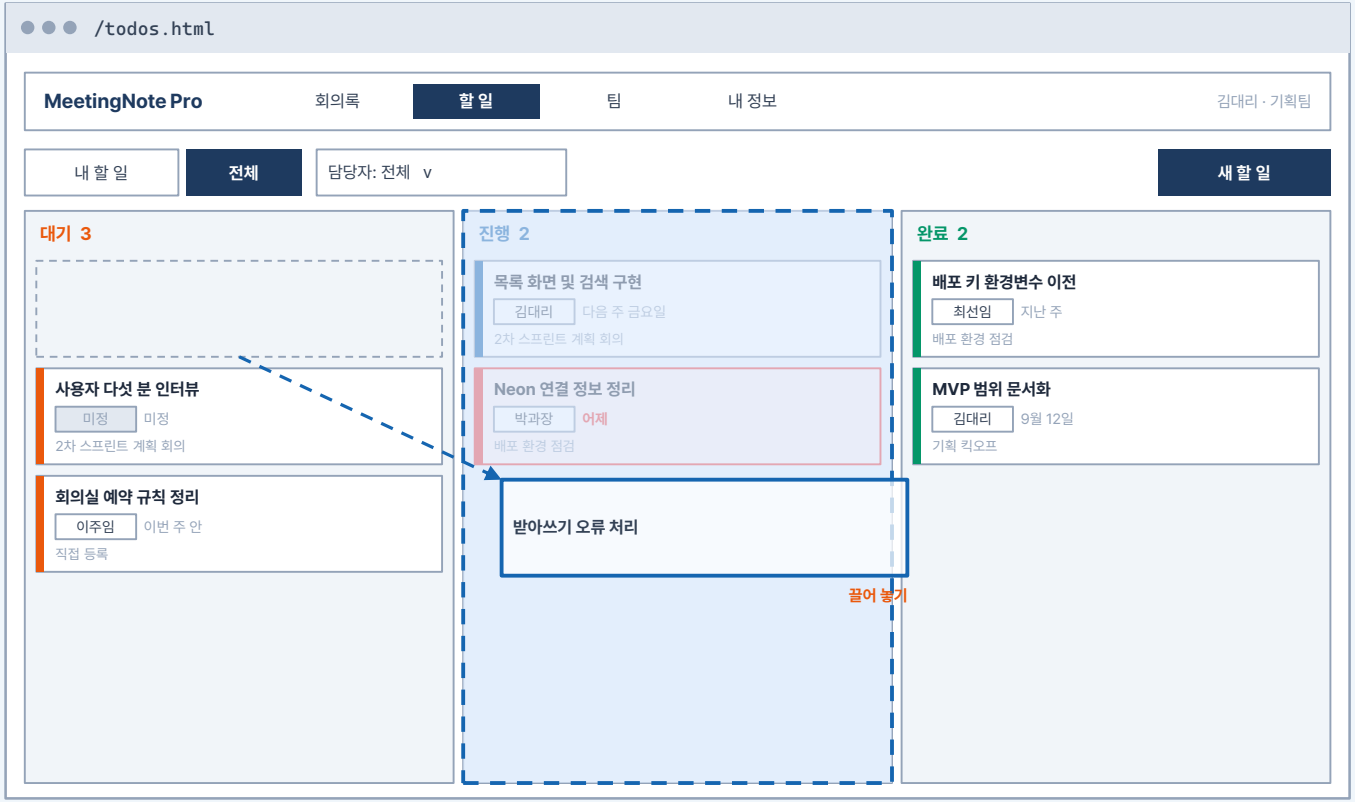
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-03. 할 일 칸반 - 끌어 옮기는 중

카드를 집으면 반투명 사본이 따라붙고, 놓을 칸이 점선으로 변경



상태 3 / 13

응답

–

구현 대상 파일

`todos.html`

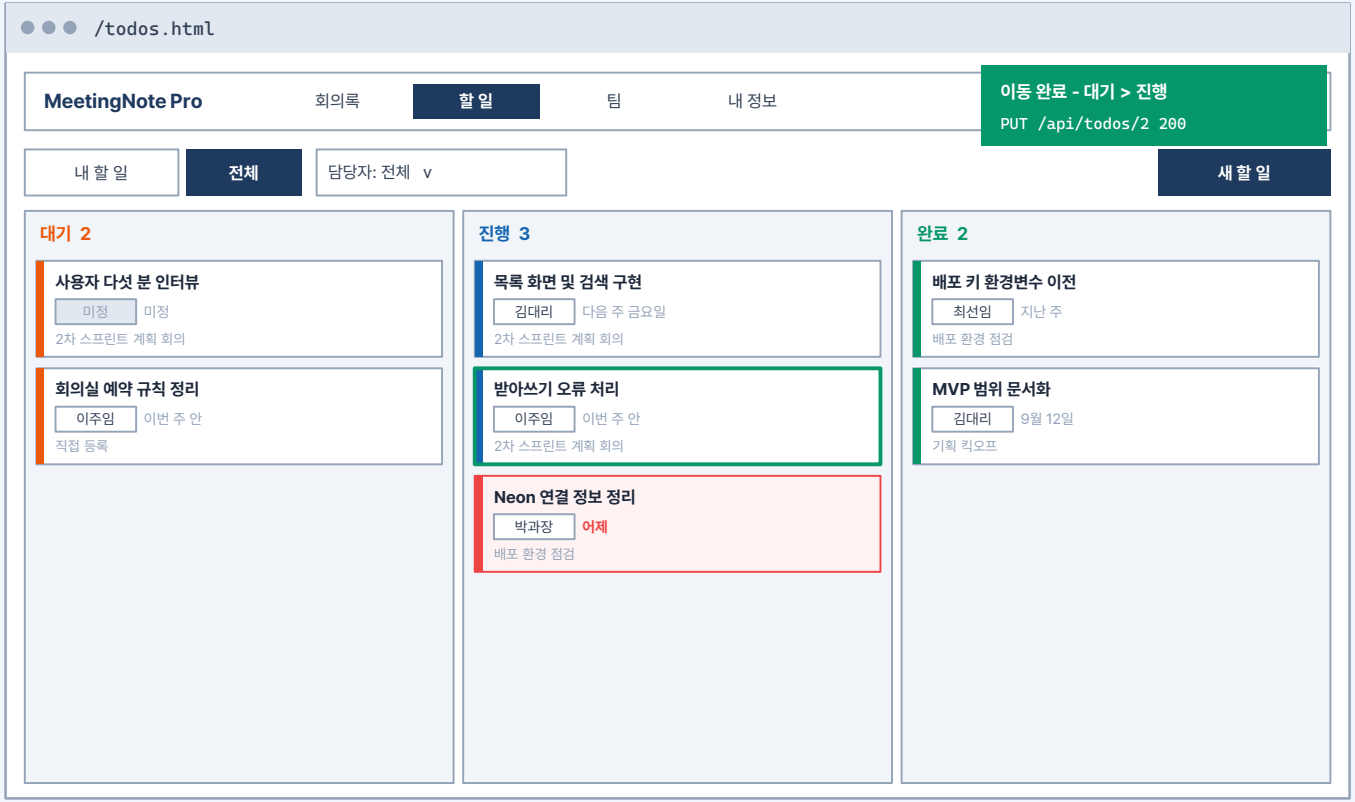
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-04. 할 일 칸반 - 이동 완료

놓는 순간 한 번만 호출. 칸의 빈 자리에 놓아도 이동. 실패하면 카드가 원래 칸으로



상태 4 / 13

응답

`PUT /api/todos/{id} 200`

구현 대상 파일

`todos.html`

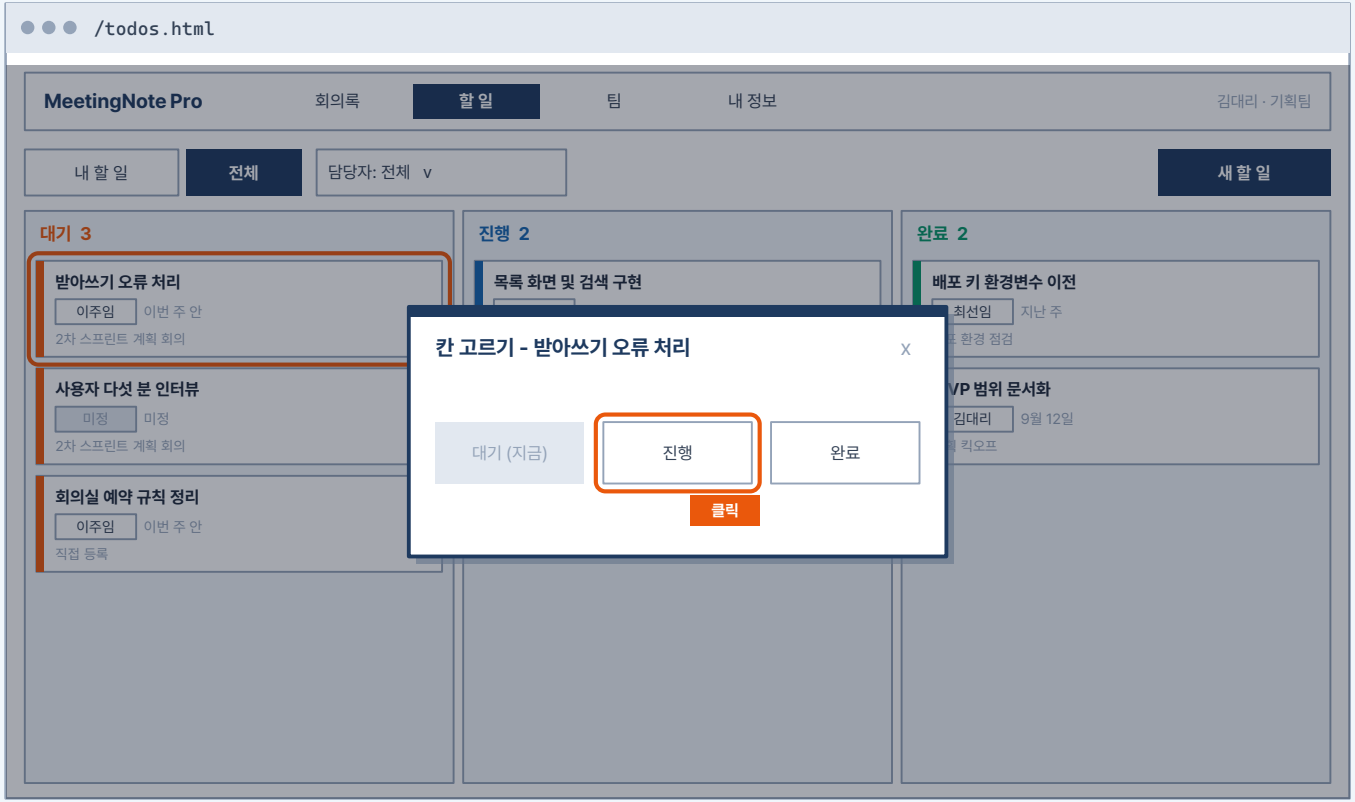
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-05. 할 일 칸반 - 눌러서 칸 고르기

좁은 화면은 칸이 세로로 쌓여 끌 수 없음. 카드를 누르면 대기 · 진행 · 완료 버튼이 뜸



상태 5 / 13

응답

`PUT /api/todos/{id} 200`

구현 대상 파일

`todos.html`

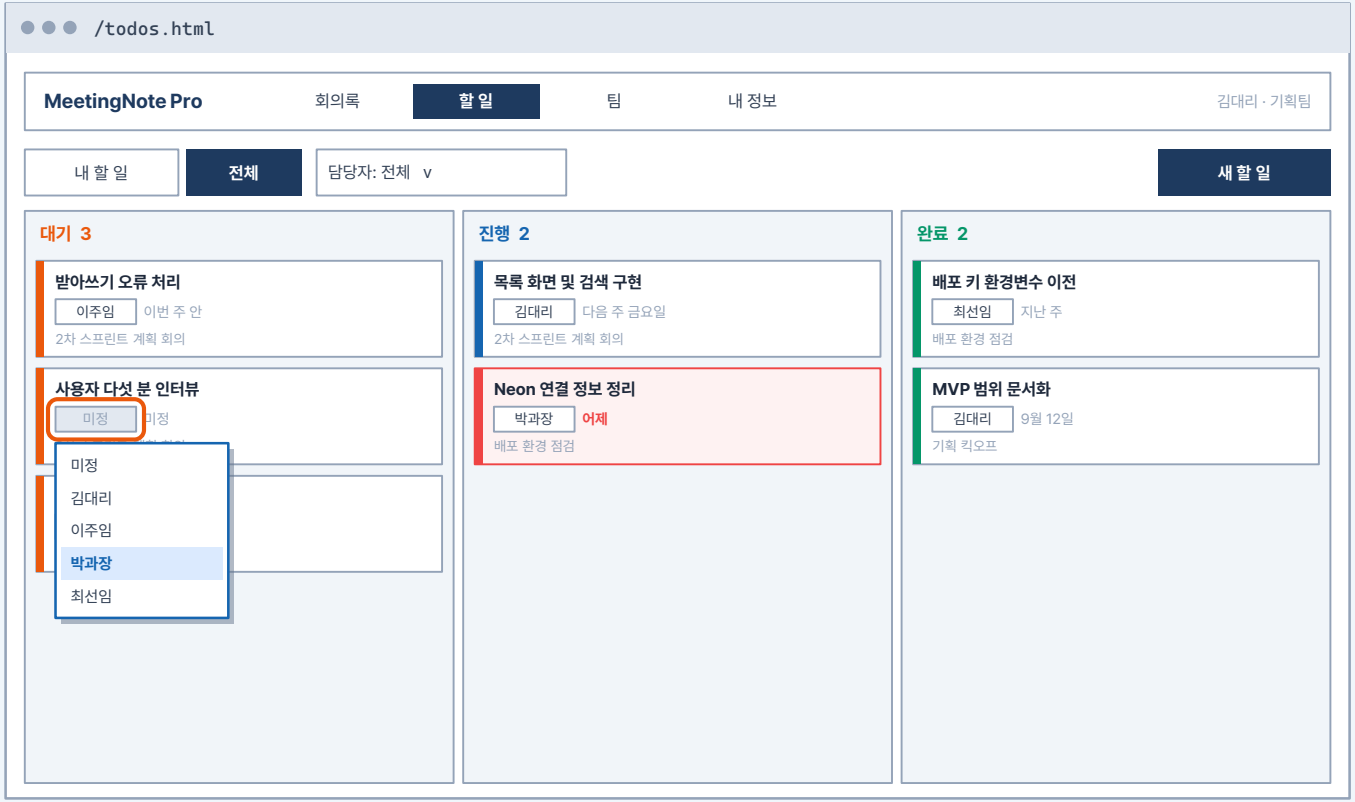
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-06. 할 일 칸반 - 담당자 배정

배지를 눌러 담당자를 바꿈. 미정도 한 값



상태 6 / 13

응답

`PUT /api/todos/{id} 200`

구현 대상 파일

`todos.html`

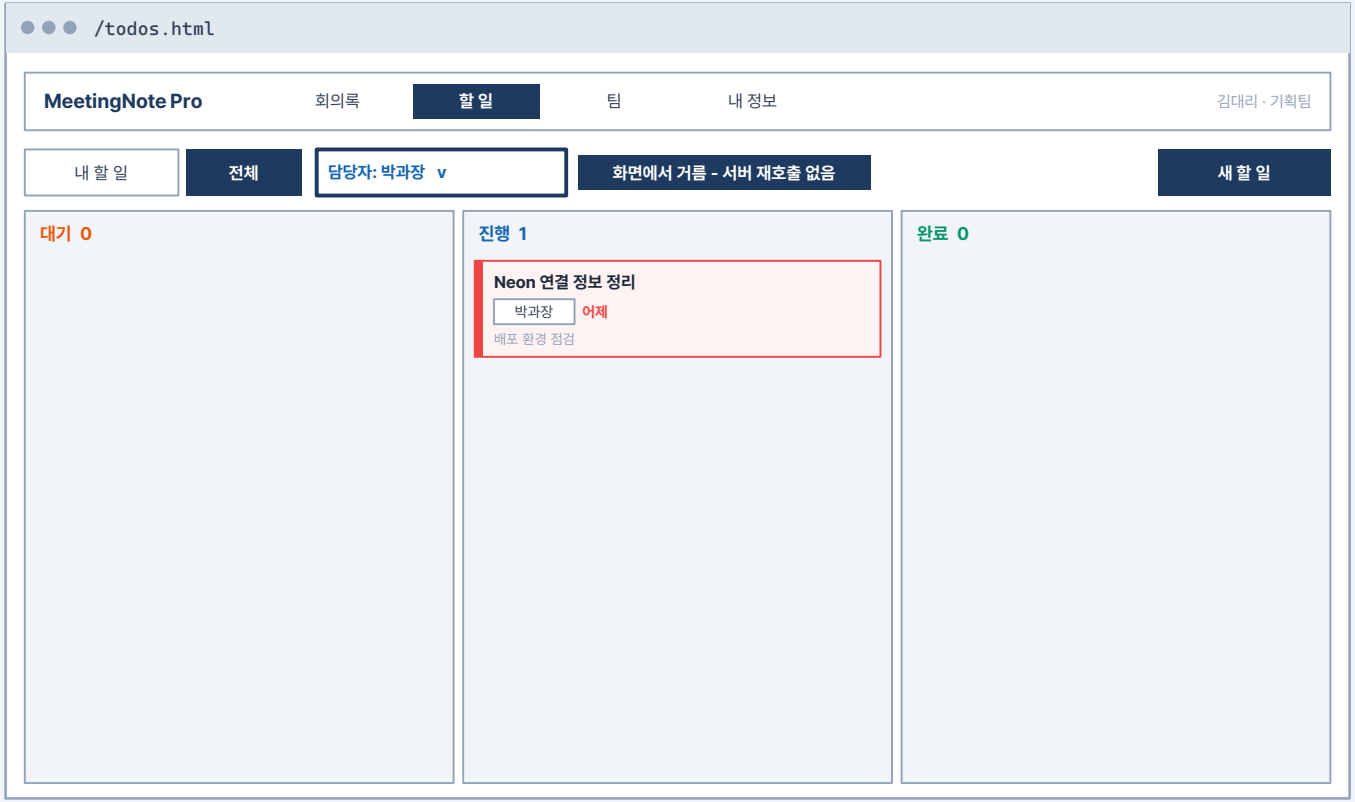
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-07. 할 일 칸반 - 담당자 필터

화면에서 거름. 서버를 다시 부르지 않음



상태 7 / 13

응답

–

구현 대상 파일

`todos.html`

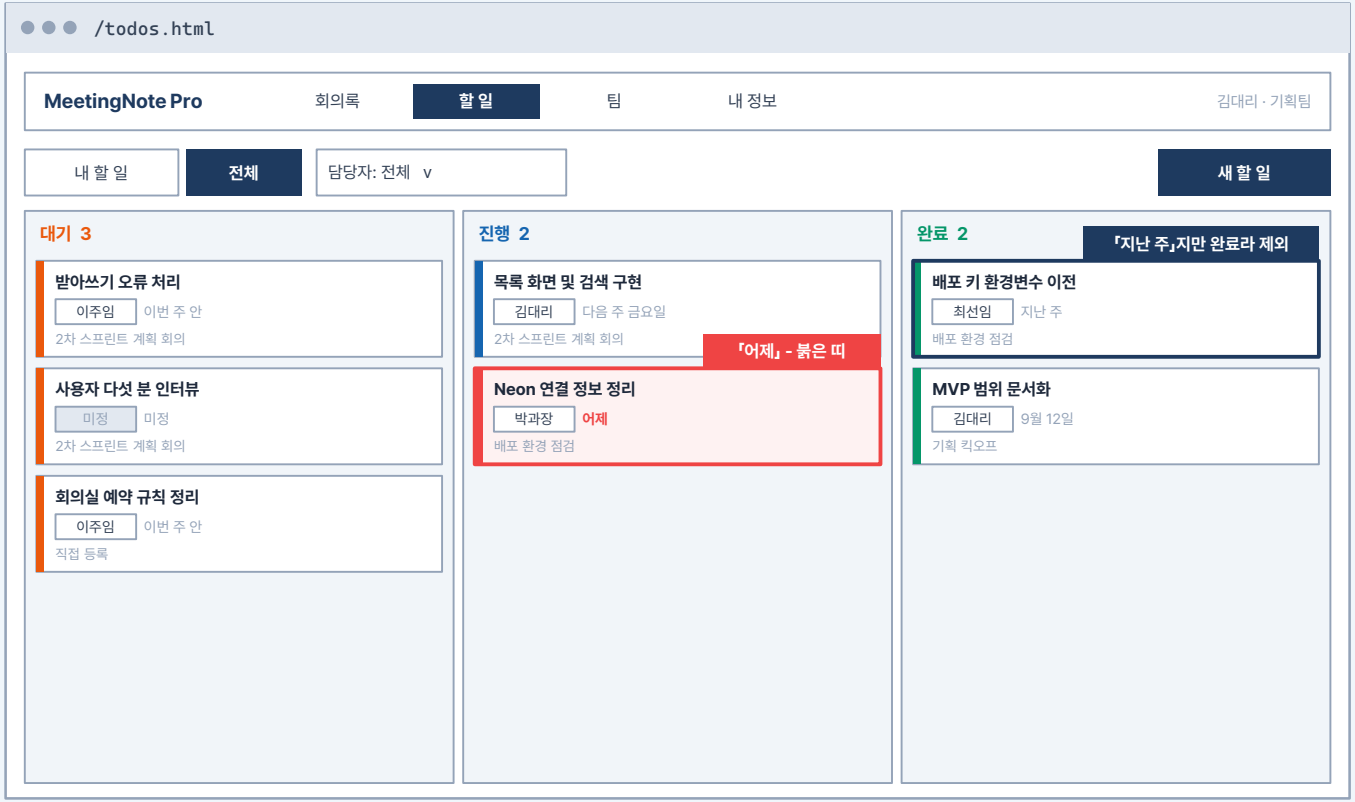
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-08. 할 일 칸반 - 기한 지남

화면이 판정. 「어제 · 지난 주 · 지난 달」 셋만 붉은 띠. 그 밖의 글자는 판정하지 않음. 완료는 제외



상태 8 / 13

응답

–

구현 대상 파일

`todos.html`

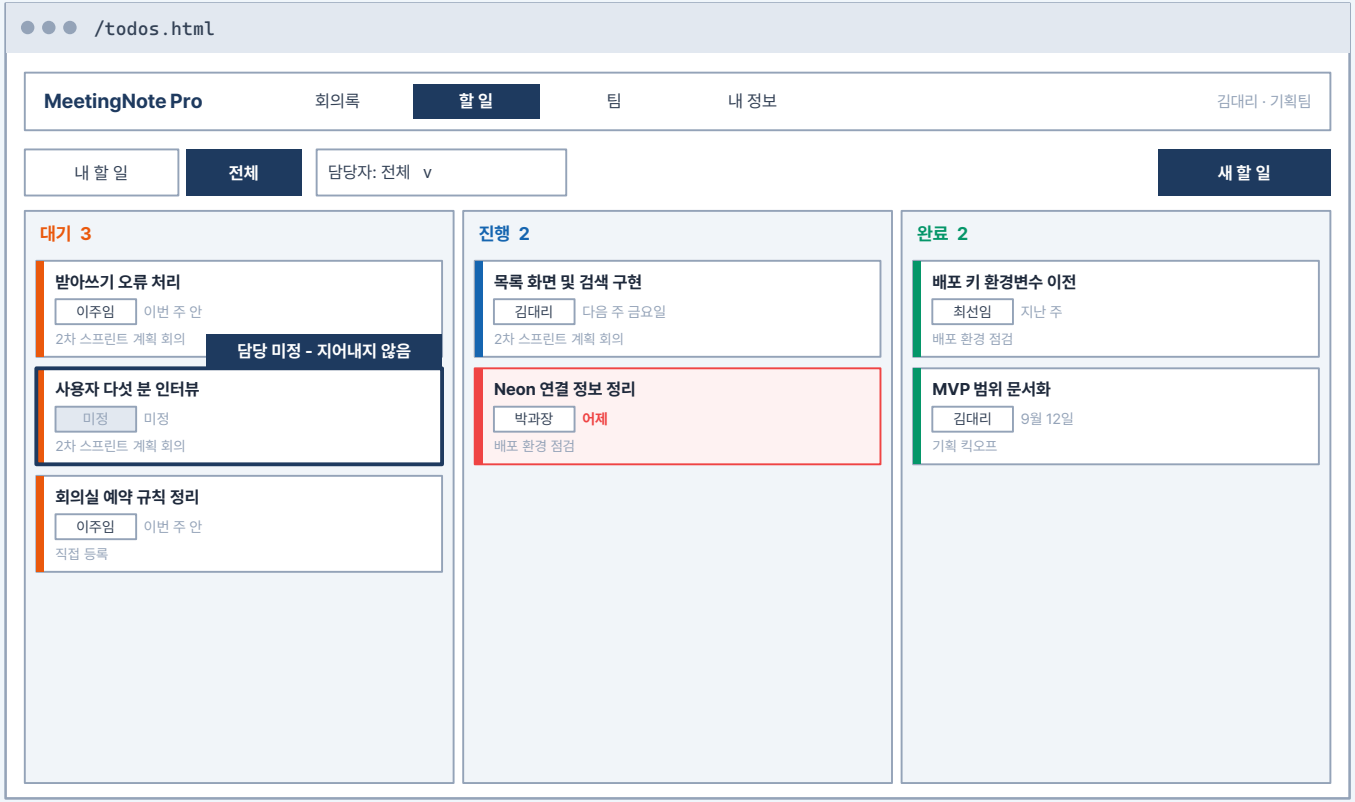
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-09. 할 일 칸반 - 담당자 미정

회의에서 정해지지 않은 것. 지어내지 않음



상태 9 / 13

응답

–

구현 대상 파일

`todos.html`

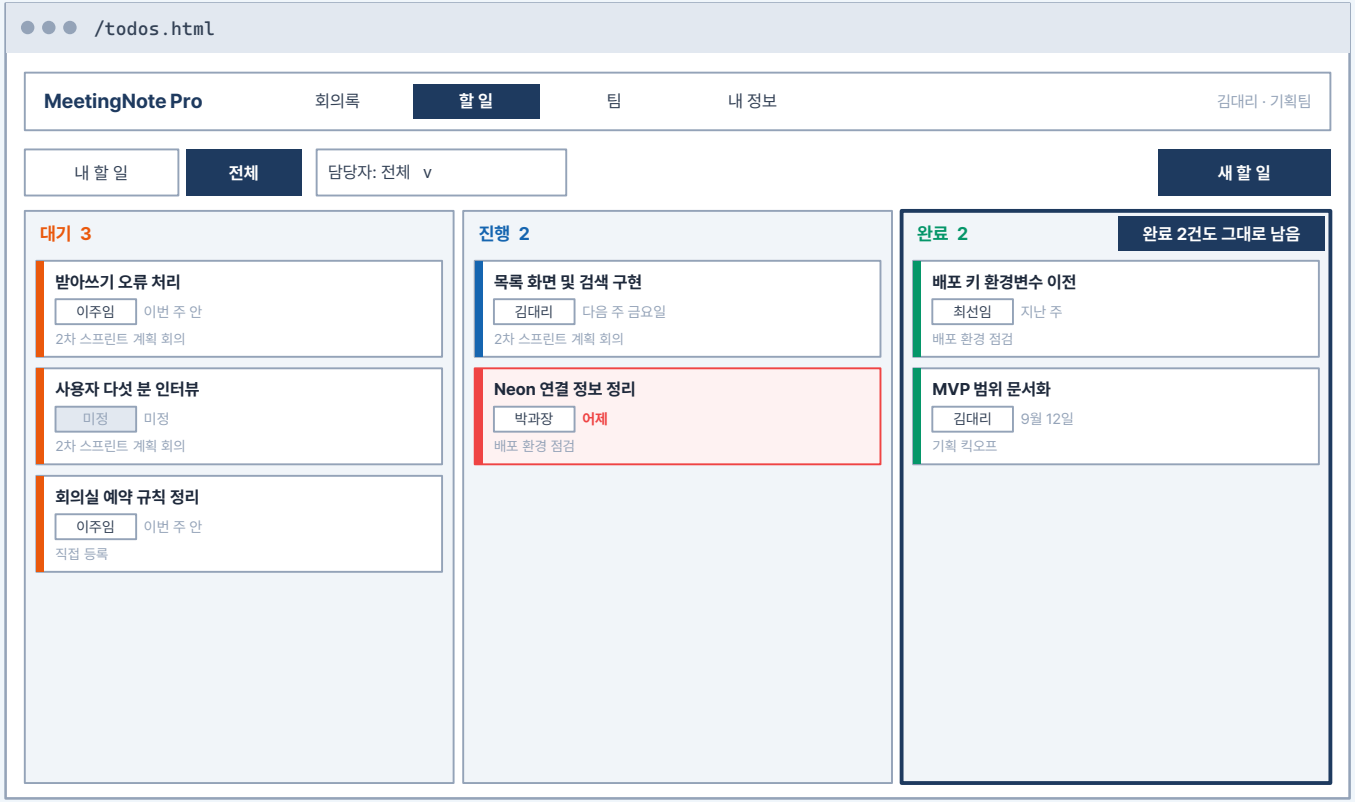
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-10. 할 일 칸반 - 완료만

완료도 지우지 않고 남김. 활동 기록의 근거



상태 10 / 13

응답

–

구현 대상 파일

`todos.html`

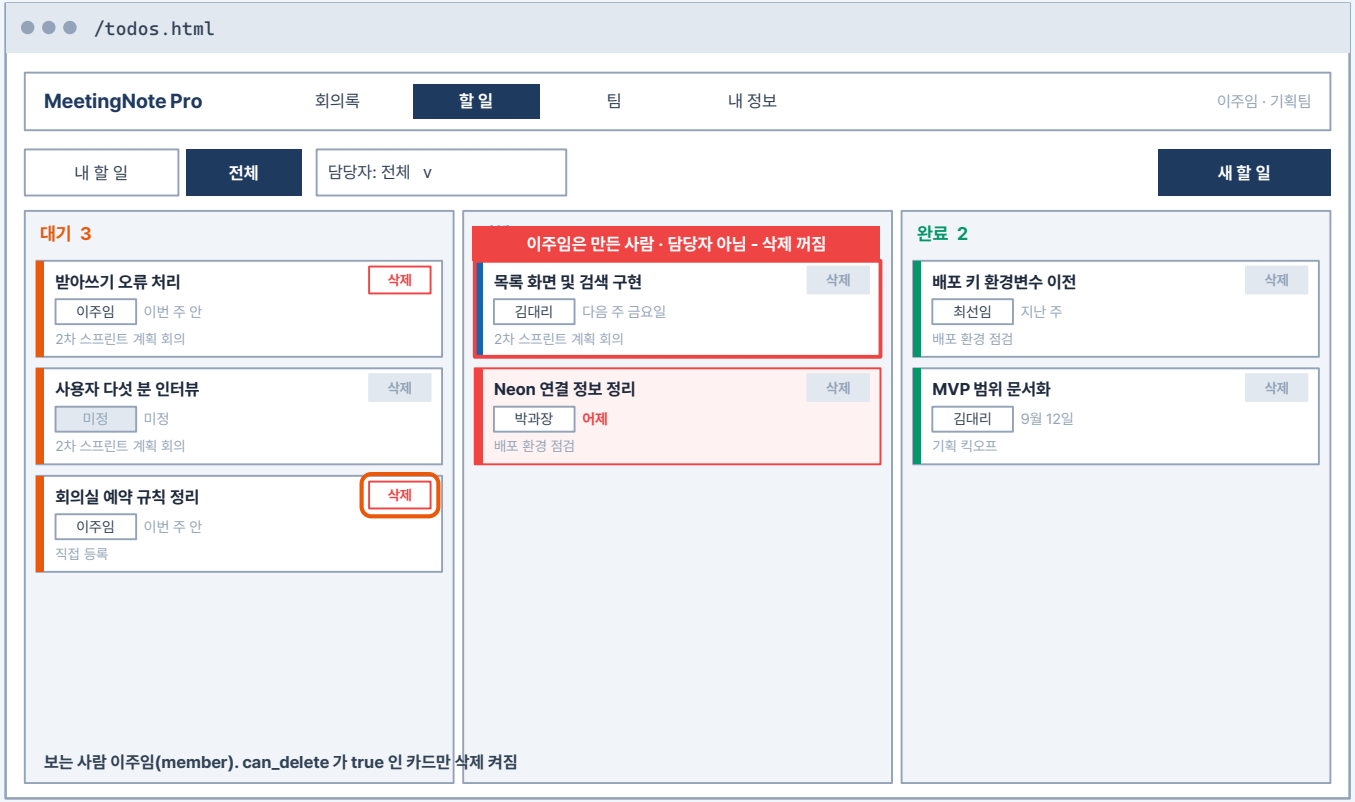
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-11. 할 일 칸반 - 할 일 삭제

만든 사람 · 담당자 · owner 만. 목록 응답의 can_delete 로 버튼 비활성 판정, 부르면 403 FORBIDDEN



상태 11 / 13

응답

```
DELETE /api/todos/{id} 204
```

구현 대상 파일

`todos.html`

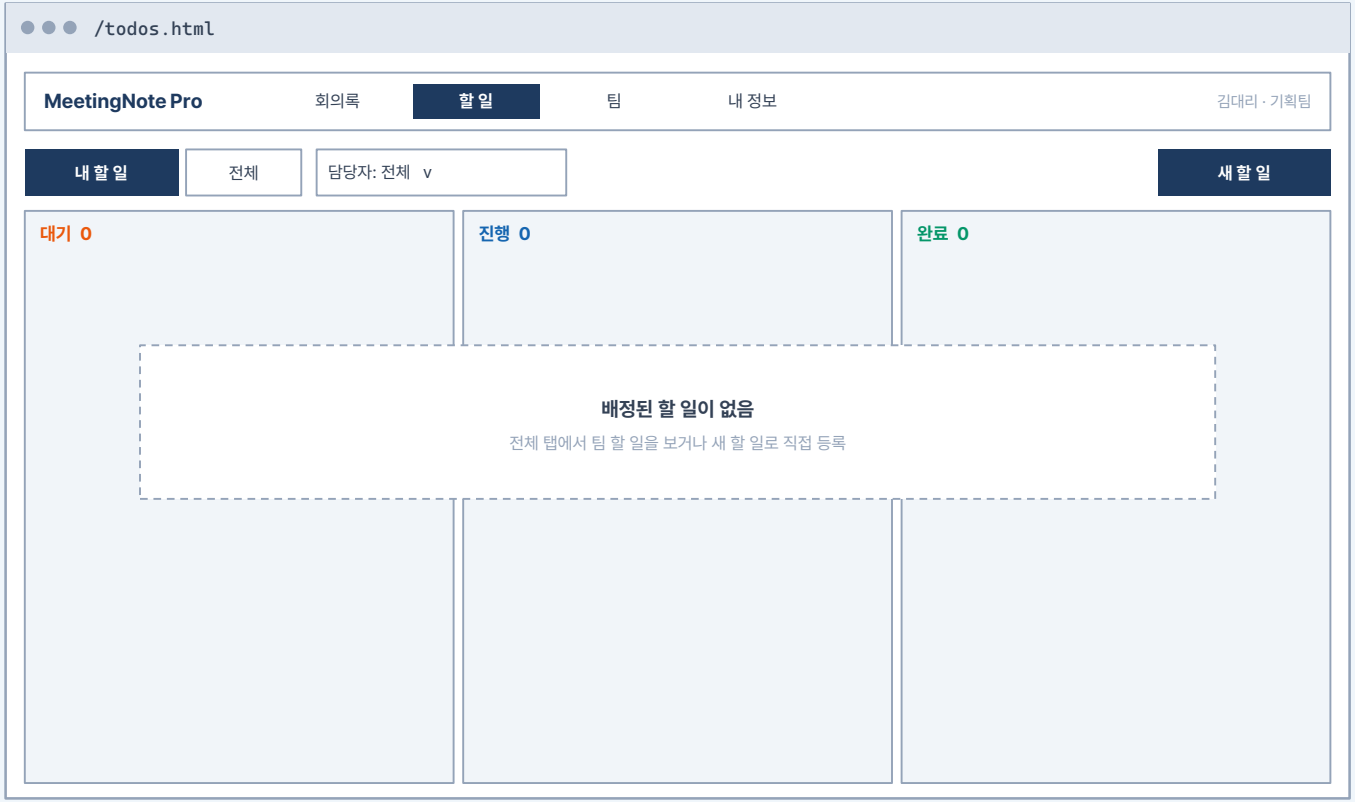
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-12. 할 일 칸반 - 빈 상태

배정된 할 일 0건



상태 12 / 13

응답

`200 + 빈 배열`

구현 대상 파일

`todos.html`

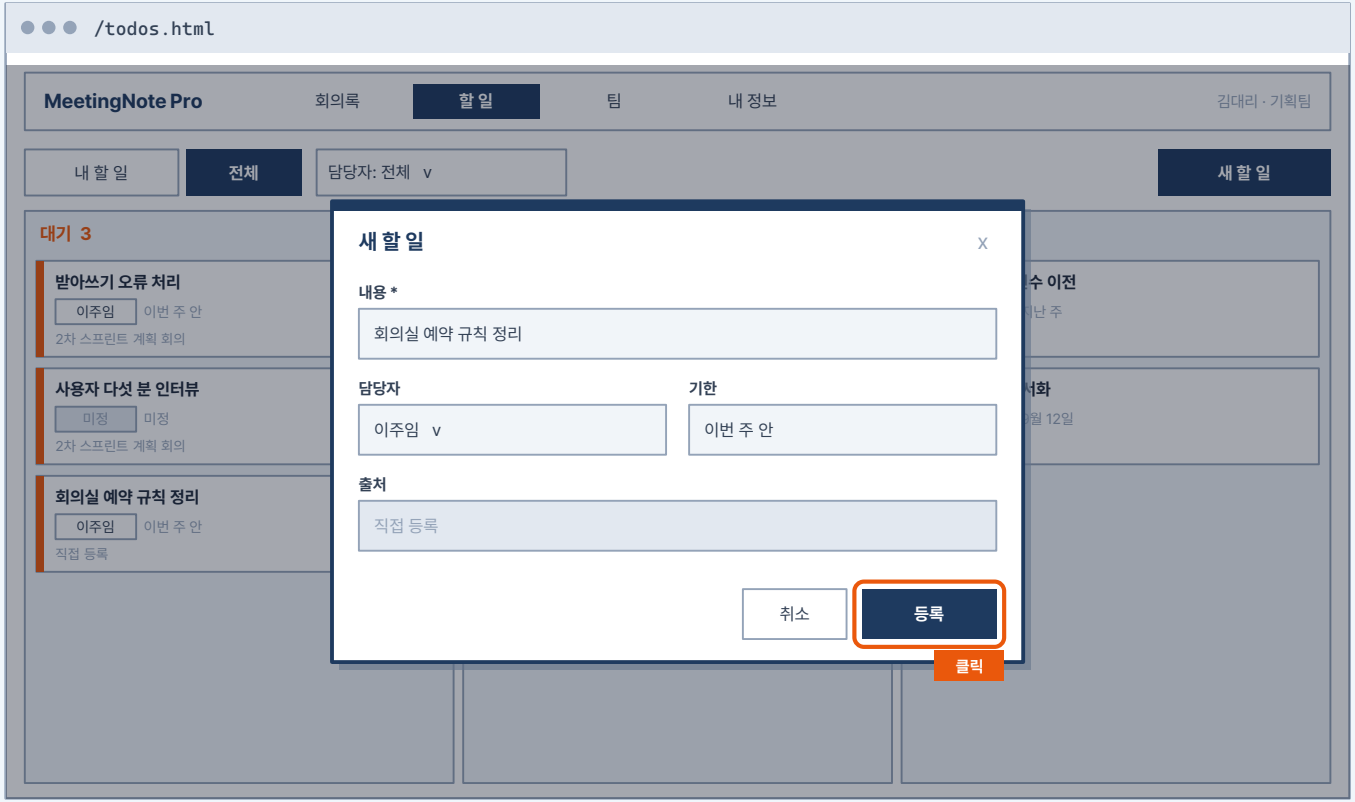
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

E-13. 할 일 칸반 - 직접 등록

회의록 없이 칸반에서 바로 등록. 내용 · 담당자 · 기한. 카드 출처는 「직접 등록」



상태 13 / 13

응답

`POST /api/teams/{id}/todos`
`201`

구현 대상 파일

`todos.html`

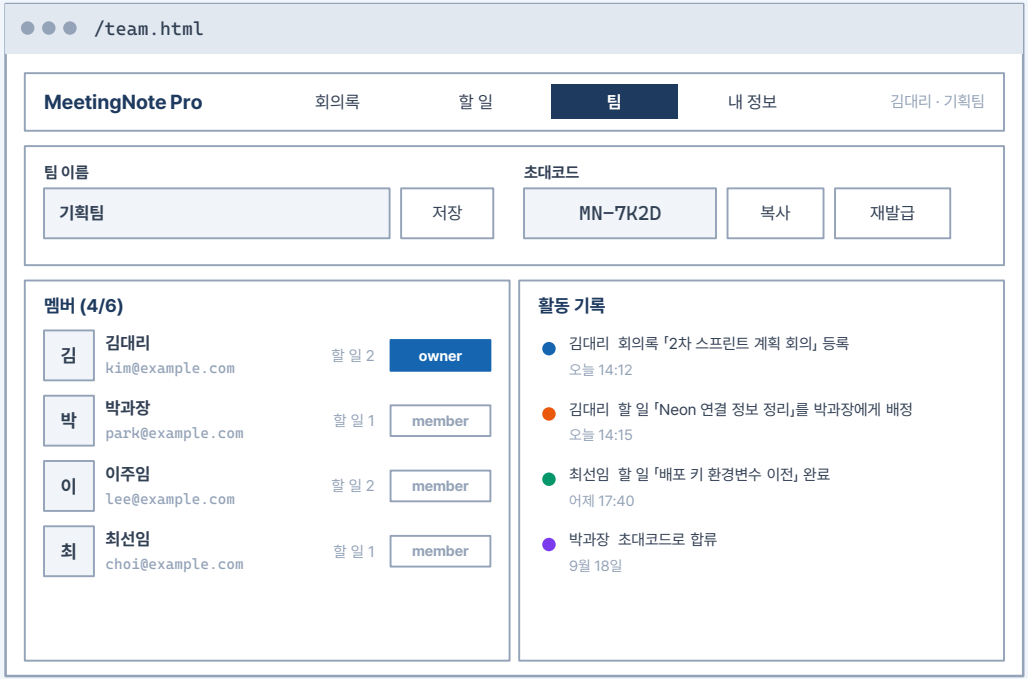
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/teams/{id}/todos
POST /api/teams/{id}/todos
PUT /api/todos/{id}
GET /api/me/todos
DELETE /api/todos/{id}
GET /api/teams/{id}/members
```

F. 팀 설정

초대코드 하나로 팀 생성과 합류가 모두 처리됨



상태 8종

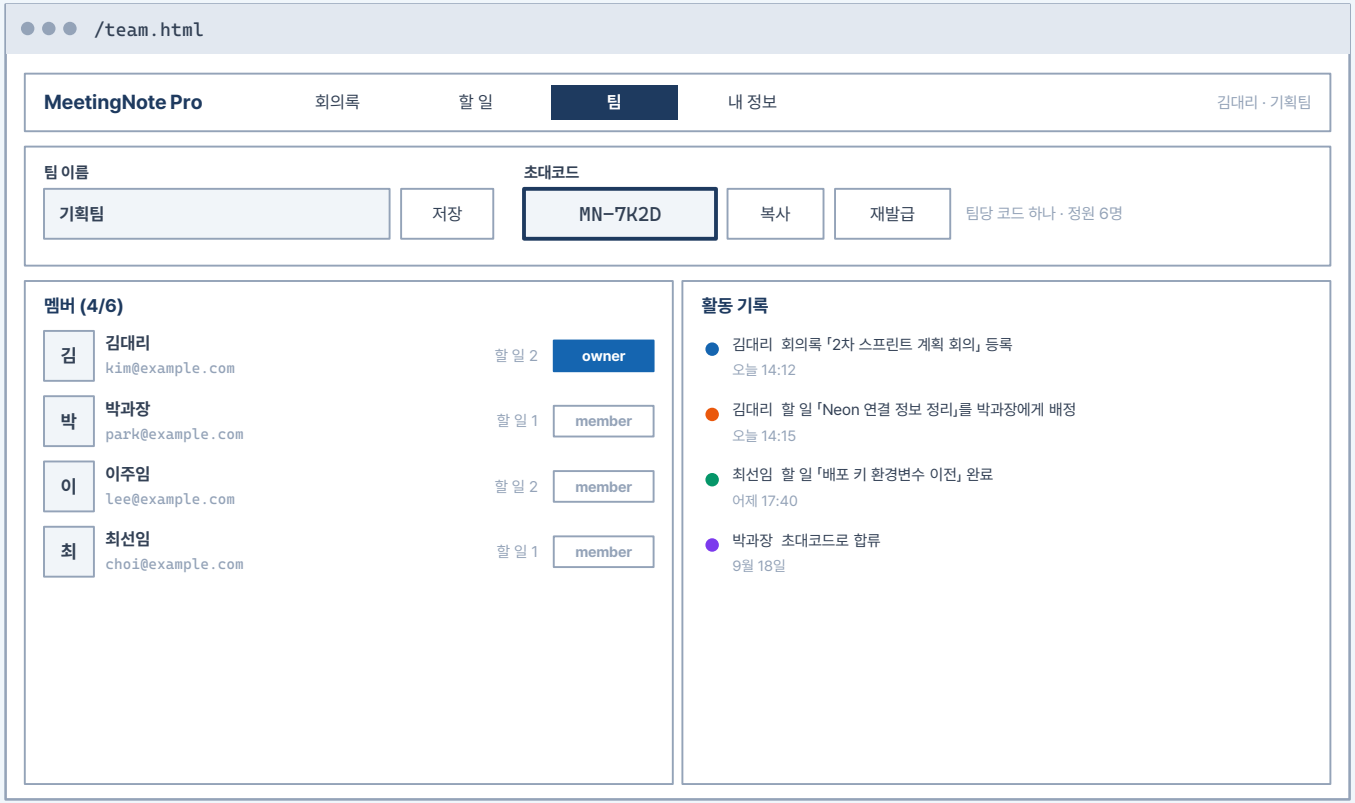
01	팀 설정	200
02	소속 팀 없음	
03	초대코드 오류	404 INVITE_NOT_FOUND
04	코드 복사	
05	코드 재발급	PUT 200
06	정원 초과	409 TEAM_FULL
07	팀 이름 저장	PUT /api/teams/{id} 200
08	member 권한	403 OWNER_ONLY

쓰이는 엔드포인트 `POST /api/teams` · `GET /api/teams` · `POST /api/teams/join` · `GET /api/teams/{id}/members` · `PUT /api/teams/{id}` · `PUT /api/teams/{id}/code` · `GET /api/teams/{id}/activities`

상태 8종: 정상 흐름뿐 아니라 빈 상태 · 오류 · 처리 중까지 화면으로 고정

F-01. 팀 설정 - 팀 설정

팀 이름(owner 만 수정) · 초대코드 · 멤버 · 활동 기록



상태 1 / 8

응답

200

구현 대상 파일

`team.html`

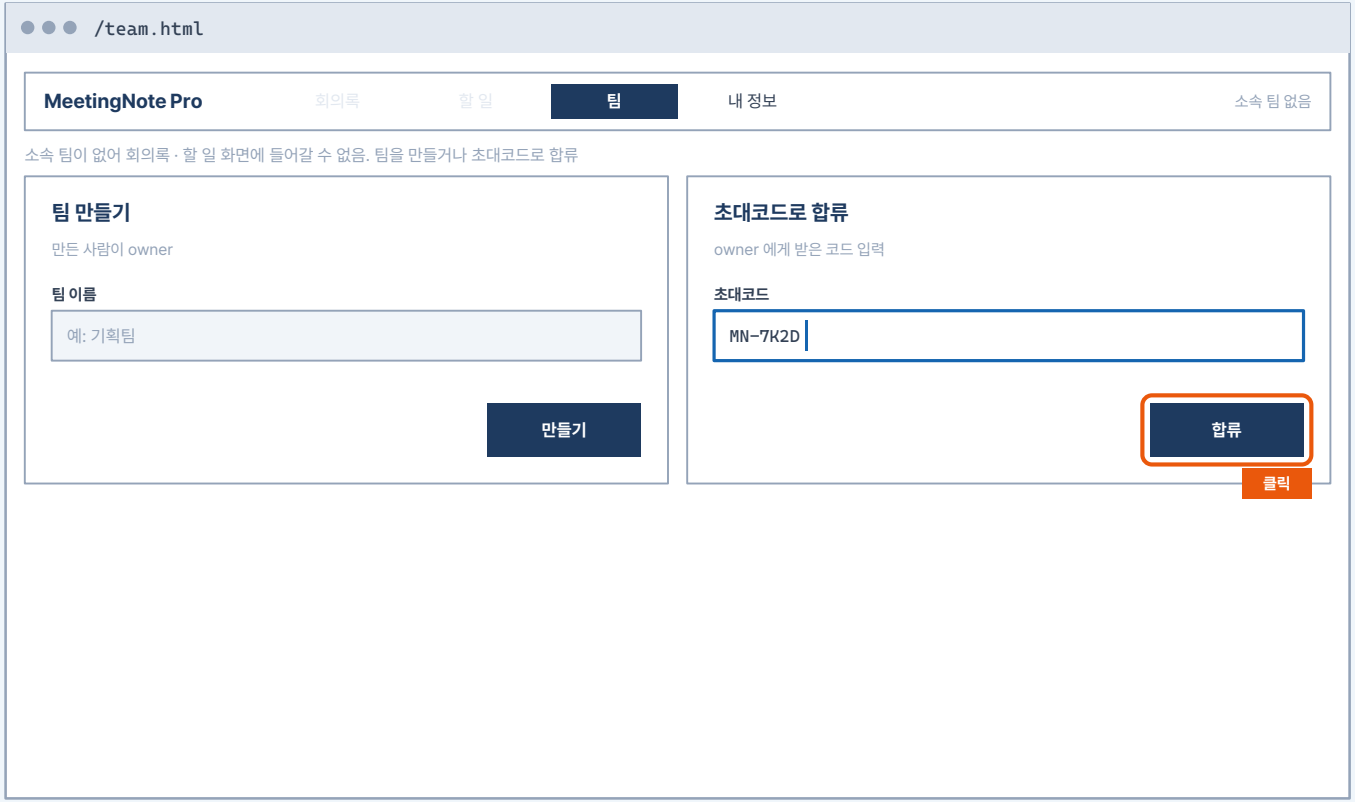
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/teams
GET /api/teams
POST /api/teams/join
GET /api/teams/{id}/members
PUT /api/teams/{id}
PUT /api/teams/{id}/code
GET /api/teams/{id}/activities
```

F-02. 팀 설정 - 소속 팀 없음

로그인 직후 소속 팀 0개인 상태의 화면. 회의록으로 넘어가지 못함



상태 2 / 8

응답

–

구현 대상 파일

`team.html`

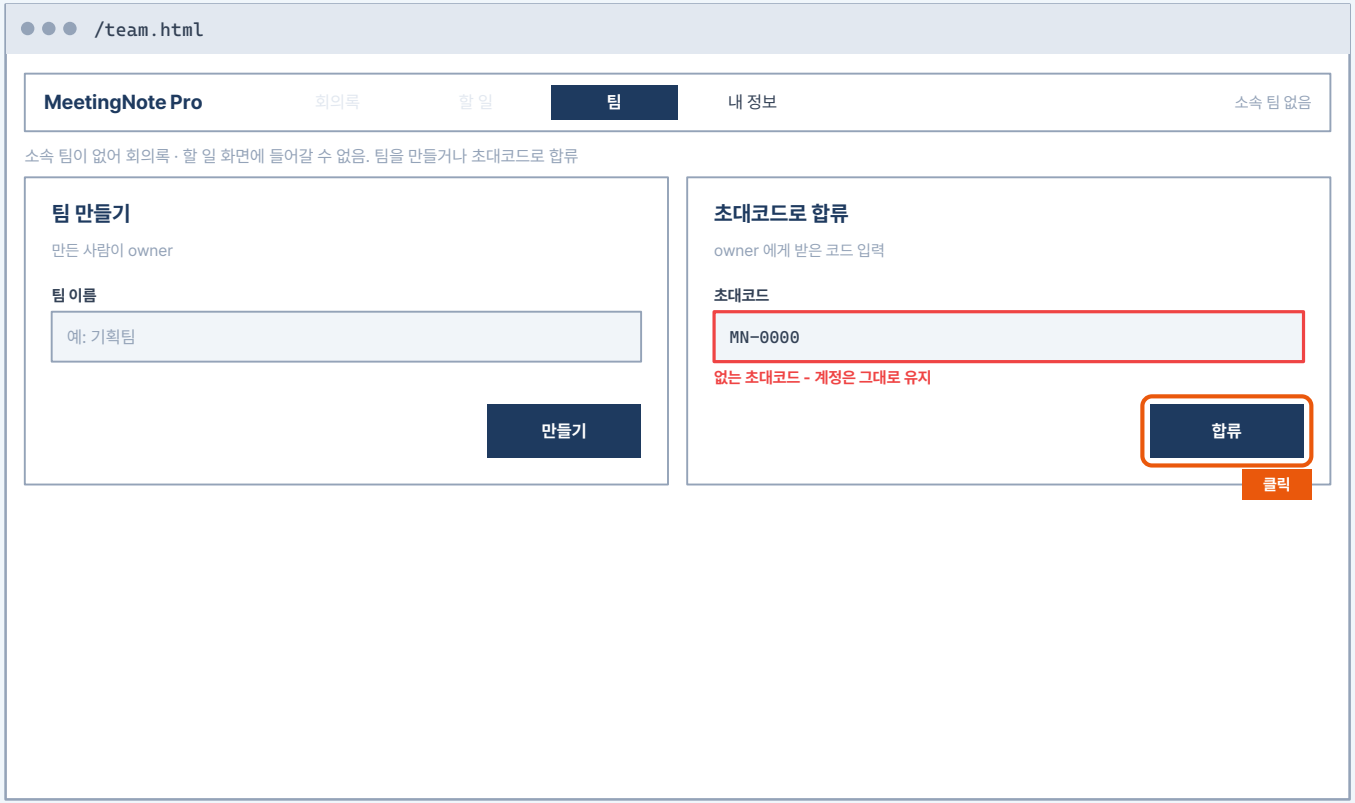
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/teams
GET /api/teams
POST /api/teams/join
GET /api/teams/{id}/members
PUT /api/teams/{id}
PUT /api/teams/{id}/code
GET /api/teams/{id}/activities
```

F-03. 팀 설정 - 초대코드 오류

가입 계정은 그대로 유지



상태 3 / 8

응답

`404 INVITE_NOT_FOUND`

구현 대상 파일

`team.html`

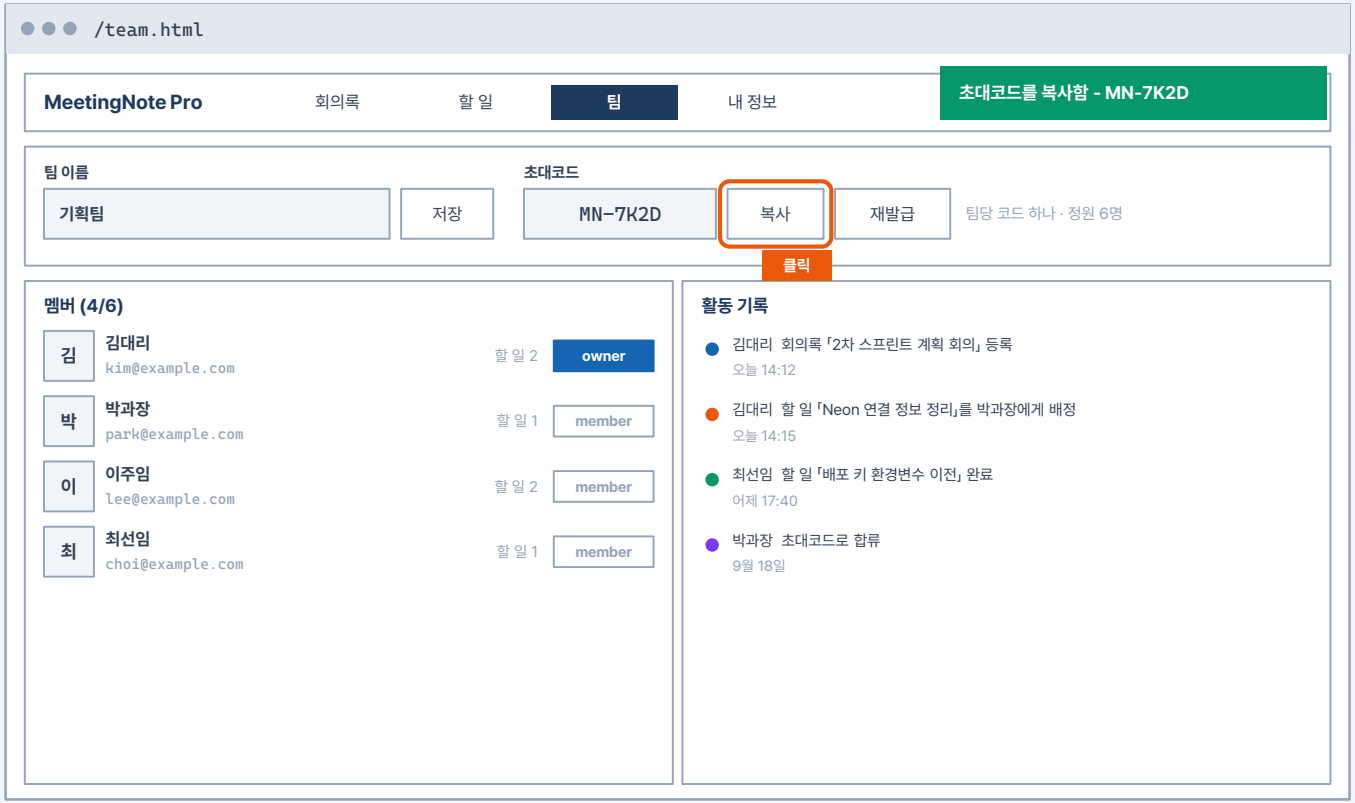
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/teams
GET /api/teams
POST /api/teams/join
GET /api/teams/{id}/members
PUT /api/teams/{id}
PUT /api/teams/{id}/code
GET /api/teams/{id}/activities
```

F-04. 팀 설정 - 코드 복사

팀당 코드 하나. 사람마다 따로 발급하지 않음



상태 4 / 8

응답

–

구현 대상 파일

`team.html`

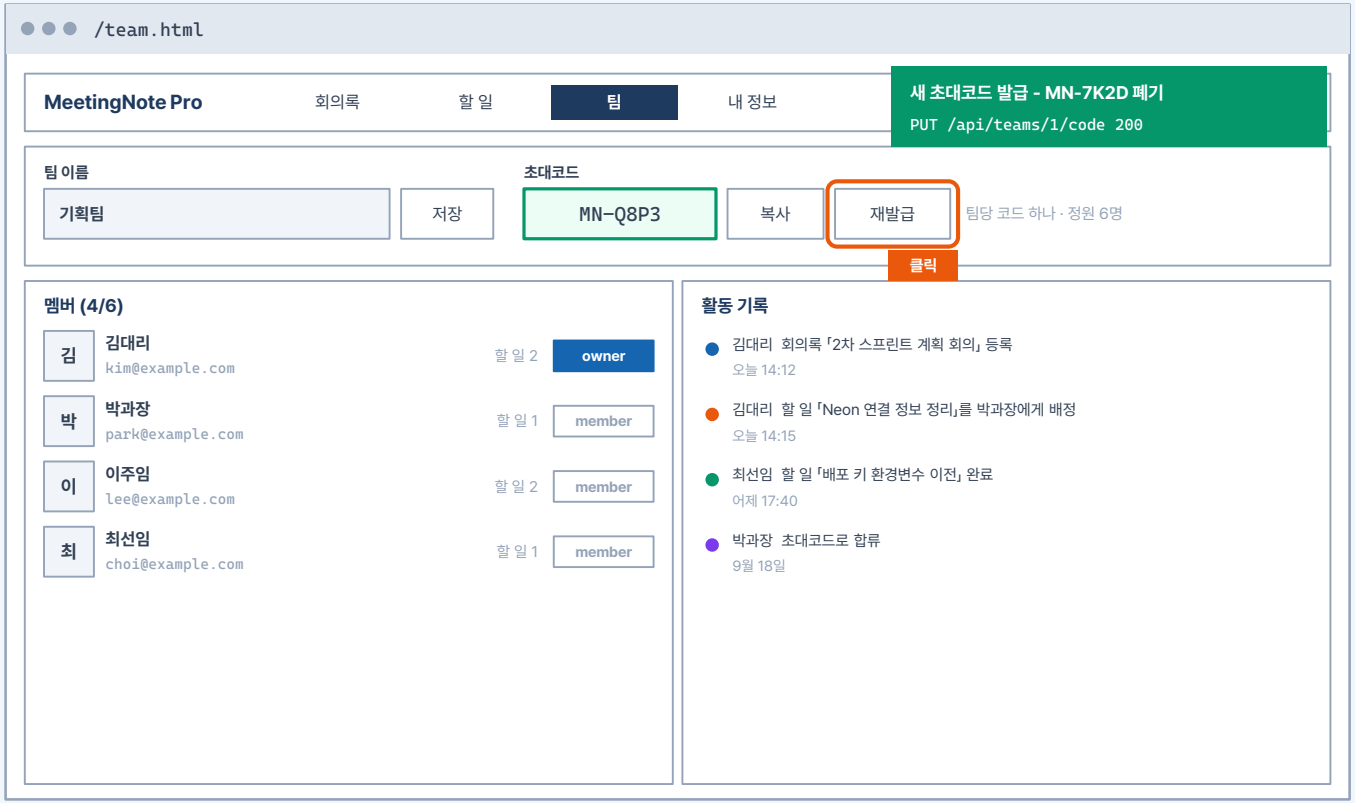
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/teams
GET /api/teams
POST /api/teams/join
GET /api/teams/{id}/members
PUT /api/teams/{id}
PUT /api/teams/{id}/code
GET /api/teams/{id}/activities
```

F-05. 팀 설정 - 코드 재발급

앞의 코드는 폐기. 이미 합류한 멤버는 그대로



상태 5 / 8

응답

`PUT 200`

구현 대상 파일

`team.html`

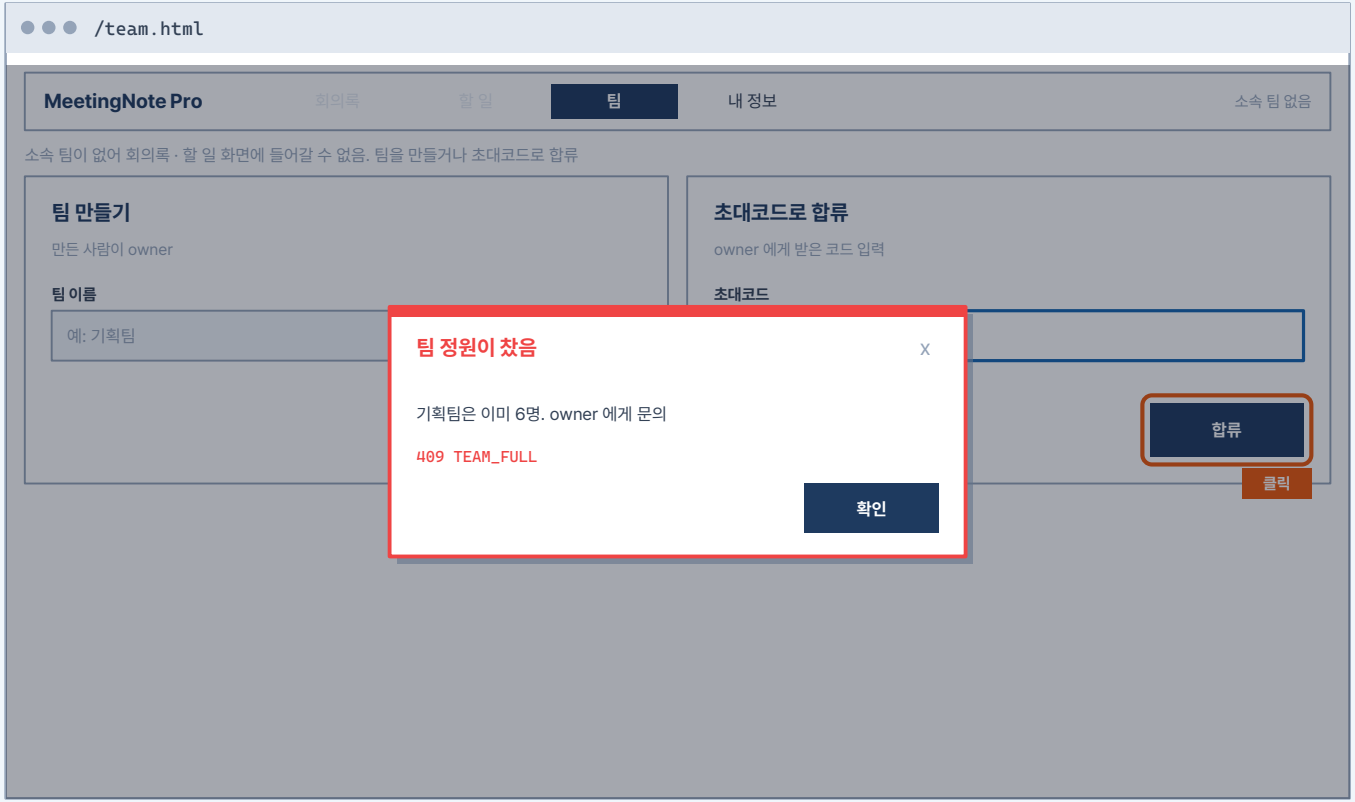
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/teams
GET /api/teams
POST /api/teams/join
GET /api/teams/{id}/members
PUT /api/teams/{id}
PUT /api/teams/{id}/code
GET /api/teams/{id}/activities
```

F-06. 팀 설정 - 정원 초과

팀당 6명은 ACME 의 Constraints 에서 온 값



상태 6 / 8

응답

409 TEAM_FULL

구현 대상 파일

`team.html`

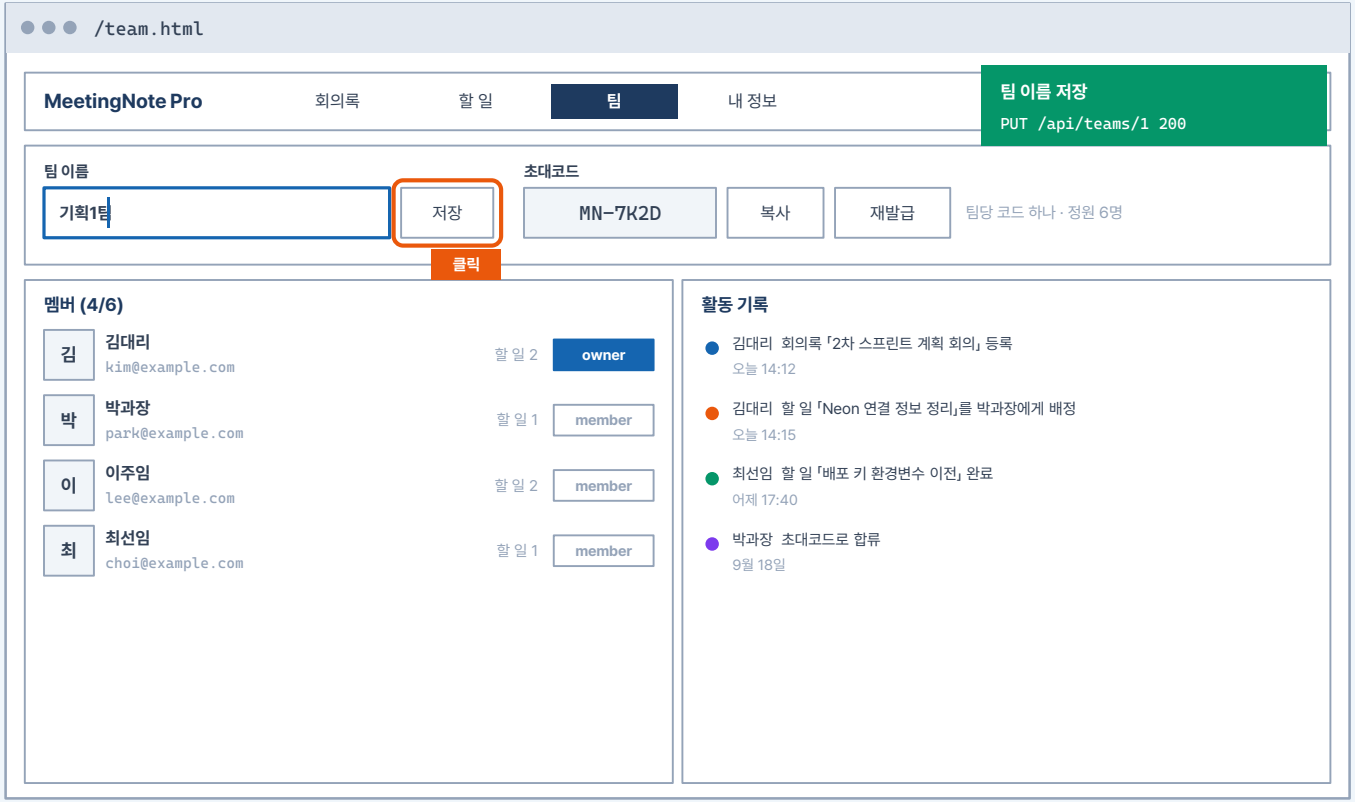
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/teams
GET /api/teams
POST /api/teams/join
GET /api/teams/{id}/members
PUT /api/teams/{id}
PUT /api/teams/{id}/code
GET /api/teams/{id}/activities
```

F-07. 팀 설정 - 팀 이름 저장

owner 만. member 가 부르면 403



상태 7 / 8

응답

`PUT /api/teams/{id} 200`

구현 대상 파일

`team.html`

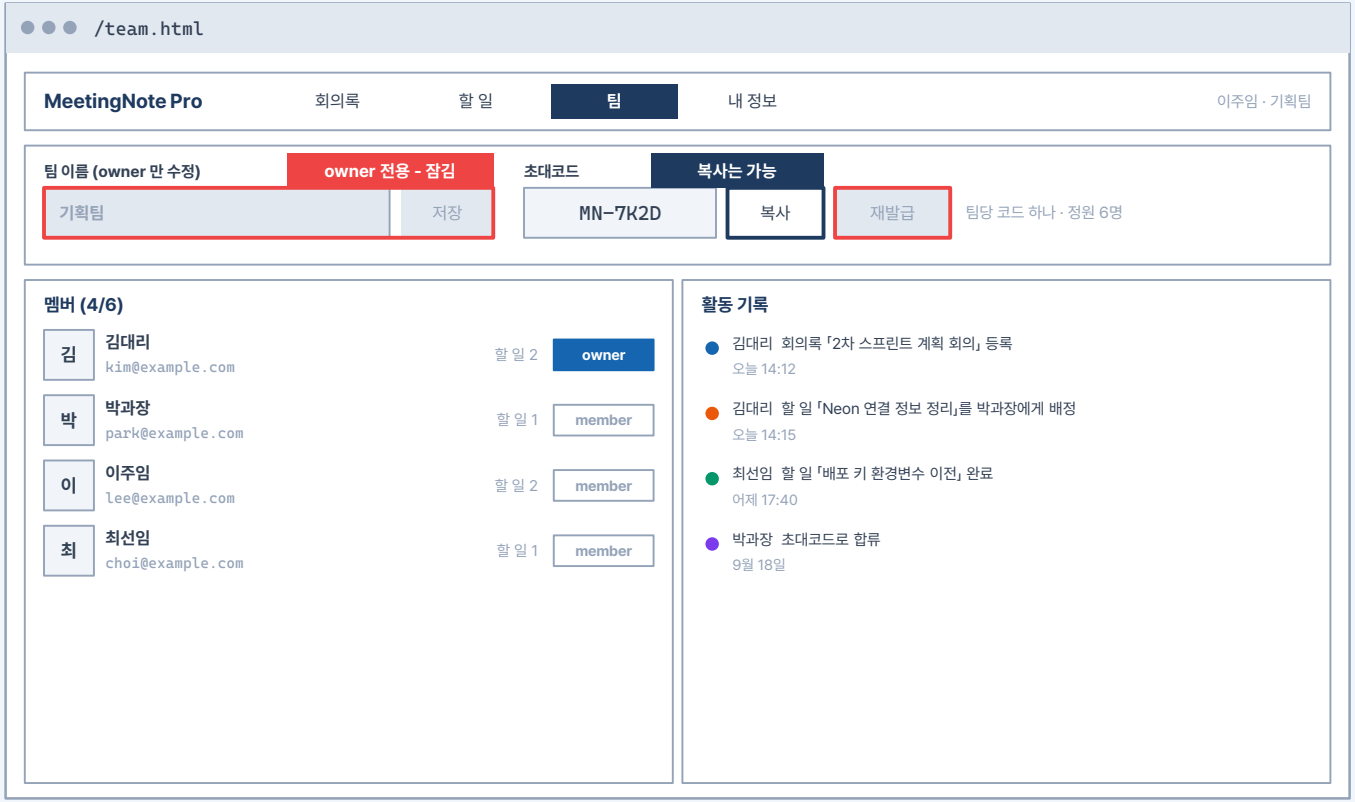
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/teams
GET /api/teams
POST /api/teams/join
GET /api/teams/{id}/members
PUT /api/teams/{id}
PUT /api/teams/{id}/code
GET /api/teams/{id}/activities
```

F-08. 팀 설정 - member 권한

팀 설정은 읽기만. owner 전용은 팀 이름 · 코드 재발급 둘. 코드 복사는 member 도 가능



상태 8 / 8

응답

403 OWNER_ONLY

구현 대상 파일

`team.html`

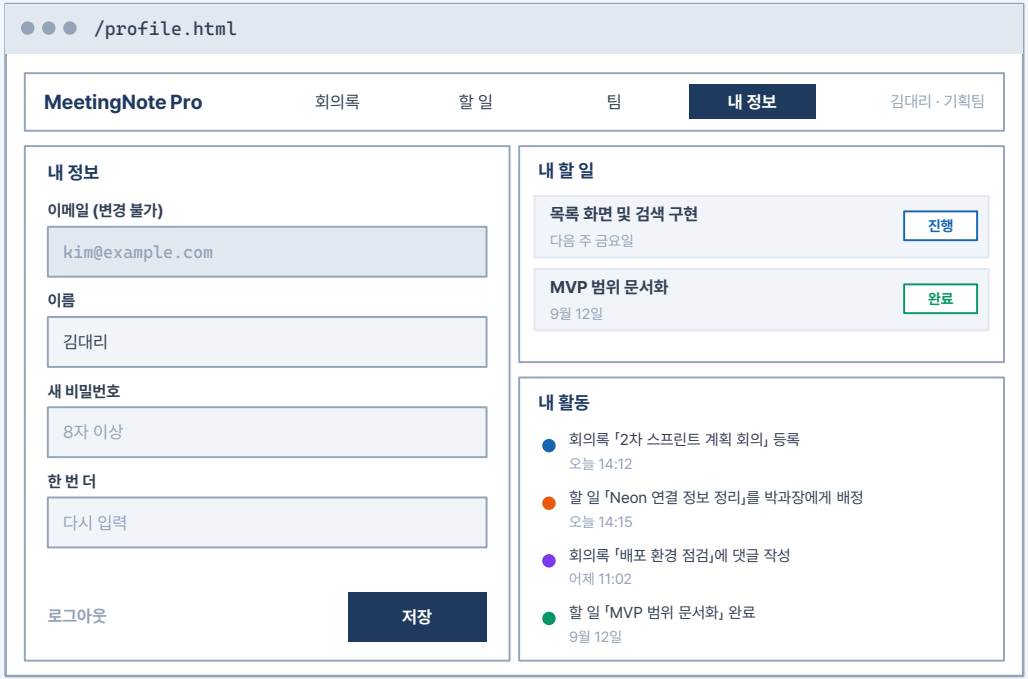
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
POST /api/teams
GET /api/teams
POST /api/teams/join
GET /api/teams/{id}/members
PUT /api/teams/{id}
PUT /api/teams/{id}/code
GET /api/teams/{id}/activities
```

J. 내 정보

이름과 비밀번호를 고치고, 내 할 일과 내 활동을 한 화면에서



상태 7종

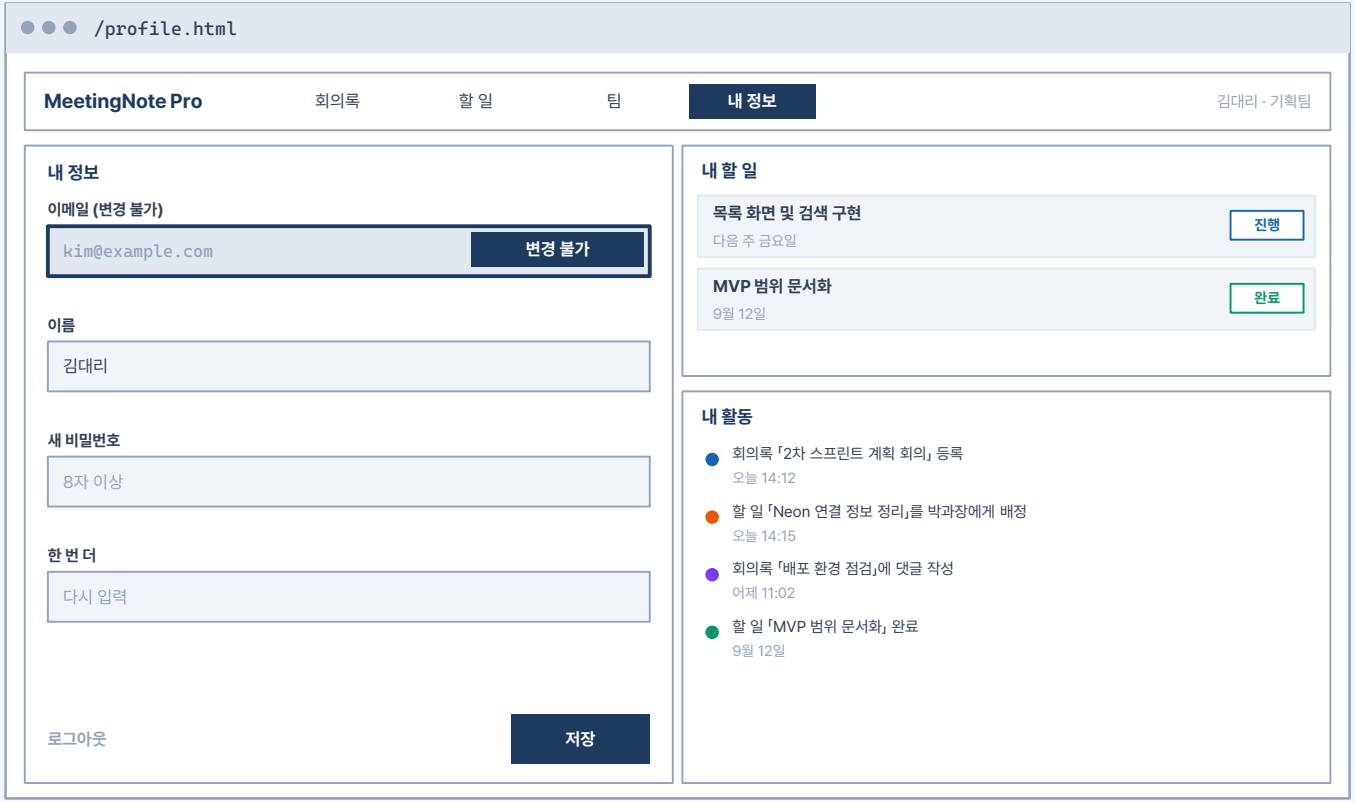
01	내 정보	`GET /api/auth/me 200`
02	이름 변경	`PUT /api/auth/me 200`
03	비밀번호 불일치	
04	약한 비밀번호	`400 PASSWORD_TOO_WEAK`
05	저장 중	요청 대기
06	저장 완료	`200`
07	활동 없음	`GET /api/me/activities 200 + 빈 배열`

쓰이는 엔드포인트 `GET /api/auth/me` · `PUT /api/auth/me` · `POST /api/auth/logout` · `GET /api/me/todos` · `GET /api/me/activities`

상태 7종: 정상 흐름뿐 아니라 빈 상태 · 오류 · 처리 중까지 화면으로 고정

J-01. 내 정보 - 내 정보

이메일은 고칠 수 없음



상태 1 / 7

응답

`GET /api/auth/me 200`

구현 대상 파일

`profile.html`

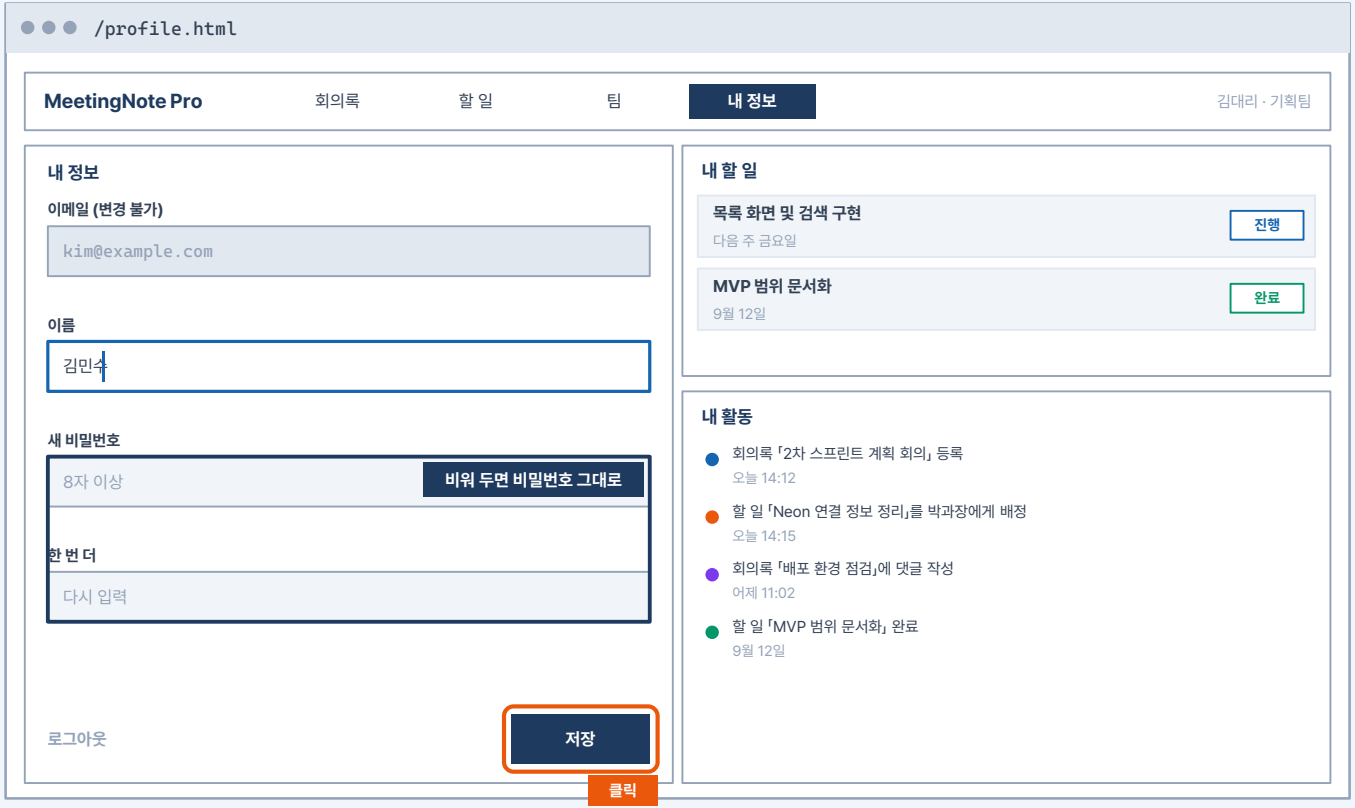
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/auth/me
PUT /api/auth/me
POST /api/auth/logout
GET /api/me/todos
GET /api/me/activities
```

J-02. 내 정보 - 이름 변경

이름만 보내면 비밀번호는 그대로



상태 2 / 7

응답

`PUT /api/auth/me 200`

구현 대상 파일

`profile.html`

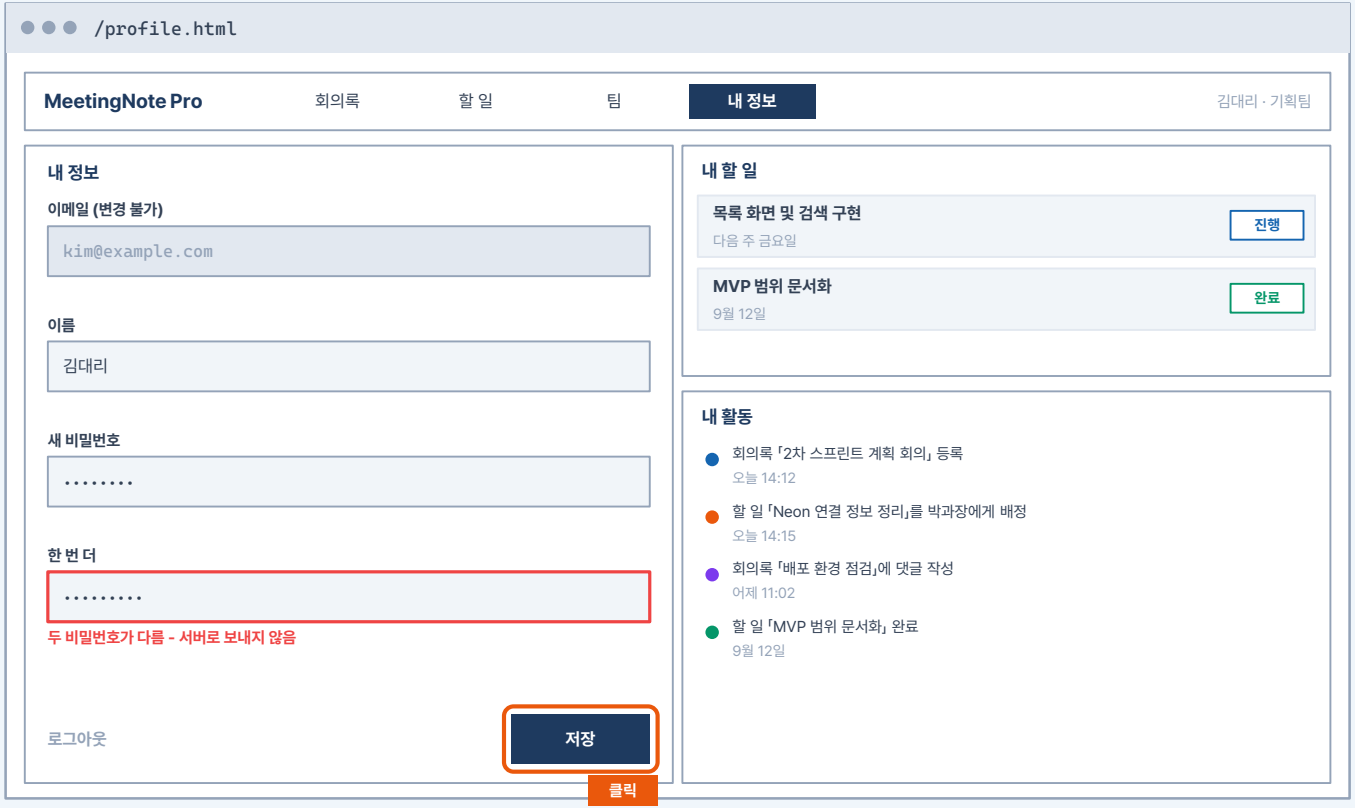
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/auth/me
PUT /api/auth/me
POST /api/auth/logout
GET /api/me/todos
GET /api/me/activities
```

J-03. 내 정보 - 비밀번호 불일치

화면에서 먼저 거름. 서버로 보내지 않음



상태 3 / 7

응답

–

구현 대상 파일

`profile.html`

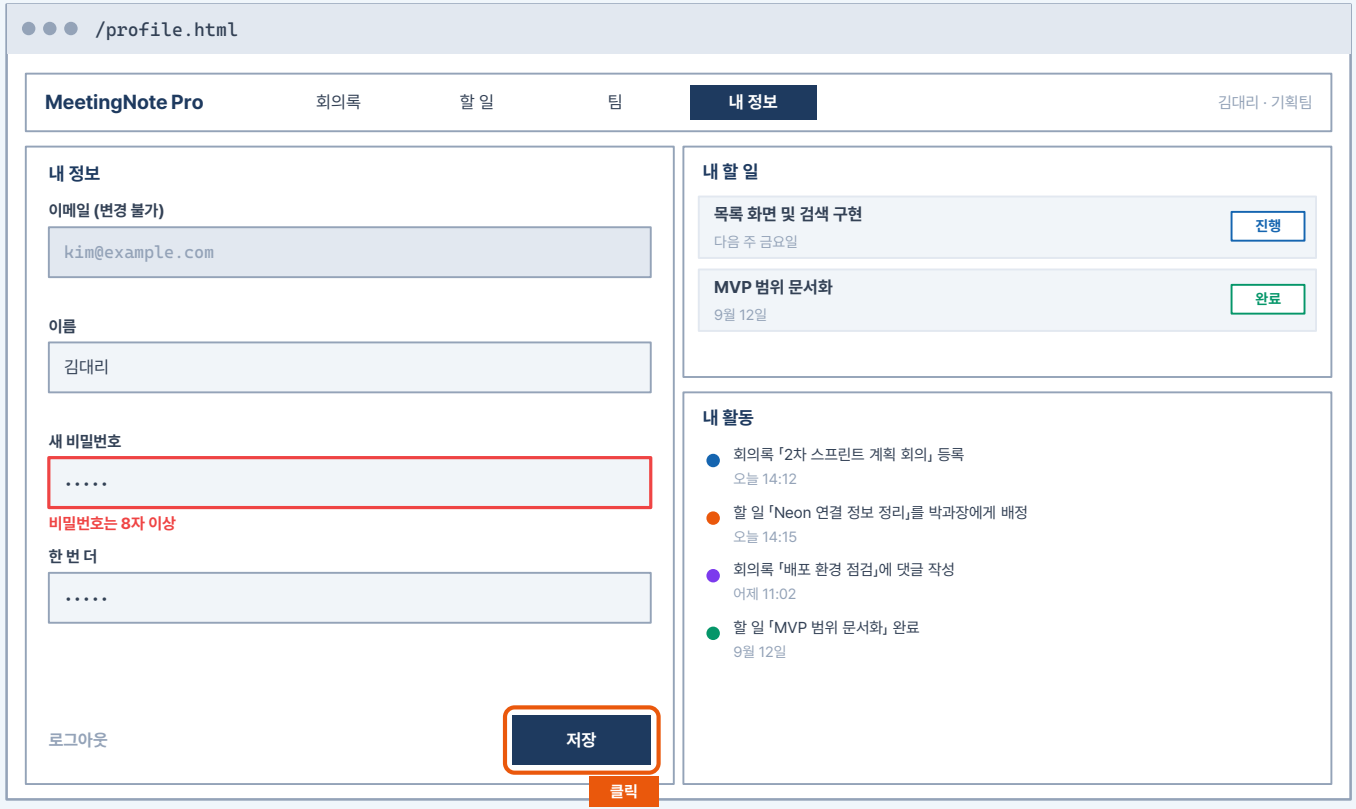
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/auth/me
PUT /api/auth/me
POST /api/auth/logout
GET /api/me/todos
GET /api/me/activities
```

J-04. 내 정보 - 약한 비밀번호

8자 미만



상태 4 / 7

응답

`400 PASSWORD_TOO_WEAK`

구현 대상 파일

`profile.html`

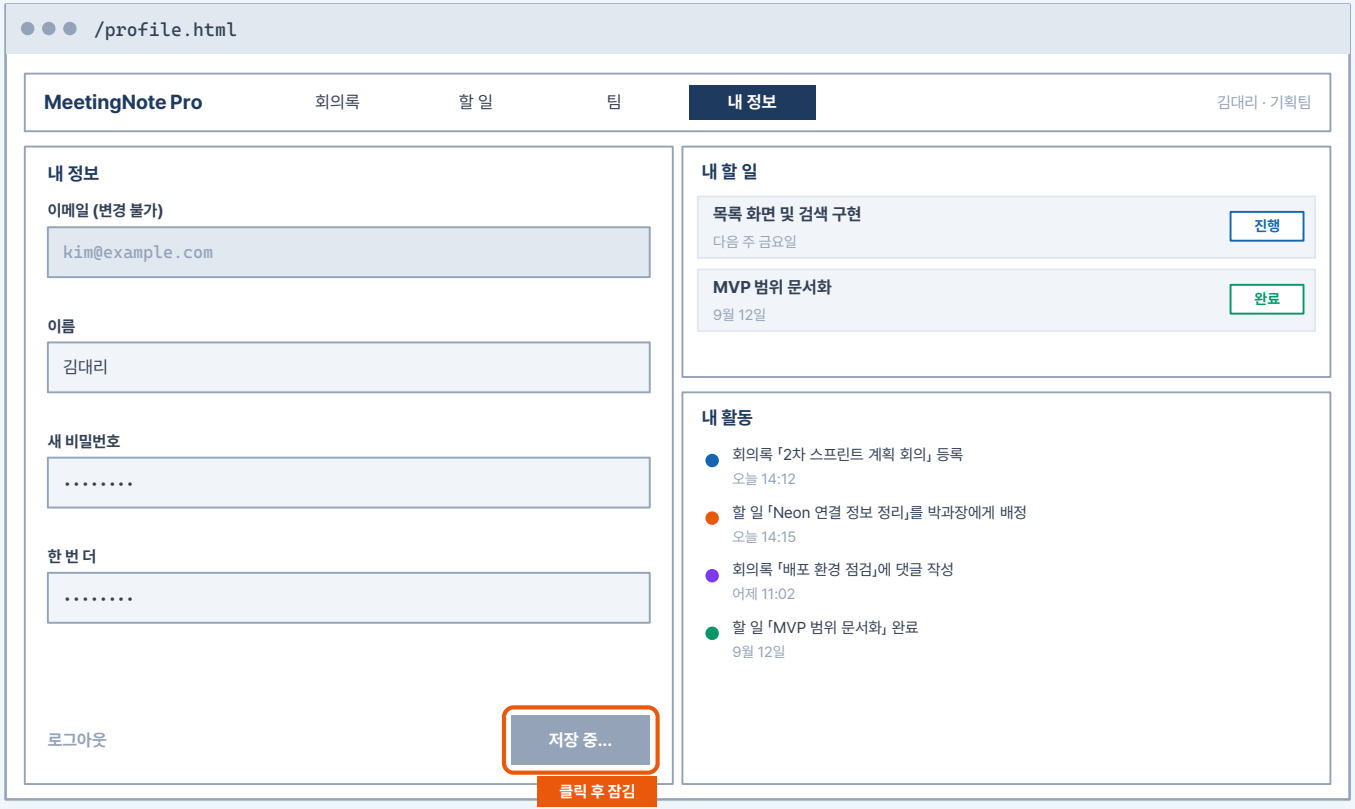
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/auth/me
PUT /api/auth/me
POST /api/auth/logout
GET /api/me/todos
GET /api/me/activities
```

J-05. 내 정보 - 저장 중

버튼을 잠가 2번 눌리지 않게



상태 5 / 7

응답

요청 대기

구현 대상 파일

`profile.html`

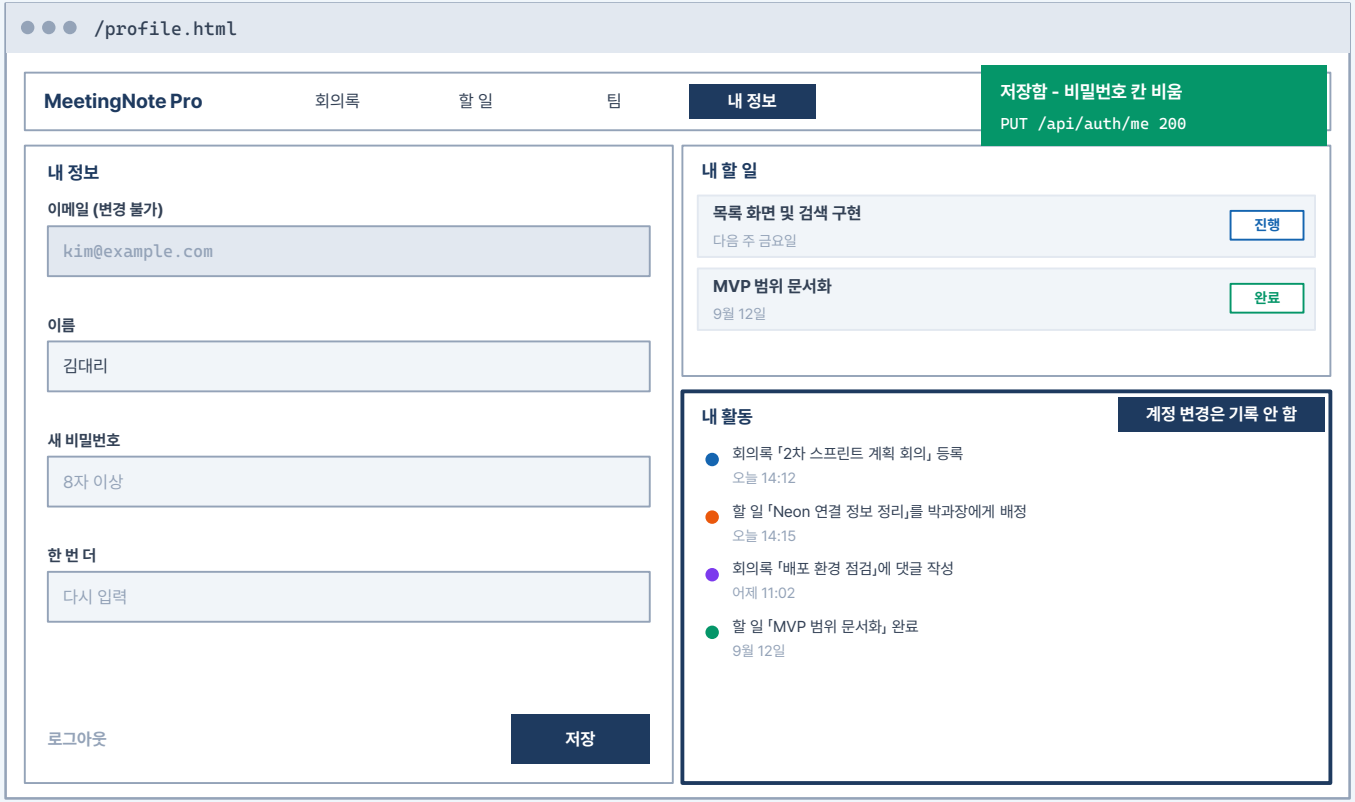
이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/auth/me
PUT /api/auth/me
POST /api/auth/logout
GET /api/me/todos
GET /api/me/activities
```

J-06. 내 정보 - 저장 완료

계정 변경은 팀 활동이 아니라 활동 기록에 남기지 않음



상태 6 / 7

응답

200

구현 대상 파일

`profile.html`

이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

쓰이는 엔드포인트

```
GET /api/auth/me
PUT /api/auth/me
POST /api/auth/logout
GET /api/me/todos
GET /api/me/activities
```

J-07. 내 정보 - 활동 없음

팀을 막 만든 owner. 팀 생성은 활동 kind 5종에 없어 0건 (합류는 member_join 이 남음)



상태 7 / 7

응답

`GET /api/me/activities 200 + 빈 배열`

구현 대상 파일

`profile.html`

이 상태를 퍼블리싱에서 그대로 구현. 색과 간격은 디자인 시스템

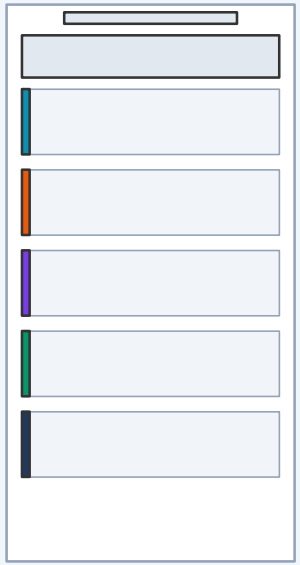
쓰이는 엔드포인트

```
GET /api/auth/me
PUT /api/auth/me
POST /api/auth/logout
GET /api/me/todos
GET /api/me/activities
```

N-01. 360px 반응형

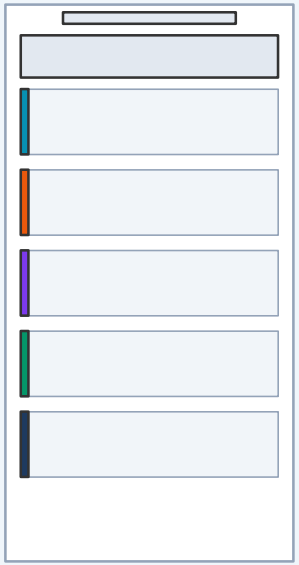
가장 좁은 화면에서 깨지지 않는지. 정의서 비기능 요구사항의 화면 폭 기준

회의록 목록



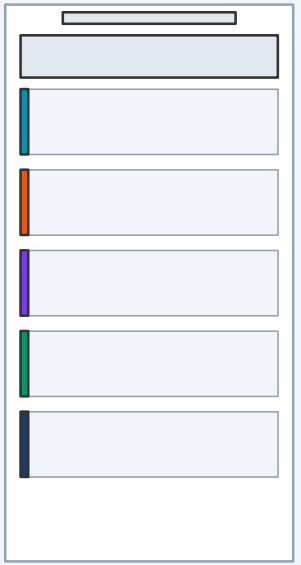
카드가 한 열로 접힘

회의록 상세



3단이 세로로 쌓임

할 일 칸반



칸이 세로로. 눌러서 칸 고르기

확인 기준: 가로 스크롤이 생기지 않고, 카드가 한 열로 접히며, 글자가 잘리지 않을 것

G-01. 컴포넌트 규격

화면마다 다시 정하지 않고 theme.js 의 이름을 가져다 사용

btnPrimary

h-11 · rounded-xl · 흰 글자

보기

btnGhost

h-11 · 테두리만

보기

input

h-11 · px-3.5 · rounded-xl

보기

card

rounded-xl · 좌측 6px 띠

panel

rounded-2xl · p-5

badge

px-2 · 11px

보기

구현 규칙: 새 클래스 조합을 화면에서 만들지 않음. 필요하면 theme.js 에 먼저 등록

I-01. 통합 매핑표 - 화면 x API x DB

화면별 API (기능 호출 규격) · DB (데이터베이스) 테이블 사용 범위

화면	엔드포인트	개수	건드리는 테이블
login.html	auth/signup · auth/login · teams (GET) · teams/join	4	users · teams · memberships
meetings.html	teams (GET) · teams/{id}/meetings (POST·GET) · upload	4	meetings · todos · users
detail.html	meetings/{id} (GET·PUT·DELETE) · todos/{id} (PUT) · meetings/{id}/comments (POST·GET) · comments/{id} (DELETE) · teams/{id}/members	8	meetings · todos · comments · users
todos.html	teams/{id}/todos (GET·POST) · todos/{id} (PUT·DELETE) · me/todos · teams/{id}/members	6	todos · meetings · users
team.html	teams (POST·GET·PUT) · teams/join · teams/{id}/members · teams/{id}/code · teams/{id}/activities	7	teams · memberships · users · activities
profile.html	auth/me (GET·PUT) · auth/logout · me/todos · me/activities	5	users · todos · activities

합계: 칸 합 34. 여러 화면이 같이 쓰는 경로를 한 번만 세면 고유 경로 27개 · 테이블 7개. 쓰이지 않는 경로 0개

H-01. 환경 분리 - 로컬과 배포

코드는 한 벌. DATABASE_URL 유무로 구분

로컬 개발

1. FastAPI + uvicorn --reload
2. SQLite 파일 (meetingnote.db)
3. frontend/ 를 StaticFiles 로 제공
4. http://127.0.0.1:8000
5. .env 에서 키를 읽음

배포

1. Vercel 한 프로젝트 · vercel.json {"framework": "fastapi"}
2. Neon Postgres (Vercel Storage) · 환경변수에서 키를 읽음
3. pyproject.toml [tool.vercel] entrypoint = "backend.main:app"
4. entrypoint 는 모듈:객체 형식. 파일 경로로 적으면 빌드 실패
5. .vercelignore - .agents/ · .claude/ · skills-lock.json · .env.local

SQLite 는 배포 환경에서 죽음 - 요청마다 새 환경이라 파일을 쓸 수 없음

코드에서 구분하는 지점: DATABASE_URL 이 있으면 Postgres, 없으면 SQLite. 한 줄 분기